

ods_pipeline Code Reference

Generated from C:\Users\Holdel\development\aviva ODS\ods_pipeline on 2026-05-21 15:11.

Included files: 36.

File Index

#	File	Lines
1	ods_pipeline/__init__.py	56
2	ods_pipeline/_db.py	47
3	ods_pipeline/config/__init__.py	20
4	ods_pipeline/config/validator.py	332
5	ods_pipeline/dlq.py	201
6	ods_pipeline/events.py	191
7	ods_pipeline/files.py	129
8	ods_pipeline/ingest/__init__.py	5
9	ods_pipeline/ingest/api_pull/__init__.py	41
10	ods_pipeline/ingest/api_pull/archive.py	183
11	ods_pipeline/ingest/api_pull/auth.py	80
12	ods_pipeline/ingest/api_pull/cursors.py	174
13	ods_pipeline/ingest/api_pull/linkage.py	108
14	ods_pipeline/ingest/api_pull/poller.py	156
15	ods_pipeline/ingest/api_pull/watermark.py	229
16	ods_pipeline/ingest/api_pull_kafka/__init__.py	34
17	ods_pipeline/ingest/api_pull_kafka/loop.py	259
18	ods_pipeline/ingest/api_pull_kafka/runner.py	352
19	ods_pipeline/lineage.py	46
20	ods_pipeline/messages.py	308
21	ods_pipeline/metadata.py	267
22	ods_pipeline/models.py	131
23	ods_pipeline/offsets.py	250
24	ods_pipeline/ops/__init__.py	1
25	ods_pipeline/ops/__main__.py	34
26	ods_pipeline/ops/dlq.py	215
27	ods_pipeline/ops/runs.py	206
28	ods_pipeline/patterns/__init__.py	18
29	ods_pipeline/patterns/api_pull.py	40
30	ods_pipeline/patterns/base.py	64
31	ods_pipeline/patterns/event.py	29
32	ods_pipeline/patterns/file.py	51
33	ods_pipeline/publish.py	90
34	ods_pipeline/reconciliation.py	378

#	File	Lines
35	ods_pipeline/runs.py	238
36	ods_pipeline/stages.py	437

1. ods_pipeline/ __init__.py

56 lines

```
1 | """ODS Pipeline control-plane client.
2 |
3 | Usage
4 | ----
5 | ::
6 |
7 |     import ods_pipeline
8 |
9 |     with ods_pipeline.connect(dsn) as conn:
10 |         file_id = ods_pipeline.files.upsert(conn, domain="insurance", ...)
11 |         ods_pipeline.runs.start(conn, run_id=run_id, ...)
12 |         ods_pipeline.stages.write(conn, run_id=run_id, stage=ods_pipeline.Stage.RAW_READ, ...)
13 |         ods_pipeline.lineage.write_edge(conn, child_run_id=run_id, ...)
14 |         ods_pipeline.runs.finish(conn, run_id=run_id, status="succeeded")
15 |
16 |         ods_pipeline.events.produce("run_succeeded", run_id=run_id, ...)
17 | """
18 |
19 | from ods_pipeline import (
20 |     dlq,
21 |     events,
22 |     files,
23 |     lineage,
24 |     messages,
25 |     metadata,
26 |     offsets,
27 |     publish,
28 |     reconciliation,
29 |     runs,
30 |     stages,
31 | )
32 | from ods_pipeline._db import build_dsn, connect
33 | from ods_pipeline.models import TERMINAL_STATUSES, RunStatus, Stage, StageEvent
34 |
35 | __all__ = [
36 |     # connection helpers
37 |     "build_dsn",
38 |     "connect",
39 |     # sub-modules
40 |     "files",
41 |     "runs",
42 |     "stages",
43 |     "lineage",
44 |     "reconciliation",
45 |     "events",
46 |     "metadata",
47 |     "offsets",
48 |     "messages",
49 |     "dlq",
50 |     "publish",
51 |     # constants
52 |     "Stage",
53 |     "StageEvent",
54 |     "RunStatus",
55 |     "TERMINAL_STATUSES",
56 | ]
```

2. ods_pipeline/_db.py

47 lines

```
1 | """Connection helpers for the ODS pipeline control-plane client."""
2 | from __future__ import annotations
3 |
4 | import os
5 |
6 | import psycopg2
7 |
8 |
9 | def build_dsn(dsn: str | None = None) -> str:
10 |     """Return a libpq DSN string from explicit arg or environment variables.
11 |
12 |     Precedence:
13 |         1. ``dsn`` argument
14 |         2. ``PIPELINE_PG_DSN`` env var
15 |         3. ``PG_DSN`` env var
16 |         4. Individual ``POSTGRES_*`` env vars
17 |     """
18 |     if dsn:
19 |         return dsn
20 |     for key in ("PIPELINE_PG_DSN", "PG_DSN"):
21 |         val = os.environ.get(key)
22 |         if val:
23 |             return val
24 |     host = os.environ.get("POSTGRES_HOST")
25 |     if not host:
26 |         raise ValueError(
27 |             "No Postgres DSN configured. "
28 |             "Set PIPELINE_PG_DSN, PG_DSN, or POSTGRES_HOST."
29 |         )
30 |     return (
31 |         f"host={host} "
32 |         f"port={os.environ.get('POSTGRES_PORT', '5432')} "
33 |         f"dbname={os.environ.get('POSTGRES_DB', 'ods_dev')} "
34 |         f"user={os.environ.get('POSTGRES_USER', 'ods')} "
35 |         f"password={os.environ.get('POSTGRES_PASSWORD', 'ods')}"
36 |     )
37 |
38 |
39 | def connect(dsn: str | None = None) -> psycopg2.extensions.connection:
40 |     """Open and return a psycopg2 connection.
41 |
42 |     Use as a context manager::
43 |
44 |         with ods_pipeline.connect(dsn) as conn:
45 |             ods_pipeline.stages.write(conn, ...)
46 |     """
47 |     return psycopg2.connect(build_dsn(dsn))
```

3. ods_pipeline/config/__init__.py

20 lines

```
1 | """Dataset config validation helpers.
2 |
3 | Centralises pre-INSERT checks the YAML loader runs against
4 | ``pipeline.dataset_config`` rows so impossible combinations (e.g.
5 | ``delivery=direct_postgres`` paired with a ``target_topic``, or
6 | ``write_mode=upsert`` with empty ``key_fields``) fail at sync time
7 | with an operator-readable message instead of silently producing
8 | nonsense at run time.
9 | """
10 | from ods_pipeline.config.validator import (
11 |     DatasetConfigError,
12 |     check_no_filename_pattern_overlap,
13 |     validate_dataset_config,
14 | )
15 |
16 | __all__ = [
17 |     "DatasetConfigError",
18 |     "check_no_filename_pattern_overlap",
19 |     "validate_dataset_config",
20 | ]
```

4. ods_pipeline/config/validator.py

332 lines

```
1 | """Pre-sync validation of dataset_config YAML.
2 |
3 | Catches impossible combinations BEFORE they reach
4 | ``pipeline.dataset_config`` so:
5 |
6 | - operators see the error in the dag_config_sync output, not in a
7 |   failed run six hours later;
8 | - ``yaml_loader.sync_to_db`` does not need to repeat the rules
9 |   inline – the validator module owns the contract;
10 | - new combos can be added by extending one function with one rule
11 |   + one unit test, no SQL changes.
12 |
13 | Two functions:
14 |
15 | - ``validate_dataset_config(cfg)`` – pure-Python single-row checks.
16 |   No I/O. Reusable from a future ``odscli config validate``.
17 | - ``check_no_filename_pattern_overlap(cfg, peers)`` – given a row
18 |   + the active peers (a list of dicts pulled from
19 |   ``pipeline.dataset_config``), reject regex collisions across
20 |   different ``(domain, dataset)`` tuples that would cause
21 |   ``dag_drop_to_raw`` to register the same physical file under two
22 |   delivery routes.
23 | """
24 | from __future__ import annotations
25 |
26 | import re
27 | from typing import Any, Iterable, Mapping
28 |
29 |
30 | class DatasetConfigError(ValueError):
31 |     """Raised when a dataset_config row violates a structural invariant."""
32 |
33 |
34 | _ALLOWED_DELIVERIES = {"file_pipeline", "direct_kafka", "direct_postgres"}
35 | _ALLOWED_SOURCE_TYPES = {"s3_batch", "api_pull", "cdc", "event"}
36 | _ALLOWED_WRITE_MODES = {"upsert", "append", "replace"}
37 | _ALLOWED_DQ_SECTIONS = {"hard_blocks", "soft_warns"}
38 |
39 |
40 | def _qualified(cfg: Mapping[str, Any]) -> str:
41 |     return f"{cfg.get('domain', '<no-domain>')}/{cfg.get('dataset', '<no-dataset>')}"
42 |
43 |
44 | def _validate_dq_rules_shape(name: str, cfg: Mapping[str, Any]) -> None:
45 |     dq_rules = cfg.get("dq_rules") or {}
46 |     if not isinstance(dq_rules, Mapping):
47 |         raise DatasetConfigError(f"{name}: dq_rules must be a mapping")
48 |
49 |     legacy_sections = {"hard", "soft"} & set(dq_rules)
50 |     if legacy_sections:
51 |         raise DatasetConfigError(
52 |             f"{name}: dq_rules uses legacy section(s) "
53 |             f"{sorted(legacy_sections)}; use hard_blocks/soft_warns"
54 |         )
55 |
56 |     unknown_sections = set(dq_rules) - _ALLOWED_DQ_SECTIONS
57 |     if unknown_sections:
58 |         raise DatasetConfigError(
59 |             f"{name}: dq_rules has unknown section(s) "
60 |             f"{sorted(unknown_sections)}; expected hard_blocks/soft_warns"
61 |         )
62 |
63 |     for section in _ALLOWED_DQ_SECTIONS:
64 |         rules = dq_rules.get(section, [])
65 |         if not isinstance(rules, list):
66 |             raise DatasetConfigError(f"{name}: dq_rules.{section} must be a list")
67 |         for rule in rules:
68 |             if not isinstance(rule, Mapping):
69 |                 raise DatasetConfigError(
70 |                     f"{name}: dq_rules.{section} entries must be mappings"
71 |                 )
72 |             if "column" in rule or "min_pct" in rule:
73 |                 raise DatasetConfigError(
74 |                     f"{name}: dq_rules.{section} uses legacy keys "
75 |                     "'column'/'min_pct'; use runtime keys 'field' or "
76 |                     "'fields'/'threshold'"
77 |                 )
78 |
79 |
80 | def validate_dataset_config(cfg: Mapping[str, Any]) -> None:
81 |     """Validate ``cfg`` (parsed YAML) against the dataset_config contract.
82 |
83 |     Raises :class:`DatasetConfigError` with a single, operator-readable
84 |     message identifying the dataset and the broken rule. Does not log
85 |     or print; callers decide how to surface failures.
86 |     """
87 |     if not isinstance(cfg, Mapping):
88 |         raise DatasetConfigError(
89 |             f"dataset_config must be a mapping, got {type(cfg).__name__}"
90 |         )
91 |
92 |     domain = cfg.get("domain")
93 |     dataset = cfg.get("dataset")
94 |     if not domain or not dataset:
95 |         raise DatasetConfigError(
96 |             "dataset_config requires non-empty 'domain' and 'dataset'"
97 |         )
```

```

97 |         )
98 |
99 |     name = _qualified(cfg)
100 |     source_type = cfg.get("source_type", "s3_batch")
101 |     delivery = cfg.get("delivery", "file_pipeline")
102 |     is_canonical = bool(cfg.get("is_canonical", True))
103 |     write_mode = cfg.get("write_mode", "upsert")
104 |     key_fields = cfg.get("key_fields") or []
105 |     target_topic = cfg.get("target_topic")
106 |     canonical_topic = cfg.get("canonical_topic")
107 |     transform_yaml_path = cfg.get("transform_yaml_path")
108 |     filename_pattern = cfg.get("filename_pattern")
109 |
110 |     # Enum domains.
111 |     if source_type not in _ALLOWED_SOURCE_TYPES:
112 |         raise DatasetConfigError(
113 |             f"{name}: unknown source_type={source_type!r}; "
114 |             f"expected one of {sorted(_ALLOWED_SOURCE_TYPES)}"
115 |         )
116 |     if delivery not in _ALLOWED_DELIVERIES:
117 |         raise DatasetConfigError(
118 |             f"{name}: unknown delivery={delivery!r}; "
119 |             f"expected one of {sorted(_ALLOWED_DELIVERIES)}"
120 |         )
121 |     if write_mode not in _ALLOWED_WRITE_MODES:
122 |         raise DatasetConfigError(
123 |             f"{name}: unknown write_mode={write_mode!r}; "
124 |             f"expected one of {sorted(_ALLOWED_WRITE_MODES)}"
125 |         )
126 |
127 |     # Source-type / file-pattern coherence.
128 |     if source_type == "s3_batch" and not filename_pattern:
129 |         raise DatasetConfigError(
130 |             f"{name}: source_type='s3_batch' requires filename_pattern"
131 |         )
132 |     if source_type != "s3_batch" and filename_pattern:
133 |         raise DatasetConfigError(
134 |             f"{name}: filename_pattern only applies to source_type='s3_batch'"
135 |         )
136 |
137 |     # Delivery-specific topic requirements.
138 |     if delivery in {"file_pipeline", "direct_kafka"} and not target_topic:
139 |         raise DatasetConfigError(
140 |             f"{name}: delivery={delivery!r} requires target_topic"
141 |         )
142 |     if delivery == "direct_postgres" and target_topic:
143 |         raise DatasetConfigError(
144 |             f"{name}: delivery='direct_postgres' must not set target_topic "
145 |             f"(no Kafka leg). Got target_topic={target_topic!r}"
146 |         )
147 |     if delivery == "direct_postgres" and canonical_topic:
148 |         raise DatasetConfigError(
149 |             f"{name}: delivery='direct_postgres' must not set canonical_topic"
150 |         )
151 |
152 |     # Canonicalize → transform_yaml_path requirement, except direct_postgres
153 |     # which may run inline canonicalize OR pre-canonicalized curated.
154 |     if not is_canonical and not transform_yaml_path:
155 |         raise DatasetConfigError(
156 |             f"{name}: is_canonical=False requires transform_yaml_path"
157 |         )
158 |
159 |     # Write-mode / key-field coherence.
160 |     if write_mode == "upsert" and not key_fields:
161 |         raise DatasetConfigError(
162 |             f"{name}: write_mode='upsert' requires non-empty key_fields"
163 |         )
164 |     if write_mode == "replace" and not key_fields:
165 |         raise DatasetConfigError(
166 |             f"{name}: write_mode='replace' requires non-empty key_fields "
167 |             f"(slot identity)"
168 |         )
169 |
170 |     # api_pull source block – runtime requirements that the runner / poller
171 |     # would otherwise discover only when called.
172 |     # Probe-string overlap with self is allowed (one filename matches
173 |     # only the dataset that owns it); cross-dataset overlap is checked
174 |     # by ``check_no_filename_pattern_overlap`` when peers are known.
175 |
176 |     _validate_dq_rules_shape(name, cfg)
177 |
178 |     if source_type == "api_pull":
179 |         source = cfg.get("source") or {}
180 |         if not isinstance(source, Mapping):
181 |             raise DatasetConfigError(
182 |                 f"{name}: source_type='api_pull' requires a 'source' mapping"
183 |             )
184 |         if not source.get("url"):
185 |             raise DatasetConfigError(
186 |                 f"{name}: source_type='api_pull' requires source.url"
187 |             )
188 |         cursor = source.get("cursor") or {}
189 |         if not cursor.get("style"):
190 |             raise DatasetConfigError(
191 |                 f"{name}: source_type='api_pull' requires source.cursor.style"
192 |             )
193 |         auth = source.get("auth") or {}
194 |         auth_type = auth.get("type", "none")
195 |         if auth_type not in {"none", "bearer", "basic", "mtls"}:
196 |             raise DatasetConfigError(

```

```

197 |         f"{name}: unknown source.auth.type={auth_type!r}"
198 |     )
199 |     if auth_type == "bearer" and not auth.get("secret_ref"):
200 |         raise DatasetConfigError(
201 |             f"{name}: source.auth.type='bearer' requires secret_ref "
202 |             f"(env var name; never the token itself)"
203 |         )
204 |     # Defence-in-depth: never persist literal token strings even if a
205 |     # YAML accidentally inlines one.
206 |     for forbidden in ("token", "password", "client_secret", "api_key"):
207 |         if forbidden in auth:
208 |             raise DatasetConfigError(
209 |                 f"{name}: source.auth must not contain {forbidden!r} "
210 |                 f"directly; use secret_ref instead"
211 |             )
212 |
213 |
214 | # -----
215 | # Cross-dataset overlap (called by yaml_loader before INSERT)
216 | # -----
217 |
218 |
219 | # A handful of probe filenames covering the formats this codebase ingests.
220 | # The overlap check matches each peer's ``filename_pattern`` against probes
221 | # generated from the candidate's pattern; if any probe matches BOTH, the
222 | # regexes share input and `dag_drop_to_raw` would register the same SFTP
223 | # file under two distinct ``(domain, dataset)`` rows.
224 | _PROBE_FILENAMES: tuple[str, ...] = (
225 |     # Common shapes produced by the patterns in patterns/insurance/.
226 |     "policies_20260501.csv",
227 |     "risk_20260501.csv",
228 |     "events_20260501.csv",
229 |     "country_codes_20260501.csv",
230 |     "claims_20260501.csv",
231 | )
232 |
233 |
234 | def _generate_probes(pattern: str) -> list[str]:
235 |     """Synthesise sample filenames a regex would accept.
236 |
237 |     Pure heuristic – substitutes named groups with sensible defaults
238 |     and returns the literal-stripped form. Used only for cross-dataset
239 |     overlap detection so a precise regex parser isn't required.
240 |     """
241 |     if not pattern:
242 |         return []
243 |     probes: list[str] = []
244 |     # Replace named-group YYYYMMDD captures with a known date.
245 |     candidate = re.sub(
246 |         r"(\?P<bd>[^\?>]+)\)",
247 |         "20260501",
248 |         pattern,
249 |     )
250 |     # Replace any other named group with a token.
251 |     candidate = re.sub(r"(\?P<[^\?>]+>[^\?>]+)\)", "x", candidate)
252 |     # Strip anchors + escapes.
253 |     candidate = candidate.replace("^", "").replace("$", "").replace("\\.", ".")
254 |     candidate = candidate.replace("\\d{8}", "20260501")
255 |     candidate = candidate.replace(r"d{8}", "20260501")
256 |     probes.append(candidate)
257 |     probes.extend(_PROBE_FILENAMES)
258 |     return probes
259 |
260 |
261 | def check_no_filename_pattern_overlap(
262 |     cfg: Mapping[str, Any],
263 |     peers: Iterable[Mapping[str, Any]],
264 | ) -> None:
265 |     """Reject the candidate if its ``filename_pattern`` matches a sample
266 |     that another active dataset's pattern ALSO matches (excluding
267 |     same ``(domain, dataset)``).
268 |
269 |     Why: ``dag_drop_to_raw`` walks the SFTP listing and, for each file,
270 |     iterates every active ``s3_batch`` dataset's regex. Two regexes
271 |     that accept the same filename → same physical byte stream
272 |     registered as two distinct ``file_catalogue`` rows under two
273 |     distinct deliveries. T2 recon then double-counts and the file
274 |     fans into both downstream DAGs. The protection is regex
275 |     disjointness across active s3_batch datasets.
276 |
277 |     Heuristic only: synthesises probe filenames from each pattern and
278 |     runs every other pattern against them. False positives possible
279 |     on heavily-anchored regexes, false negatives possible on patterns
280 |     that accept only filenames not in the probe set. For production-
281 |     grade isolation, use distinct SFTP folders per delivery.
282 |     """
283 |     if cfg.get("source_type", "s3_batch") != "s3_batch":
284 |         return
285 |     pattern = cfg.get("filename_pattern")
286 |     if not pattern:
287 |         return
288 |
289 |     name_self = (cfg.get("domain"), cfg.get("dataset"))
290 |     self_delivery = cfg.get("delivery", "file_pipeline")
291 |     self_probes = _generate_probes(pattern)
292 |     if not self_probes:
293 |         return
294 |     self_re = re.compile(pattern)
295 |
296 |     for peer in peers:

```



```

297 |         if peer.get("source_type", "s3_batch") != "s3_batch":
298 |             continue
299 |         peer_pattern = peer.get("filename_pattern")
300 |         if not peer_pattern:
301 |             continue
302 |         if (peer.get("domain"), peer.get("dataset")) == name_self:
303 |             continue
304 |         # Two datasets sharing the SAME delivery route are an accepted
305 |         # dual-write pattern: e.g. policies (current) + policies_history
306 |         # both read the same file and write to different downstream
307 |         # tables via different Kafka topics. The reviewer concern is
308 |         # about CROSS-route collisions (file_pipeline + direct_postgres)
309 |         # where the file would be ingested through two divergent
310 |         # pipelines. Skip same-delivery peers.
311 |         peer_delivery = peer.get("delivery", "file_pipeline")
312 |         if peer_delivery == self_delivery:
313 |             continue
314 |         try:
315 |             peer_re = re.compile(peer_pattern)
316 |         except re.error:
317 |             continue
318 |         # Generate probes from BOTH patterns so we don't miss collisions
319 |         # in either direction.
320 |         probes = list(set(self_probes + _generate_probes(peer_pattern)))
321 |         for probe in probes:
322 |             if self_re.match(probe) and peer_re.match(probe):
323 |                 raise DatasetConfigError(
324 |                     f"{name_self[0]}/{name_self[1]}: filename_pattern "
325 |                     f"{pattern!r} overlaps with peer "
326 |                     f"{peer.get('domain')}/{peer.get('dataset')!r} "
327 |                     f"pattern {peer_pattern!r} (both accept {probe!r}). "
328 |                     f"dag_drop_to_raw would register the same SFTP file "
329 |                     f"under two distinct deliveries. Disambiguate the "
330 |                     f"regexes (anchors, separator chars) or split SFTP "
331 |                     f"folders per dataset."
332 |                 )

```

5. ods_pipeline/dlq.py

201 lines

```
1 | """DLQ helpers shared by file, canonicalize, and message/API flows."""
2 | from __future__ import annotations
3 |
4 | import json
5 | import logging
6 | import time
7 | from collections.abc import Mapping
8 | from typing import Any
9 | from urllib.parse import urlparse
10 |
11 | from ods_pipeline import metadata
12 |
13 | _log = logging.getLogger(__name__)
14 |
15 |
16 | def s3_prefix(
17 |     *,
18 |     env: str,
19 |     domain: str,
20 |     dataset: str,
21 |     stage: str,
22 |     run_id: str,
23 |     business_date: str | None = None,
24 | ) -> str:
25 |     """Return the standard S3 prefix for failed records/events."""
26 |     date_part = business_date or "unknown"
27 |     return (
28 |         f"s3://ods-dlq-{env}/{domain}/{dataset}/{stage}/"
29 |         f"date={date_part}/run_id={run_id}/"
30 |     )
31 |
32 |
33 | def envelope(
34 |     *,
35 |     payload: Mapping[str, Any],
36 |     run_id: str,
37 |     domain: str,
38 |     dataset: str,
39 |     source_application: str,
40 |     error_type: str,
41 |     error_message: str,
42 |     stage: str,
43 |     source_metadata: Mapping[str, Any] | None = None,
44 |     source_message_id: str | None = None,
45 |     source_event_id: str | None = None,
46 |     source_request_id: str | None = None,
47 |     source_batch_id: str | None = None,
48 |     file_id: str | None = None,
49 |     business_date: str | None = None,
50 |     replayable: bool = True,
51 | ) -> dict[str, Any]:
52 |     """Build a JSON/JSONL-friendly DLQ envelope."""
53 |     source_metadata = dict(source_metadata or {})
54 |     correlation = {
55 |         "_ods_source_message_id": source_message_id or source_metadata.get("_ods_source_message_id"),
56 |         "_ods_source_event_id": source_event_id or source_metadata.get("_ods_source_event_id"),
57 |         "_ods_source_request_id": source_request_id or source_metadata.get("_ods_source_request_id"),
58 |         "_ods_source_batch_id": source_batch_id or source_metadata.get("_ods_source_batch_id"),
59 |     }
60 |     result = {
61 |         **{k: v for k, v in correlation.items() if v},
62 |         "_ods_file_id": file_id or source_metadata.get("_ods_file_id"),
63 |         "_ods_run_id": run_id,
64 |         "_ods_domain": domain,
65 |         "_ods_dataset": dataset,
66 |         "_ods_business_date": business_date or source_metadata.get("_ods_business_date"),
67 |         "_ods_source_application": source_application,
68 |         "_ods_failed_stage": stage,
69 |         "_ods_error_type": error_type,
70 |         "_ods_error_message": error_message,
71 |         "_ods_replayable": replayable,
72 |         "payload": dict(payload),
73 |     }
74 |     if not result.get("_ods_file_id"):
75 |         metadata.require_message_correlation(result, context="DLQ envelope")
76 |     metadata.require_fields(
77 |         result,
78 |         (
79 |             "_ods_run_id",
80 |             "_ods_domain",
81 |             "_ods_dataset",
82 |             "_ods_source_application",
83 |             "_ods_failed_stage",
84 |             "_ods_error_type",
85 |             "_ods_error_message",
86 |             "payload",
87 |         ),
88 |         context="DLQ envelope",
89 |     )
90 |     return {key: value for key, value in result.items() if value is not None}
91 |
92 |
93 | class DlqWriter:
94 |     """Persist DLQ envelopes to S3 with bounded exponential-backoff retry.
95 |
96 |     The writer is intentionally small: callers (Glue jobs, the message/API
```

```

97 | service, replay tooling) hand it a fully-formed envelope built via
98 | :func:`envelope` and a ``stage`` (matching the prefix layout from
99 | :func:`s3_prefix`). It computes the canonical key, JSON-encodes, and
100 | PUTs to S3, retrying transient errors with exp backoff (1s, 2s, 4s, 8s,
101 | 16s by default; configurable). Permanent errors raise immediately.
102 |
103 | The S3 key is::
104 |
105 |     s3://<dlq-bucket>/<domain>/<dataset>/<stage>/date=<bd>/run_id=<rid>/<attempt>.json
106 |
107 | where ``attempt`` is the per-write counter the caller supplies, so multiple
108 | failed records from the same run land at distinct keys.
109 | """
110 |
111 | DEFAULT_BACKOFF = (1.0, 2.0, 4.0, 8.0, 16.0)
112 |
113 | def __init__(
114 |     self,
115 |     s3_client,
116 |     *,
117 |     env: str = "local",
118 |     backoff: tuple[float, ...] = DEFAULT_BACKOFF,
119 |     sleep=time.sleep,
120 | ) -> None:
121 |     self._s3 = s3_client
122 |     self._env = env
123 |     self._backoff = backoff
124 |     self._sleep = sleep
125 |
126 | def write(
127 |     self,
128 |     envelope_dict: Mapping[str, Any],
129 |     *,
130 |     stage: str,
131 |     attempt: int,
132 | ) -> str:
133 |     """Write ``envelope_dict`` to S3 and return the resulting ``s3://`` URI.
134 |
135 |     Raises the last underlying exception if every retry fails. Caller is
136 |     expected to surface the failure to the run ledger; we do not write
137 |     run_log here to keep the writer side-effect-isolated.
138 |     """
139 |     prefix = s3_prefix(
140 |         env=self._env,
141 |         domain=str(envelope_dict["_ods_domain"]),
142 |         dataset=str(envelope_dict["_ods_dataset"]),
143 |         stage=stage,
144 |         run_id=str(envelope_dict["_ods_run_id"]),
145 |         business_date=envelope_dict.get("_ods_business_date"),
146 |     )
147 |     bucket, key = _split_s3_uri(f"{prefix}{attempt}.json")
148 |     body = json.dumps(envelope_dict, default=str).encode("utf-8")
149 |
150 |     last_exc: Exception | None = None
151 |     for attempt_idx in range(len(self._backoff) + 1):
152 |         try:
153 |             self._s3.put_object(
154 |                 Bucket=bucket,
155 |                 Key=key,
156 |                 Body=body,
157 |                 ContentType="application/json",
158 |             )
159 |             return f"s3://{bucket}/{key}"
160 |         except Exception as exc: # noqa: BLE001 – retry intentionally broad
161 |             last_exc = exc
162 |             if attempt_idx >= len(self._backoff):
163 |                 break
164 |             delay = self._backoff[attempt_idx]
165 |             _log.warning(
166 |                 "DlqWriter put_object failed (attempt %d/%d), retrying in %.1fs: %s",
167 |                 attempt_idx + 1,
168 |                 len(self._backoff) + 1,
169 |                 delay,
170 |                 exc,
171 |             )
172 |             self._sleep(delay)
173 |     assert last_exc is not None # for type checkers
174 |     raise last_exc
175 |
176 |
177 | def _split_s3_uri(uri: str) -> tuple[str, str]:
178 |     parsed = urlparse(uri)
179 |     if parsed.scheme != "s3" or not parsed.netloc:
180 |         raise ValueError(f"not an s3 URI: {uri!r}")
181 |     return parsed.netloc, parsed.path.lstrip("/")
182 |
183 |
184 | def replay_request(
185 |     *,
186 |     dlq_uri: str,
187 |     run_id: str,
188 |     domain: str,
189 |     dataset: str,
190 |     target_stage: str,
191 |     reason: str,
192 | ) -> dict[str, Any]:
193 |     """Return a small structured object operators can pass to replay tooling."""
194 |     return {
195 |         "dlq_uri": dlq_uri,
196 |         "source_run_id": run_id,

```

```
197 |         "domain": domain,  
198 |         "dataset": dataset,  
199 |         "target_stage": target_stage,  
200 |         "reason": reason,  
201 |     }
```

6. ods_pipeline/events.py

191 lines

```
1 | """Publish pipeline run lifecycle events to ods.pipeline.run-events (Kafka + Postgres)."""
2 | from __future__ import annotations
3 |
4 | import datetime
5 | import json
6 | import os
7 | import sys
8 |
9 | from ods_pipeline.db import build_dsn
10 |
11 | TOPIC = "ods.pipeline.run-events"
12 | _SUBJECT = f"{TOPIC}-value"
13 |
14 | SCHEMA_STR = json.dumps({
15 |     "type": "record",
16 |     "name": "RunEvent",
17 |     "namespace": "com.aviva.ods.pipeline",
18 |     "fields": [
19 |         {"name": "run_id", "type": "string"},
20 |         {"name": "event_type", "type": "string"},
21 |         {"name": "pipeline_type", "type": ["null", "string"], "default": None},
22 |         {"name": "domain", "type": "string"},
23 |         {"name": "dataset", "type": "string"},
24 |         {"name": "business_date", "type": "string"},
25 |         {"name": "status", "type": "string"},
26 |         {"name": "record_count_source", "type": ["null", "int"], "default": None},
27 |         {"name": "record_count_dq_pass", "type": ["null", "int"], "default": None},
28 |         {"name": "record_count_dq_fail", "type": ["null", "int"], "default": None},
29 |         {"name": "record_count_published", "type": ["null", "int"], "default": None},
30 |         {"name": "kafka_topic", "type": ["null", "string"], "default": None},
31 |         {"name": "kafka_offset_end", "type": ["null", "long"], "default": None},
32 |         {"name": "error_summary", "type": ["null", "string"], "default": None},
33 |         {"name": "occurred_at", "type": "string"},
34 |         {"name": "file_id", "type": ["null", "string"], "default": None},
35 |         {"name": "s3_raw_path", "type": ["null", "string"], "default": None},
36 |         {"name": "s3_curated_path", "type": ["null", "string"], "default": None},
37 |         {"name": "file_md5", "type": ["null", "string"], "default": None},
38 |         {"name": "kafka_offset_start", "type": ["null", "long"], "default": None},
39 |         {"name": "stages", "type": ["null", "string"], "default": None},
40 |     ],
41 | })
42 |
43 |
44 | def produce(
45 |     event_type: str,
46 |     run_id: str,
47 |     domain: str,
48 |     dataset: str,
49 |     business_date: str,
50 |     status: str,
51 |     *,
52 |     pipeline_type: str | None = None,
53 |     record_count_source: int | None = None,
54 |     record_count_dq_pass: int | None = None,
55 |     record_count_dq_fail: int | None = None,
56 |     record_count_published: int | None = None,
57 |     kafka_topic: str | None = None,
58 |     kafka_offset_end: int | None = None,
59 |     error_summary: str | None = None,
60 |     file_id: str | None = None,
61 |     s3_raw_path: str | None = None,
62 |     s3_curated_path: str | None = None,
63 |     file_md5: str | None = None,
64 |     kafka_offset_start: int | None = None,
65 |     stages: str | None = None,
66 | ) -> None:
67 |     """Emit a run lifecycle event to Postgres (always) and Kafka (best-effort)."""
68 |     payload = {
69 |         "run_id": str(run_id),
70 |         "event_type": event_type,
71 |         "pipeline_type": pipeline_type,
72 |         "domain": domain,
73 |         "dataset": dataset,
74 |         "business_date": str(business_date) if business_date else "",
75 |         "status": status,
76 |         "record_count_source": int(record_count_source) if record_count_source is not None else None,
77 |         "record_count_dq_pass": int(record_count_dq_pass) if record_count_dq_pass is not None else None,
78 |         "record_count_dq_fail": int(record_count_dq_fail) if record_count_dq_fail is not None else None,
79 |         "record_count_published": int(record_count_published) if record_count_published is not None else None,
80 |         "kafka_topic": kafka_topic,
81 |         "kafka_offset_end": int(kafka_offset_end) if kafka_offset_end is not None else None,
82 |         "error_summary": error_summary,
83 |         "occurred_at": datetime.datetime.utcnow().isoformat(),
84 |         "file_id": file_id,
85 |         "s3_raw_path": s3_raw_path,
86 |         "s3_curated_path": s3_curated_path,
87 |         "file_md5": file_md5,
88 |         "kafka_offset_start": int(kafka_offset_start) if kafka_offset_start is not None else None,
89 |         "stages": stages,
90 |     }
91 |
92 |     # Always write to Postgres regardless of Kafka outcome
93 |     _write_pg(payload)
94 |
95 |     # Kafka publish is best-effort – failure must never block the pipeline
96 |     try:
```


7. ods_pipeline/files.py

129 lines

```
1 | """pipeline.file_catalogue and pipeline.file_state operations."""
2 | from __future__ import annotations
3 |
4 | import uuid as _uuid
5 |
6 |
7 | def upsert(
8 |     conn,
9 |     *,
10 |     domain: str,
11 |     dataset: str,
12 |     business_date: str,
13 |     file_md5: str,
14 |     s3_raw_path: str | None = None,
15 |     sftp_path: str | None = None,
16 |     s3_curated_path: str | None = None,
17 |     file_size_bytes: int | None = None,
18 |     source_row_count: int | None = None,
19 |     state: str = "ingested",
20 |     last_run_id: str | None = None,
21 | ) -> str:
22 |     """Upsert ``file_catalogue`` keyed on ``(domain, dataset, s3_raw_path)``.
23 |
24 |     Returns the canonical ``file_id`` UUID string for this landed raw file.
25 |     ``file_md5`` remains a content fingerprint; it is not the identity because
26 |     different files can legitimately have identical content.
27 |     """
28 |     try:
29 |         with conn.cursor() as cur:
30 |             cur.execute(
31 |                 """
32 |                 INSERT INTO pipeline.file_catalogue
33 |                 (file_id, domain, dataset, business_date, file_md5,
34 |                  s3_raw_path, sftp_path, s3_curated_path,
35 |                  file_size_bytes, source_row_count, state, last_run_id)
36 |                 VALUES (%s,%s,%s,%s,%s, %s,%s,%s, %s,%s,%s,%s)
37 |                 ON CONFLICT (domain, dataset, s3_raw_path)
38 |                 WHERE s3_raw_path IS NOT NULL
39 |                 DO UPDATE SET
40 |                     state = EXCLUDED.state,
41 |                     business_date = EXCLUDED.business_date,
42 |                     file_md5 = EXCLUDED.file_md5,
43 |                     s3_raw_path = COALESCE(EXCLUDED.s3_raw_path,
44 |                                             pipeline.file_catalogue.s3_raw_path),
45 |                     s3_curated_path = COALESCE(EXCLUDED.s3_curated_path,
46 |                                                  pipeline.file_catalogue.s3_curated_path),
47 |                     sftp_path = COALESCE(EXCLUDED.sftp_path,
48 |                                           pipeline.file_catalogue.sftp_path),
49 |                     file_size_bytes = COALESCE(EXCLUDED.file_size_bytes,
50 |                                                  pipeline.file_catalogue.file_size_bytes),
51 |                     source_row_count = COALESCE(EXCLUDED.source_row_count,
52 |                                                  pipeline.file_catalogue.source_row_count),
53 |                     last_run_id = EXCLUDED.last_run_id,
54 |                     state_updated_at= NOW()
55 |                 RETURNING file_id
56 |                 """,
57 |                 (
58 |                     str(_uuid.uuid4()), domain, dataset, str(business_date), file_md5,
59 |                     s3_raw_path, sftp_path, s3_curated_path,
60 |                     file_size_bytes, source_row_count, state, last_run_id,
61 |                 ),
62 |             )
63 |             file_id = str(cur.fetchone()[0])
64 |             conn.commit()
65 |             return file_id
66 |     except Exception:
67 |         conn.rollback()
68 |         raise
69 |
70 |
71 | def update_catalogue(conn, file_id: str, **fields) -> None:
72 |     """Update arbitrary columns on a ``file_catalogue`` row by ``file_id``."""
73 |     if not fields:
74 |         return
75 |     sets = ", ".join(f"{c}=%s" for c in fields.keys())
76 |     try:
77 |         with conn.cursor() as cur:
78 |             cur.execute(
79 |                 f"UPDATE pipeline.file_catalogue SET {sets}, state_updated_at=NOW()"
80 |                 f" WHERE file_id=%s",
81 |                 list(fields.values()) + [file_id],
82 |             )
83 |             conn.commit()
84 |     except Exception:
85 |         conn.rollback()
86 |         raise
87 |
88 |
89 | def set_state(
90 |     conn,
91 |     s3_path: str,
92 |     run_id: str,
93 |     status: str,
94 |     *,
95 |     record_count: int | None = None,
96 |     error_reason: str | None = None,
```

```

97 | ) -> None:
98 |     """Upsert ``pipeline.file_state`` for *s3_path*."""
99 |     try:
100 |         with conn.cursor() as cur:
101 |             cur.execute(
102 |                 """
103 |                 INSERT INTO pipeline.file_state
104 |                 (s3_path, run_id, status, record_count, error_reason)
105 |                 VALUES (%s, %s, %s, %s, %s)
106 |                 ON CONFLICT (s3_path) DO UPDATE SET
107 |                     run_id      = EXCLUDED.run_id,
108 |                     status      = EXCLUDED.status,
109 |                     record_count = EXCLUDED.record_count,
110 |                     error_reason = EXCLUDED.error_reason,
111 |                     updated_at   = NOW()
112 |                 """
113 |                 (s3_path, run_id, status, record_count, error_reason),
114 |             )
115 |             conn.commit()
116 |     except Exception:
117 |         conn.rollback()
118 |         raise
119 |
120 |
121 | def get_state(conn, s3_path: str) -> str | None:
122 |     """Return the current status string for *s3_path*, or ``None`` if not found."""
123 |     with conn.cursor() as cur:
124 |         cur.execute(
125 |             "SELECT status FROM pipeline.file_state WHERE s3_path = %s",
126 |             (s3_path,),
127 |         )
128 |         row = cur.fetchone()
129 |         return row[0] if row else None

```


8. ods_pipeline/ingest/__init__.py

5 lines

```
1 | """ODS ingest entry-points outside the Glue Spark jobs.  
2 |  
3 | Currently exposes the api_pull poller, used by dag_api_pull to fetch a  
4 | batch of records from an external HTTP API and archive them to S3.  
5 | """
```

9. ods_pipeline/ingest/api_pull/__init__.py

41 lines

```
1 | """API pull poller – fetch external HTTP API on a schedule and archive
2 | records as gzipped JSONL on S3.
3 |
4 | Public surface kept small on purpose; dag_api_pull only needs:
5 |
6 |     poll_and_archive(...)      the one-shot batch poller
7 |     ArchivedBatch              the typed return value
8 |     AuthProvider, BearerAuth   auth Protocol + bearer impl
9 |     Cursor, build_cursor       cursor Protocol + factory
10 |    WatermarkStore              pending/committed cursor store
11 |
12 | Each strategy lives in its own module so adding etag/offset/full_replace
13 | later is one new file, not a refactor.
14 | """
15 | from __future__ import annotations
16 |
17 | from ods_pipeline.ingest.api_pull.archive import ArchivedBatch, write_jsonl_archive
18 | from ods_pipeline.ingest.api_pull.auth import AuthProvider, BearerAuth, build_auth
19 | from ods_pipeline.ingest.api_pull.cursors import Cursor, build_cursor
20 | from ods_pipeline.ingest.api_pull.linkage import (
21 |     TRIGGERED_BY_API_PULL_EDGE,
22 |     derive_dag_ingest_parent_run_id,
23 |     ingest_status_for_api_pull_run,
24 | )
25 | from ods_pipeline.ingest.api_pull.poller import poll_and_archive
26 | from ods_pipeline.ingest.api_pull.watermark import WatermarkStore
27 |
28 | __all__ = [
29 |     "ArchivedBatch",
30 |     "AuthProvider",
31 |     "BearerAuth",
32 |     "Cursor",
33 |     "TRIGGERED_BY_API_PULL_EDGE",
34 |     "WatermarkStore",
35 |     "build_auth",
36 |     "build_cursor",
37 |     "derive_dag_ingest_parent_run_id",
38 |     "ingest_status_for_api_pull_run",
39 |     "poll_and_archive",
40 |     "write_jsonl_archive",
41 | ]
```

10. ods_pipeline/ingest/api_pull/archive.py

183 lines

```
1 | """Gzipped JSONL archive writer for api_pull.
2 |
3 | Each fetched record is wrapped in the standard ODS metadata envelope
4 | (_ods_run_id, _ods_source_request_id, _ods_source_application, ...)
5 | before being written to S3 as one line of gzipped JSONL. The downstream
6 | Glue ingestion job reads JSONL via spark.read.json and applies the same
7 | schema validation and DQ pipeline as the file pattern.
8 |
9 | Identity: the archive is keyed by (domain, dataset, business_date,
10 | run_id) – one immutable object per poll. Replay is safe because the
11 | file_catalogue upsert is keyed on s3_raw_path.
12 | """
13 | from __future__ import annotations
14 |
15 | import gzip
16 | import hashlib
17 | import io
18 | import json
19 | from dataclasses import dataclass
20 | from typing import Any, Mapping, Sequence
21 |
22 | from ods_pipeline import metadata as _metadata
23 |
24 |
25 | @dataclass
26 | class ArchivedBatch:
27 |     """Outcome of a single api_pull poll.
28 |
29 |     ``no_changes=True`` means the source returned 0 records (or 304); the
30 |     DAG should mark the run skipped, not trigger dag_ingest, and leave
31 |     the watermark unchanged.
32 |     """
33 |
34 |     domain: str
35 |     dataset: str
36 |     business_date: str
37 |     source_application: str
38 |     s3_uri: str
39 |     s3_bucket: str
40 |     s3_key: str
41 |     file_md5: str
42 |     file_size_bytes: int
43 |     record_count: int
44 |     page_count: int
45 |     old_cursor_value: str | None
46 |     new_cursor_value: str | None
47 |     source_request_id: str
48 |     no_changes: bool = False
49 |
50 |
51 | def _envelope(
52 |     *,
53 |     record: Mapping[str, Any],
54 |     run_id: str,
55 |     source_application: str,
56 |     domain: str,
57 |     dataset: str,
58 |     business_date: str,
59 |     source_request_id: str,
60 |     cursor_value: str | None,
61 |     archive_uri: str,
62 |     schema_id: str,
63 |     schema_version: int | str,
64 | ) -> dict[str, Any]:
65 |     """One JSONL line: ODS metadata envelope wrapping the source record."""
66 |     meta = _metadata.message_metadata(
67 |         run_id=run_id,
68 |         domain=domain,
69 |         dataset=dataset,
70 |         source_application=source_application,
71 |         source_request_id=source_request_id,
72 |     )
73 |     envelope = _metadata.archive_envelope(
74 |         payload=dict(record),
75 |         metadata=meta,
76 |         schema_id=schema_id,
77 |         schema_version=schema_version,
78 |         archive_s3_uri=archive_uri,
79 |     )
80 |     envelope["_ods_business_date"] = business_date
81 |     envelope["_ods_source_cursor"] = cursor_value
82 |     return envelope
83 |
84 |
85 | def _gzip_jsonl(lines: Sequence[Mapping[str, Any]]) -> bytes:
86 |     buffer = io.BytesIO()
87 |     with gzip.GzipFile(fileobj=buffer, mode="wb", mtime=0) as gz:
88 |         for line in lines:
89 |             gz.write(json.dumps(line, default=str, sort_keys=True).encode("utf-8"))
90 |             gz.write(b"\n")
91 |     return buffer.getvalue()
92 |
93 |
94 | def write_jsonl_archive(
95 |     *,
96 |     s3_client,
```

```

97 |     bucket: str,
98 |     records: Sequence[Mapping[str, Any]],
99 |     domain: str,
100 |     dataset: str,
101 |     business_date: str,
102 |     run_id: str,
103 |     source_application: str,
104 |     source_request_id: str,
105 |     cursor_value: str | None,
106 |     schema_id: str,
107 |     schema_version: int | str = 1,
108 |     page_count: int = 1,
109 |     old_cursor_value: str | None = None,
110 |     new_cursor_value: str | None = None,
111 | ) -> ArchivedBatch:
112 |     """Wrap ``records`` in ODS envelopes, gzip as JSONL, and put to S3.
113 |
114 |     Returns ArchivedBatch with file_md5, size and S3 location so the caller
115 |     can register the archive in pipeline.file_catalogue and write lineage.
116 |     """
117 |     s3_key = (
118 |         f"api_pull/{domain}/{dataset}/"
119 |         f"date={business_date}/run_id={run_id}.jsonl.gz"
120 |     )
121 |     s3_uri = f"s3://{bucket}/{s3_key}"
122 |
123 |     if not records:
124 |         return ArchivedBatch(
125 |             domain=domain,
126 |             dataset=dataset,
127 |             business_date=business_date,
128 |             source_application=source_application,
129 |             s3_uri=s3_uri,
130 |             s3_bucket=bucket,
131 |             s3_key=s3_key,
132 |             file_md5="",
133 |             file_size_bytes=0,
134 |             record_count=0,
135 |             page_count=page_count,
136 |             old_cursor_value=old_cursor_value,
137 |             new_cursor_value=new_cursor_value,
138 |             source_request_id=source_request_id,
139 |             no_changes=True,
140 |         )
141 |
142 |     enveloped = [
143 |         _envelope(
144 |             record=record,
145 |             run_id=run_id,
146 |             source_application=source_application,
147 |             domain=domain,
148 |             dataset=dataset,
149 |             business_date=business_date,
150 |             source_request_id=source_request_id,
151 |             cursor_value=cursor_value,
152 |             archive_uri=s3_uri,
153 |             schema_id=schema_id,
154 |             schema_version=schema_version,
155 |         )
156 |         for record in records
157 |     ]
158 |     body = _gzip_jsonl(enveloped)
159 |     file_md5 = hashlib.md5(body).hexdigest()
160 |     s3_client.put_object(
161 |         Bucket=bucket,
162 |         Key=s3_key,
163 |         Body=body,
164 |         ContentType="application/x-jsonlines",
165 |         ContentEncoding="gzip",
166 |     )
167 |     return ArchivedBatch(
168 |         domain=domain,
169 |         dataset=dataset,
170 |         business_date=business_date,
171 |         source_application=source_application,
172 |         s3_uri=s3_uri,
173 |         s3_bucket=bucket,
174 |         s3_key=s3_key,
175 |         file_md5=file_md5,
176 |         file_size_bytes=len(body),
177 |         record_count=len(records),
178 |         page_count=page_count,
179 |         old_cursor_value=old_cursor_value,
180 |         new_cursor_value=new_cursor_value,
181 |         source_request_id=source_request_id,
182 |         no_changes=False,
183 |     )

```

11. ods_pipeline/ingest/api_pull/auth.py

80 lines

```
1 | """HTTP auth providers for the api_pull poller.
2 |
3 | A provider is anything that mutates a ``requests.Session`` so subsequent
4 | requests carry the correct credentials. Bearer is implemented for slice 1;
5 | ``none`` is allowed for stub/test sources. ``basic``, ``mtls`` and OAuth
6 | plug into the same Protocol – adding one is one new class + one
7 | ``build_auth`` branch.
8 |
9 | Secret material is never read from ``source_config``. It is looked up by
10 | ``secret_ref`` from the environment (Airflow Secrets Backend mounts as
11 | env vars). This keeps tokens out of the dataset_config table.
12 | """
13 | from __future__ import annotations
14 |
15 | import os
16 | from typing import Mapping, Protocol
17 |
18 | import requests
19 |
20 |
21 | class AuthProvider(Protocol):
22 |     """Anything that knows how to attach credentials to a Session."""
23 |
24 |     type: str
25 |
26 |     def apply(self, session: requests.Session) -> None: ...
27 |
28 |
29 | class NoAuth:
30 |     """No-op auth – used by tests and public stub APIs."""
31 |
32 |     type = "none"
33 |
34 |     def apply(self, session: requests.Session) -> None: # noqa: D401 – Protocol impl
35 |         return None
36 |
37 |
38 | class BearerAuth:
39 |     """``Authorization: Bearer <token>`` from a named env var.
40 |
41 |     The env var name comes from ``secret_ref`` in source_config. Missing or
42 |     blank token raises immediately so we fail fast on misconfig rather
43 |     than emit unauthenticated requests.
44 |     """
45 |
46 |     type = "bearer"
47 |
48 |     def __init__(self, secret_ref: str, *, env: Mapping[str, str] | None = None):
49 |         if not secret_ref:
50 |             raise ValueError("bearer auth requires a non-empty secret_ref")
51 |         env = env if env is not None else os.environ
52 |         token = env.get(secret_ref)
53 |         if not token:
54 |             raise ValueError(
55 |                 f"bearer auth secret_ref={secret_ref!r} not set in environment"
56 |             )
57 |         self._token = token
58 |
59 |     def apply(self, session: requests.Session) -> None:
60 |         session.headers["Authorization"] = f"Bearer {self._token}"
61 |
62 |
63 | def build_auth(
64 |     auth_config: Mapping[str, object] | None,
65 |     *,
66 |     env: Mapping[str, str] | None = None,
67 | ) -> AuthProvider:
68 |     """Construct an AuthProvider from the ``source.auth`` block in YAML."""
69 |     if not auth_config:
70 |         return NoAuth()
71 |     auth_type = str(auth_config.get("type", "none")).lower()
72 |     if auth_type == "none":
73 |         return NoAuth()
74 |     if auth_type == "bearer":
75 |         secret_ref = str(auth_config.get("secret_ref", ""))
76 |         return BearerAuth(secret_ref, env=env)
77 |     raise ValueError(
78 |         f"unsupported auth.type={auth_type!r}; "
79 |         f"slice 1 supports 'none' and 'bearer' only"
80 |     )
```

12. ods_pipeline/ingest/api_pull/cursors.py

174 lines

```
1 | """Cursor strategies for the api_pull poller.
2 |
3 | A Cursor is the small piece of state that decides:
4 | - what request parameters carry the watermark forward,
5 | - how to walk pages within one poll,
6 | - what the next watermark should be after a successful poll.
7 |
8 | Slice 1 implements ``since_timestamp`` only. ``etag``, ``offset`` and
9 | ``full_replace`` plug in by adding new classes and one ``build_cursor``
10 | branch – the poller body never branches on style.
11 |
12 | since_timestamp:
13 |     request: GET ?{request_param}={committed_or_initial}
14 |     paging: follow RFC 5988 Link rel=next; if absent stop after one page.
15 |     watermark: max({response_field}) over fetched records, ISO-8601 string.
16 |               If the page is empty, the cursor is left unchanged so the
17 |               next poll re-issues the same window.
18 | """
19 | from __future__ import annotations
20 |
21 | from dataclasses import dataclass
22 | from typing import Any, Mapping, Protocol, Sequence
23 |
24 |
25 | @dataclass
26 | class CursorRequest:
27 |     """One HTTP request the poller should issue.
28 |
29 |     The poller does not decide URLs – Cursors do. ``url`` may be an absolute
30 |     next-page link (link_header paging) or the configured base URL with the
31 |     cursor query string applied (first page).
32 |     """
33 |
34 |     url: str
35 |     params: dict[str, str]
36 |
37 |
38 | class Cursor(Protocol):
39 |     """Strategy that drives one poll of one dataset.
40 |
41 |     A Cursor is constructed once per poll with the committed watermark and
42 |     is consumed by the poller via ``initial_request``, ``next_request``
43 |     and ``advance``. It carries no I/O – pure state machine.
44 |     """
45 |
46 |     style: str
47 |
48 |     def initial_request(self) -> CursorRequest: ...
49 |
50 |     def next_request(
51 |         self,
52 |         last_response_headers: Mapping[str, str],
53 |         last_response_body: Any,
54 |     ) -> CursorRequest | None:
55 |         """Return the next page request, or None if paging is exhausted."""
56 |
57 |     def advance(self, all_records: Sequence[Mapping[str, Any]]) -> str | None:
58 |         """Compute the new watermark value after a successful poll.
59 |
60 |         ``None`` means leave the committed cursor unchanged (e.g. empty page,
61 |         304 Not Modified, full_replace one-shot).
62 |         """
63 |
64 |
65 | class SinceTimestampCursor:
66 |     """``?{request_param}={cursor}`` watermark with optional Link paging."""
67 |
68 |     style = "since_timestamp"
69 |
70 |     def __init__(
71 |         self,
72 |         *,
73 |         base_url: str,
74 |         request_param: str,
75 |         response_field: str,
76 |         committed_value: str | None,
77 |         initial_value: str,
78 |         page_style: str = "link_header",
79 |         extra_params: Mapping[str, str] | None = None,
80 |     ):
81 |         if not request_param:
82 |             raise ValueError("since_timestamp cursor requires request_param")
83 |         if not response_field:
84 |             raise ValueError("since_timestamp cursor requires response_field")
85 |         if page_style not in {"link_header", "none"}:
86 |             raise ValueError(
87 |                 f"since_timestamp cursor: page.style={page_style!r} not supported "
88 |                 f"in slice 1; expected 'link_header' or 'none'"
89 |             )
90 |         self._base_url = base_url
91 |         self._request_param = request_param
92 |         self._response_field = response_field
93 |         self._cursor_value = committed_value or initial_value
94 |         self._page_style = page_style
95 |         self._extra_params = dict(extra_params or {})
96 |
```

```

97 | def initial_request(self) -> CursorRequest:
98 |     params = {**self._extra_params, self._request_param: self._cursor_value}
99 |     return CursorRequest(url=self._base_url, params=params)
100 |
101 | def next_request(
102 |     self,
103 |     last_response_headers: Mapping[str, str],
104 |     last_response_body: Any,
105 | ) -> CursorRequest | None:
106 |     if self._page_style != "link_header":
107 |         return None
108 |     link_header = last_response_headers.get("Link") or last_response_headers.get("link")
109 |     if not link_header:
110 |         return None
111 |     next_url = _parse_next_link(link_header)
112 |     if not next_url:
113 |         return None
114 |     # Subsequent pages carry the cursor in the Link URL itself; do not
115 |     # re-apply request_param so we don't double-stamp the query string.
116 |     return CursorRequest(url=next_url, params={})
117 |
118 | def advance(self, all_records: Sequence[Mapping[str, Any]]) -> str | None:
119 |     if not all_records:
120 |         return None
121 |     max_value: str | None = None
122 |     for record in all_records:
123 |         value = record.get(self._response_field)
124 |         if value is None:
125 |             continue
126 |         value_s = str(value)
127 |         if max_value is None or value_s > max_value:
128 |             max_value = value_s
129 |     return max_value
130 |
131 | def build_cursor(
132 |     source: Mapping[str, Any],
133 |     *,
134 |     committed_value: str | None,
135 | ) -> Cursor:
136 |     """Construct the Cursor for a dataset's ``source`` config block."""
137 |     cursor_cfg = source.get("cursor") or {}
138 |     style = str(cursor_cfg.get("style", "since_timestamp")).lower()
139 |     if style == "since_timestamp":
140 |         page_cfg = source.get("page") or {}
141 |         return SinceTimestampCursor(
142 |             base_url=str(source["url"]),
143 |             request_param=str(cursor_cfg.get("request_param", "updated_since")),
144 |             response_field=str(cursor_cfg.get("response_field", "updated_at")),
145 |             committed_value=committed_value,
146 |             initial_value=str(cursor_cfg.get("initial", "")),
147 |             page_style=str(page_cfg.get("style", "link_header")),
148 |         )
149 |     raise ValueError(
150 |         f"unsupported cursor.style={style!r}; "
151 |         f"slice 1 supports 'since_timestamp' only"
152 |     )
153 |
154 | def _parse_next_link(link_header: str) -> str | None:
155 |     """Pull the ``rel=next`` URL out of an RFC 5988 Link header.
156 |
157 |     Format example::
158 |
159 |         Link: <https://api.example/items?page=2>; rel="next", <...>; rel="prev"
160 |
161 |     """
162 |     for part in link_header.split(","):
163 |         segments = [s.strip() for s in part.split(";") if s.strip()]
164 |         if not segments:
165 |             continue
166 |         url_segment = segments[0]
167 |         if not (url_segment.startswith("<") and url_segment.endswith(">")):
168 |             continue
169 |         url = url_segment[1:-1]
170 |         for attr in segments[1:]:
171 |             if attr.replace(" ", "").lower() in {'rel="next"', 'rel=next'}:
172 |                 return url
173 |     return None
174 |

```

13. ods_pipeline/ingest/api_pull/linkage.py

108 lines

```
1 | """Run-linkage helpers for api_pull <-> dag_ingest.
2 |
3 | The api_pull control-plane needs to look up the EXACT dag_ingest parent
4 | run launched by a given api_pull poll, not "the latest s3_batch run for
5 | this file_id" or "any run carrying our edge". Two scenarios drove the
6 | two-layer match below:
7 |
8 | 1. A replay of the same ``file_id`` with a different api_pull poll
9 |    would match a "latest by file_id" lookup. Fixed by requiring the
10 |    ``triggered_by_api_pull`` edge in ``run_log.parents``.
11 | 2. TriggerDagRunOperator retries, manual re-triggers, or bugs could
12 |    produce TWO rows that both carry the ``triggered_by_api_pull``
13 |    edge for the same ``api_pull_run_id``. A "latest by edge" lookup
14 |    could then promote/clear the wrong cursor. Fixed by requiring the
15 |    caller (dag_api_pull) to pre-mint a deterministic
16 |    ``expected_parent_run_id`` and matching it as the run_log primary
17 |    key. PK uniqueness eliminates ambiguity by definition. The edge
18 |    check stays as defence-in-depth.
19 |
20 | If the caller cannot supply ``expected_parent_run_id`` (legacy / test
21 | paths), the lookup falls back to JSONB containment but returns ``None``
22 | when more than one row matches – "ambiguous, do not promote".
23 |
24 | Lives outside the Airflow DAG module so plain pytest (no Airflow
25 | runtime) can exercise the SQL contract.
26 | """
27 | from __future__ import annotations
28 |
29 | import json
30 | import uuid
31 |
32 | TRIGGERED_BY_API_PULL_EDGE = "triggered_by_api_pull"
33 |
34 |
35 | def derive_dag_ingest_parent_run_id(api_pull_run_id: str) -> str:
36 |     """Compute the deterministic ``run_id`` that dag_ingest.init_run will
37 |     use as its parent run when triggered by this api_pull poll.
38 |
39 |     Both ends derive the same value from the same api_pull_run_id so
40 |     no extra column or coordination state is needed. UUID5 namespacing
41 |     keeps it stable across processes / restarts and collision-free with
42 |     randomly-minted UUIDs.
43 |     """
44 |     return str(
45 |         uuid.uuid5(uuid.NAMESPACE_OID, f"api_pull:{api_pull_run_id}")
46 |     )
47 |
48 |
49 | def ingest_status_for_api_pull_run(
50 |     conn,
51 |     api_pull_run_id: str,
52 |     *,
53 |     expected_parent_run_id: str | None = None,
54 | ) -> str | None:
55 |     """Return the status of the exact dag_ingest parent run launched by
56 |     THIS api_pull poll, or ``None`` if no unambiguous match exists.
57 |
58 |     Behaviour:
59 |
60 |     * If ``expected_parent_run_id`` is supplied, look up by PK and
61 |       verify the ``triggered_by_api_pull`` edge is present in
62 |       ``run_log.parents``. Returns the status iff both match.
63 |     * Otherwise, fall back to JSONB containment on the edge alone.
64 |       If MORE THAN ONE row matches, return ``None`` – the lookup is
65 |       ambiguous and the caller (finalise_watermark) MUST NOT promote
66 |       or clear the watermark on an ambiguous result.
67 |     """
68 |     needle = json.dumps([
69 |         "run_id": api_pull_run_id,
70 |         "edge_type": TRIGGERED_BY_API_PULL_EDGE,
71 |     ])
72 |     with conn.cursor() as cur:
73 |         if expected_parent_run_id:
74 |             # Exact-PK match. PK uniqueness in run_log makes the result
75 |             # at most one row; the edge check rejects same-PK rows that
76 |             # somehow lack the api_pull linkage.
77 |             cur.execute(
78 |                 """
79 |                 SELECT status
80 |                 FROM pipeline.run_log
81 |                 WHERE run_id = %s::uuid
82 |                   AND pipeline_type = 's3_batch'
83 |                   AND parents @> %s::jsonb
84 |                 """,
85 |                 (expected_parent_run_id, needle),
86 |             )
87 |             row = cur.fetchone()
88 |             return row[0] if row else None
89 |
90 |         # Fallback: edge-only lookup. Pull at most 2 rows so we can
91 |         # detect ambiguity without scanning the full table.
92 |         cur.execute(
93 |             """
94 |             SELECT status
95 |             FROM pipeline.run_log
96 |             WHERE pipeline_type = 's3_batch'
```



```

97 |         AND parents @> %s::jsonb
98 |         ORDER BY started_at DESC NULLS LAST, run_id::text DESC
99 |         LIMIT 2
100 |         """,
101 |         (needle, ),
102 |     )
103 |     rows = cur.fetchall()
104 |     if not rows:
105 |         return None
106 |     if len(rows) > 1:
107 |         return None # ambiguous – caller must NOT promote/clear
108 |     return rows[0][0]

```

14. ods_pipeline/ingest/api_pull/poller.py

156 lines

```
1 | """api_pull HTTP poller.
2 |
3 | Single public entry-point: :func:`poll_and_archive`. Builds an HTTP
4 | session with the configured auth, walks pages via the configured cursor
5 | strategy, accumulates all records into one logical batch, and writes
6 | that batch to S3 as one immutable gzipped JSONL archive. Watermark
7 | read/record_pending lifecycle is owned by the caller (dag_api_pull) so
8 | locking and the post-trigger promote/clear_pending hooks live in one
9 | place.
10 | """
11 | from __future__ import annotations
12 |
13 | import uuid
14 | from typing import Any, Mapping, Sequence
15 |
16 | import requests
17 | from requests.adapters import HTTPAdapter
18 | from urllib3.util.retry import Retry
19 |
20 | from ods_pipeline.ingest.api_pull.archive import ArchivedBatch, write_jsonl_archive
21 | from ods_pipeline.ingest.api_pull.auth import AuthProvider, build_auth
22 | from ods_pipeline.ingest.api_pull.cursors import Cursor, CursorRequest, build_cursor
23 |
24 |
25 | def _build_session(
26 |     *,
27 |     auth: AuthProvider,
28 |     timeout_seconds: float,
29 |     retries: int,
30 | ) -> requests.Session:
31 |     session = requests.Session()
32 |     retry = Retry(
33 |         total=retries,
34 |         backoff_factor=0.5,
35 |         status_forcelist=(429, 500, 502, 503, 504),
36 |         allowed_methods=frozenset({"GET", "HEAD"}),
37 |         raise_on_status=False,
38 |     )
39 |     adapter = HTTPAdapter(max_retries=retry)
40 |     session.mount("http://", adapter)
41 |     session.mount("https://", adapter)
42 |     session.headers["Accept"] = "application/json"
43 |     session.request = _with_default_timeout(session.request, timeout_seconds) # type: ignore[assignment]
44 |     auth.apply(session)
45 |     return session
46 |
47 |
48 | def _with_default_timeout(method, timeout_seconds: float):
49 |     """Wrap Session.request so callers always get a default timeout."""
50 |
51 |     def wrapper(*args, **kwargs):
52 |         kwargs.setdefault("timeout", timeout_seconds)
53 |         return method(*args, **kwargs)
54 |
55 |     return wrapper
56 |
57 |
58 | def _records_from_body(body: Any) -> Sequence[Mapping[str, Any]]:
59 |     """Accept ``[{...}]`` or ``{"items": [...]}`` / ``{"data": [...]}``.
60 |
61 |     Avoids hard-coding a wire shape; api_pull integrations vary. Anything
62 |     else is treated as zero records – the caller decides how to surface it.
63 |     """
64 |     if isinstance(body, list):
65 |         return body
66 |     if isinstance(body, dict):
67 |         for key in ("items", "data", "records", "results"):
68 |             value = body.get(key)
69 |             if isinstance(value, list):
70 |                 return value
71 |     return []
72 |
73 |
74 | def poll_and_archive(
75 |     *,
76 |     dataset_config: Mapping[str, Any],
77 |     s3_client,
78 |     archive_bucket: str,
79 |     committed_cursor_value: str | None,
80 |     run_id: str,
81 |     business_date: str,
82 |     session: requests.Session | None = None,
83 |     cursor: Cursor | None = None,
84 |     env: Mapping[str, str] | None = None,
85 | ) -> ArchivedBatch:
86 |     """Run one HTTP poll for a dataset and archive results to S3.
87 |
88 |     Wiring:
89 |     - ``dataset_config`` carries domain, dataset, schema_id and the
90 |       ``source`` block (URL, auth, cursor, page, retries, timeout).
91 |     - ``committed_cursor_value`` is the watermark to issue this poll
92 |       from. ``None`` falls back to the cursor's configured ``initial``.
93 |     - ``s3_client``, ``session`` and ``cursor`` are injectable so tests
94 |       can drive the function without real network or S3.
95 |
96 |     Returns ArchivedBatch. ``no_changes=True`` means the source had
```

```

97 | nothing new – the DAG should skip dag_ingest and leave the watermark
98 | unchanged.
99 | """
100 | domain = str(dataset_config["domain"])
101 | dataset = str(dataset_config["dataset"])
102 | source = dict(dataset_config.get("source") or {})
103 | source_application = str(source.get("application", f"{domain}.{dataset}"))
104 | schema_id = str(dataset_config.get("schema_id", f"{domain}.{dataset}"))
105 | schema_version = dataset_config.get("schema_version", 1)
106 | timeout_seconds = float(source.get("timeout_seconds", 30))
107 | retries = int(source.get("retries", 3))
108 |
109 | if session is None:
110 |     auth = build_auth(source.get("auth"), env=env)
111 |     session = _build_session(
112 |         auth=auth,
113 |         timeout_seconds=timeout_seconds,
114 |         retries=retries,
115 |     )
116 |
117 | if cursor is None:
118 |     cursor = build_cursor(source, committed_value=committed_cursor_value)
119 |
120 | source_request_id = str(uuid.uuid4())
121 | all_records: list[Mapping[str, Any]] = []
122 | request: CursorRequest | None = cursor.initial_request()
123 | page_count = 0
124 |
125 | while request is not None:
126 |     response = session.get(request.url, params=request.params or None)
127 |     page_count += 1
128 |     if response.status_code == 304:
129 |         # Etag/no-change short-circuit. Slice 1 doesn't issue
130 |         # If-None-Match yet, but we still honour 304 if a source emits it.
131 |         break
132 |     response.raise_for_status()
133 |     body = response.json() if response.content else None
134 |     records = _records_from_body(body)
135 |     all_records.extend(records)
136 |     request = cursor.next_request(response.headers, body)
137 |
138 | new_cursor_value = cursor.advance(all_records)
139 |
140 | return write_jsonl_archive(
141 |     s3_client=s3_client,
142 |     bucket=archive_bucket,
143 |     records=all_records,
144 |     domain=domain,
145 |     dataset=dataset,
146 |     business_date=business_date,
147 |     run_id=run_id,
148 |     source_application=source_application,
149 |     source_request_id=source_request_id,
150 |     cursor_value=committed_cursor_value,
151 |     schema_id=schema_id,
152 |     schema_version=schema_version,
153 |     page_count=page_count,
154 |     old_cursor_value=committed_cursor_value,
155 |     new_cursor_value=new_cursor_value,
156 | )

```

15. ods_pipeline/ingest/api_pull/watermark.py

229 lines

```
1 | """Two-phase watermark store for api_pull.
2 |
3 | The api_pull control-plane separates ``pending`` and ``committed`` cursors:
4 |
5 | 1. read_committed()      return the cursor to issue the next poll from
6 | 2. lock(run_id)          advisory-style lock so concurrent DAG runs do
7 |                           not double-poll the same dataset
8 | 3. record_pending()      after S3 archive succeeds, stage the new cursor
9 |                           alongside the run_id that owns it
10 | 4. promote()             called by dag_api_pull's post-trigger sensor
11 |                           once the downstream dag_ingest run succeeds
12 | 5. clear_pending()       called when the downstream run failed or was
13 |                           aborted; committed cursor is left untouched
14 |
15 | The split guarantees no records are dropped if dag_ingest fails after the
16 | poll succeeds: the same window is re-issued on the next schedule.
17 | """
18 | from __future__ import annotations
19 |
20 | from dataclasses import dataclass
21 |
22 |
23 | @dataclass
24 | class WatermarkRow:
25 |     domain: str
26 |     dataset: str
27 |     source_application: str
28 |     cursor_type: str
29 |     committed_cursor_value: str | None
30 |     pending_cursor_value: str | None
31 |     pending_run_id: str | None
32 |     last_successful_run_id: str | None
33 |     locked: bool
34 |
35 |
36 | class WatermarkStore:
37 |     """Thin DAO over ``pipeline.api_pull_watermark``.
38 |
39 |     All methods take an open psycopg2 connection. Each call opens its own
40 |     short-lived cursor and commits – these are control-plane writes that
41 |     must not piggy-back on the caller's data transaction.
42 |     """
43 |
44 |     def __init__(self, conn):
45 |         self._conn = conn
46 |
47 |     # ----- read
48 |
49 |     def read(
50 |         self,
51 |         *,
52 |         domain: str,
53 |         dataset: str,
54 |         source_application: str,
55 |         cursor_type: str,
56 |     ) -> WatermarkRow:
57 |         """Return the current row, inserting a fresh one with NULL cursors
58 |         if none exists for this (domain, dataset, source_application)."""
59 |         with self._conn.cursor() as cur:
60 |             cur.execute(
61 |                 """
62 |                 INSERT INTO pipeline.api_pull_watermark
63 |                     (domain, dataset, source_application, cursor_type)
64 |                 VALUES (%s, %s, %s, %s)
65 |                 ON CONFLICT (domain, dataset, source_application) DO NOTHING
66 |                 """,
67 |                 (domain, dataset, source_application, cursor_type),
68 |             )
69 |             cur.execute(
70 |                 """
71 |                 SELECT cursor_type, committed_cursor_value, pending_cursor_value,
72 |                     pending_run_id::text, last_successful_run_id::text,
73 |                     locked_at IS NOT NULL
74 |                 FROM pipeline.api_pull_watermark
75 |                 WHERE domain=%s AND dataset=%s AND source_application=%s
76 |                 """,
77 |                 (domain, dataset, source_application),
78 |             )
79 |             row = cur.fetchone()
80 |             self._conn.commit()
81 |             existing_type, committed, pending, pending_run, last_run, locked = row
82 |             return WatermarkRow(
83 |                 domain=domain,
84 |                 dataset=dataset,
85 |                 source_application=source_application,
86 |                 cursor_type=existing_type,
87 |                 committed_cursor_value=committed,
88 |                 pending_cursor_value=pending,
89 |                 pending_run_id=pending_run,
90 |                 last_successful_run_id=last_run,
91 |                 locked=bool(locked),
92 |             )
93 |
94 |     # ----- lock
95 |
96 |     def try_lock(
```

```

97 |         self,
98 |         *,
99 |         domain: str,
100 |         dataset: str,
101 |         source_application: str,
102 |         run_id: str,
103 |     ) -> bool:
104 |         """Set ``locked_at=NOW()`` only if currently NULL.
105 |
106 |         Returns True if the caller now holds the lock for this dataset.
107 |         Caller must release via ``unlock`` even on failure.
108 |         """
109 |         with self._conn.cursor() as cur:
110 |             cur.execute(
111 |                 """
112 |                 UPDATE pipeline.api_pull_watermark
113 |                 SET locked_at=NOW(),
114 |                     pending_run_id=%s,
115 |                     updated_at=NOW()
116 |                 WHERE domain=%s AND dataset=%s AND source_application=%s
117 |                     AND locked_at IS NULL
118 |                 """,
119 |                 (run_id, domain, dataset, source_application),
120 |             )
121 |             acquired = cur.rowcount == 1
122 |             self._conn.commit()
123 |             return acquired
124 |
125 |     def unlock(
126 |         self,
127 |         *,
128 |         domain: str,
129 |         dataset: str,
130 |         source_application: str,
131 |     ) -> None:
132 |         """Release the dataset lock. Pending cursor is left untouched –
133 |         promote/clear_pending owns that lifecycle."""
134 |         with self._conn.cursor() as cur:
135 |             cur.execute(
136 |                 """
137 |                 UPDATE pipeline.api_pull_watermark
138 |                 SET locked_at=NULL,
139 |                     updated_at=NOW()
140 |                 WHERE domain=%s AND dataset=%s AND source_application=%s
141 |                 """,
142 |                 (domain, dataset, source_application),
143 |             )
144 |             self._conn.commit()
145 |
146 |     # ----- pending
147 |
148 |     def record_pending(
149 |         self,
150 |         *,
151 |         domain: str,
152 |         dataset: str,
153 |         source_application: str,
154 |         run_id: str,
155 |         new_cursor_value: str,
156 |     ) -> None:
157 |         """Stage a new cursor against ``run_id``.
158 |
159 |         Called only after the S3 archive write succeeded. promote() will
160 |         move this value into ``committed_cursor_value`` once dag_ingest
161 |         for ``run_id`` finishes successfully.
162 |         """
163 |         with self._conn.cursor() as cur:
164 |             cur.execute(
165 |                 """
166 |                 UPDATE pipeline.api_pull_watermark
167 |                 SET pending_cursor_value=%s,
168 |                     pending_run_id=%s,
169 |                     updated_at=NOW()
170 |                 WHERE domain=%s AND dataset=%s AND source_application=%s
171 |                 """,
172 |                 (new_cursor_value, run_id, domain, dataset, source_application),
173 |             )
174 |             self._conn.commit()
175 |
176 |     def promote(
177 |         self,
178 |         *,
179 |         domain: str,
180 |         dataset: str,
181 |         source_application: str,
182 |         run_id: str,
183 |     ) -> bool:
184 |         """Move pending -> committed iff ``run_id`` still owns the pending
185 |         cursor. Returns True if a row was promoted."""
186 |         with self._conn.cursor() as cur:
187 |             cur.execute(
188 |                 """
189 |                 UPDATE pipeline.api_pull_watermark
190 |                 SET committed_cursor_value=pending_cursor_value,
191 |                     last_successful_run_id=%s,
192 |                     pending_cursor_value=NULL,
193 |                     pending_run_id=NULL,
194 |                     updated_at=NOW()
195 |                 WHERE domain=%s AND dataset=%s AND source_application=%s
196 |                     AND pending_run_id::text=%s

```

```

197 |         """
198 |         (run_id, domain, dataset, source_application, run_id),
199 |     )
200 |     promoted = cur.rowcount == 1
201 |     self._conn.commit()
202 |     return promoted
203 |
204 | def clear_pending(
205 |     self,
206 |     *,
207 |     domain: str,
208 |     dataset: str,
209 |     source_application: str,
210 |     run_id: str,
211 | ) -> None:
212 |     """Discard the pending cursor without touching committed.
213 |
214 |     Called when dag_ingest failed for ``run_id``. The next poll re-issues
215 |     the same window, so records are preserved.
216 |     """
217 |     with self._conn.cursor() as cur:
218 |         cur.execute(
219 |             """
220 |             UPDATE pipeline.api_pull_watermark
221 |             SET pending_cursor_value=NULL,
222 |             pending_run_id=NULL,
223 |             updated_at=NOW()
224 |             WHERE domain=%s AND dataset=%s AND source_application=%s
225 |             AND pending_run_id::text=%s
226 |             """
227 |             (domain, dataset, source_application, run_id),
228 |         )
229 |     self._conn.commit()

```

16. ods_pipeline/ingest/api_pull_kafka/__init__.py

34 lines

```
1 | """Direct API → Kafka api_pull runner.
2 |
3 | Companion to ods_pipeline.ingest.api_pull. The file-pipeline shape
4 | polls and archives to S3 JSONL, then triggers dag_ingest. The
5 | direct-Kafka shape polls and publishes per-record Avro to the raw
6 | Kafka topic; a Kafka Connect S3 sink writes the archive in parallel
7 | and a JDBC sink writes Postgres rows. Same control-plane primitives
8 | (run_log, stages, lineage_edge, reconciliation_log,
9 | api_pull_watermark) – different downstream shape.
10 |
11 | Public entrypoints:
12 | * ``run_once`` performs one poll/publish cycle. The Airflow
13 |   dispatcher in ``dag_api_pull`` invokes it per scheduled tick.
14 | * ``run_loop`` wraps ``run_once`` in a long-running, stop-event
15 |   aware poller for sub-tick latency datasets. See
16 |   ``dag_api_pull_kafka_continuous`` for the Airflow surface.
17 | """
18 | from __future__ import annotations
19 |
20 | from ods_pipeline.ingest.api_pull_kafka.loop import run_loop
21 | from ods_pipeline.ingest.api_pull_kafka.runner import (
22 |     PublishedBatch,
23 |     build_avro_producer,
24 |     build_envelope,
25 |     run_once,
26 | )
27 |
28 | __all__ = [
29 |     "PublishedBatch",
30 |     "build_avro_producer",
31 |     "build_envelope",
32 |     "run_once",
33 |     "run_loop",
34 | ]
```

17. ods_pipeline/ingest/api_pull_kafka/loop.py

259 lines

```
1 | """Long-running runner mode for the direct-Kafka api_pull slice.
2 |
3 | The scheduled DAG (`dag_api_pull`) calls :func:`run_once` once per
4 | Airflow tick. For datasets where sub-tick latency matters, this module
5 | wraps `run_once` in a tight loop that respects an external stop
6 | signal (Airflow heartbeat, SIGTERM, etc).
7 |
8 | Design contract — see `docs/api-pull-direct-kafka-design.md` §
9 | Components → "Long-running poller":
10 |
11 | * One :func:`run_loop` invocation owns one `(domain, dataset)`.
12 | * Each iteration re-reads `committed_cursor_value` from the
13 |   watermark store so a parallel finalise step that promoted the
14 |   cursor is reflected immediately.
15 | * Transient errors from `run_once` (HTTP 5xx surfaced as
16 |   `RuntimeError`, transient kafka delivery `RuntimeErrors` are
17 |   swallowed and retried on the next tick. Config / validator errors
18 |   (`ValueError`, `KeyError`) are unrecoverable and propagate so
19 |   the operator restarts cleanly with a fresh config.
20 | * The producer (and HTTP session, when applicable) is reused across
21 |   iterations; `run_once` accepts an injected `producer` so we
22 |   only pay the broker handshake / Schema Registry round-trip once.
23 |
24 | The module deliberately does not import pycopg2 or
25 | `confluent_kafka` at module import time; both are produced via the
26 | factories supplied by the caller so unit tests stay infra-free.
27 | """
28 | from __future__ import annotations
29 |
30 | import logging
31 | import time
32 | import uuid
33 | from collections.abc import Callable, Mapping
34 | from datetime import datetime, timezone
35 | from threading import Event
36 | from typing import Any
37 |
38 | from ods_pipeline.ingest.api_pull_kafka.runner import (
39 |     PublishedBatch,
40 |     build_avro_producer,
41 |     run_once,
42 | )
43 |
44 | logger = logging.getLogger(__name__)
45 |
46 | # Sentinel exception types treated as unrecoverable. `run_once` raises
47 | # RuntimeError for transient kafka / HTTP issues; ValueError / KeyError /
48 | # TypeError indicate the dataset_config or schema is malformed and a
49 | # retry will not help.
50 | _FATAL_EXCEPTIONS: tuple[type[BaseException], ...] = (
51 |     ValueError,
52 |     KeyError,
53 |     TypeError,
54 | )
55 |
56 | _DEFAULT_POLL_INTERVAL_SECONDS = 5.0
57 |
58 |
59 | def _now_iso() -> str:
60 |     return datetime.now(timezone.utc).isoformat()
61 |
62 |
63 | def run_loop(
64 |     dataset_config: Mapping[str, Any],
65 |     *,
66 |     kafka_bootstrap: str,
67 |     schema_registry_url: str,
68 |     schema_str: str,
69 |     watermark_store_factory: Callable[[], Any],
70 |     stop_event: Event | None = None,
71 |     env: Mapping[str, str] | None = None,
72 |     sleep: Callable[[float], None] | None = None,
73 |     producer: Any | None = None,
74 | ) -> None:
75 |     """Run the direct-Kafka poller in a loop until `stop_event` fires.
76 |
77 |     Parameters
78 |     -----
79 |     dataset_config
80 |         Same shape consumed by :func:`run_once` plus the optional
81 |         `source.poll_interval_seconds` knob (default 5s) and the
82 |         `source.continuous` selector handled by the caller.
83 |     kafka_bootstrap, schema_registry_url, schema_str
84 |         Forwarded to the Avro producer factory on first iteration.
85 |     watermark_store_factory
86 |         Zero-arg callable returning an object with `read(...)` and
87 |         `record_pending(...)` methods compatible with
88 |         :class:`ods_pipeline.ingest.api_pull.WatermarkStore`. The
89 |         callable is invoked once per iteration so the underlying
90 |         pycopg2 connection can be refreshed if the loop runs for hours.
91 |     stop_event
92 |         :class:`threading.Event` set by the surrounding runtime
93 |         (Airflow operator, signal handler) to request a clean exit.
94 |     env
95 |         Forwarded to `run_once` for auth provider env-var lookup.
96 |     sleep
```



```

97 |         Injection point for unit tests – defaults to :func:`time.sleep`.
98 |     producer
99 |         Optional pre-built confluent_kafka producer. When omitted the
100 |         loop builds one from ``kafka_bootstrap`` / ``schema_registry_url``
101 |         / ``schema_str`` on the first iteration and reuses it for the
102 |         lifetime of the loop.
103 |
104 |     Raises
105 |     -----
106 |     ValueError, KeyError, TypeError
107 |         Propagated from ``run_once`` when configuration is malformed.
108 |         The supervising operator should treat these as unrecoverable
109 |         and not auto-restart without operator review.
110 |     """
111 |     domain = str(dataset_config["domain"])
112 |     dataset = str(dataset_config["dataset"])
113 |     source = dict(dataset_config.get("source") or {})
114 |     source_application = str(source.get("application", f"{domain}.{dataset}"))
115 |     cursor_style = str((source.get("cursor") or {}).get("style", "since_timestamp"))
116 |     poll_interval = float(
117 |         source.get("poll_interval_seconds", _DEFAULT_POLL_INTERVAL_SECONDS)
118 |     )
119 |     if poll_interval <= 0:
120 |         raise ValueError(
121 |             f"poll_interval_seconds must be positive, got {poll_interval!r}"
122 |         )
123 |
124 |     stop = stop_event if stop_event is not None else Event()
125 |     do_sleep = sleep if sleep is not None else time.sleep
126 |
127 |     if producer is None:
128 |         producer = build_avro_producer(
129 |             kafka_bootstrap=kafka_bootstrap,
130 |             schema_registry_url=schema_registry_url,
131 |             schema_str=schema_str,
132 |         )
133 |
134 |     iteration = 0
135 |     logger.info(
136 |         "api_pull_kafka.loop.start domain=%s dataset=%s poll_interval=%.2fs",
137 |         domain,
138 |         dataset,
139 |         poll_interval,
140 |     )
141 |
142 |     try:
143 |         while not stop.is_set():
144 |             iteration += 1
145 |             run_id = str(uuid.uuid4())
146 |             business_date = datetime.now(timezone.utc).date().isoformat()
147 |
148 |             store = watermark_store_factory()
149 |             try:
150 |                 watermark = store.read(
151 |                     domain=domain,
152 |                     dataset=dataset,
153 |                     source_application=source_application,
154 |                     cursor_type=cursor_style,
155 |                 )
156 |                 committed = getattr(watermark, "committed_cursor_value", None)
157 |             finally:
158 |                 # Best-effort close – store may wrap a connection.
159 |                 _maybe_close(store)
160 |
161 |             try:
162 |                 published: PublishedBatch = run_once(
163 |                     dataset_config=dataset_config,
164 |                     committed_cursor_value=committed,
165 |                     run_id=run_id,
166 |                     business_date=business_date,
167 |                     producer=producer,
168 |                     env=env,
169 |                 )
170 |             except _FATAL_EXCEPTIONS:
171 |                 logger.exception(
172 |                     "api_pull_kafka.loop.fatal domain=%s dataset=%s iter=%d",
173 |                     domain,
174 |                     dataset,
175 |                     iteration,
176 |                 )
177 |                 raise
178 |             except Exception as exc: # noqa: BLE001 – transient, retry next tick
179 |                 logger.warning(
180 |                     "api_pull_kafka.loop.transient domain=%s dataset=%s "
181 |                     "iter=%d error=%s",
182 |                     domain,
183 |                     dataset,
184 |                     iteration,
185 |                     exc,
186 |                 )
187 |             _wait(do_sleep, poll_interval, stop)
188 |             continue
189 |
190 |         if published.no_changes:
191 |             logger.info(
192 |                 "api_pull_kafka.loop.no_changes domain=%s dataset=%s iter=%d "
193 |                 "ts=%s",
194 |                 domain,
195 |                 dataset,
196 |                 iteration,

```

```

197 |         _now_iso(),
198 |     )
199 |     else:
200 |         if published.new_cursor_value is not None:
201 |             store2 = watermark_store_factory()
202 |             try:
203 |                 store2.record_pending(
204 |                     domain=domain,
205 |                     dataset=dataset,
206 |                     source_application=source_application,
207 |                     run_id=run_id,
208 |                     new_cursor_value=published.new_cursor_value,
209 |                 )
210 |             finally:
211 |                 _maybe_close(store2)
212 |             logger.info(
213 |                 "api_pull_kafka.loop.published domain=%s dataset=%s "
214 |                 "iter=%d records=%d topic=%s new_cursor=%s",
215 |                 domain,
216 |                 dataset,
217 |                 iteration,
218 |                 published.record_count,
219 |                 published.target_topic,
220 |                 published.new_cursor_value,
221 |             )
222 |
223 |         _wait(do_sleep, poll_interval, stop)
224 |     finally:
225 |         logger.info(
226 |             "api_pull_kafka.loop.exit domain=%s dataset=%s iterations=%d",
227 |             domain,
228 |             dataset,
229 |             iteration,
230 |         )
231 |         # Drain producer best-effort so in-flight messages aren't lost
232 |         # on operator shutdown. Producer creation is owned here when not
233 |         # injected, so we own teardown too.
234 |         try:
235 |             flush = getattr(producer, "flush", None)
236 |             if callable(flush):
237 |                 flush(timeout=5)
238 |         except Exception: # noqa: BLE001
239 |             pass
240 |
241 |
242 | def _wait(sleep_fn: Callable[[float], None], seconds: float, stop: Event) -> None:
243 |     """Sleep ``seconds`` but break early when ``stop`` is set.
244 |
245 |     The injected ``sleep_fn`` is normally :func:`time.sleep`; tests
246 |     pass a fake clock that records the requested durations.
247 |     """
248 |     if stop.is_set():
249 |         return
250 |     sleep_fn(seconds)
251 |
252 |
253 | def _maybe_close(store: Any) -> None:
254 |     closer = getattr(store, "close", None)
255 |     if callable(closer):
256 |         try:
257 |             closer()
258 |         except Exception: # noqa: BLE001
259 |             pass

```

18. ods_pipeline/ingest/api_pull_kafka/runner.py

352 lines

```
1 | """Direct API → Kafka api_pull runner – one poll/publish cycle.
2 |
3 | Reuses the polling primitives from ``ods_pipeline.ingest.api_pull``
4 | (:func:`build_auth`, :func:`build_cursor`) plus an idempotent +
5 | transactional Avro Kafka producer. Output is one Avro message per
6 | source record on the dataset's raw Kafka topic; the message envelope
7 | mirrors the file-pipeline ODS metadata block so downstream consumers
8 | (canonicalize, JDBC sink, S3 sink) see identical correlation fields.
9 |
10 | This module does not touch the Postgres control-plane. The DAG
11 | dispatcher (``dag_api_pull``) is responsible for ``run_log``,
12 | ``run_stage_log``, ``lineage_edge``, ``reconciliation_log`` and the
13 | ``api_pull_watermark`` lifecycle. Keeping I/O concerns split makes the
14 | runner unit-testable without psycopg2.
15 | """
16 | from __future__ import annotations
17 |
18 | import uuid
19 | from collections.abc import Mapping, Sequence
20 | from dataclasses import dataclass, field
21 | from typing import Any
22 |
23 | import requests
24 | from requests.adapters import HTTPAdapter
25 | from urllib3.util.retry import Retry
26 |
27 | from ods_pipeline import metadata as _metadata
28 | from ods_pipeline.ingest.api_pull.auth import AuthProvider, build_auth
29 | from ods_pipeline.ingest.api_pull.cursors import Cursor, CursorRequest, build_cursor
30 |
31 | # -----
32 | # Result type
33 | # -----
34 |
35 | @dataclass
36 | class PublishedBatch:
37 |     """Outcome of one direct-Kafka poll/publish cycle.
38 |
39 |     ``no_changes=True`` means the source returned 0 records or 304 Not
40 |     Modified. The DAG should mark the run skipped, not advance the
41 |     watermark, and skip the produce step's recon and lineage writes.
42 |     """
43 |
44 |     domain: str
45 |     dataset: str
46 |     source_application: str
47 |     run_id: str
48 |     target_topic: str
49 |     record_count: int
50 |     page_count: int
51 |     old_cursor_value: str | None
52 |     new_cursor_value: str | None
53 |     source_request_id: str
54 |     offset_start_by_partition: dict[int, int] = field(default_factory=dict)
55 |     offset_end_by_partition: dict[int, int] = field(default_factory=dict)
56 |     no_changes: bool = False
57 |
58 |
59 | # -----
60 | # HTTP session
61 | # -----
62 |
63 |
64 | def _build_session(*, auth: AuthProvider, timeout_seconds: float, retries: int) -> requests.Session:
65 |     session = requests.Session()
66 |     retry = Retry(
67 |         total=retries,
68 |         backoff_factor=0.5,
69 |         status_forcelist=(429, 500, 502, 503, 504),
70 |         allowed_methods=frozenset({"GET", "HEAD"}),
71 |         raise_on_status=False,
72 |     )
73 |     adapter = HTTPAdapter(max_retries=retry)
74 |     session.mount("http://", adapter)
75 |     session.mount("https://", adapter)
76 |     session.headers["Accept"] = "application/json"
77 |     original = session.request
78 |
79 |     def _with_timeout(*args, **kwargs):
80 |         kwargs.setdefault("timeout", timeout_seconds)
81 |         return original(*args, **kwargs)
82 |
83 |     session.request = _with_timeout # type: ignore[assignment]
84 |     auth.apply(session)
85 |     return session
86 |
87 |
88 |
89 | def _records_from_body(body: Any) -> Sequence[Mapping[str, Any]]:
90 |     """Accept ``[{...}]`` or ``{"items": [...]}`` etc. Same semantics as
91 |     the file-pipeline poller so wire-shape changes are not required when
92 |     a dataset is moved between delivery modes."""
93 |     if isinstance(body, list):
94 |         return body
95 |     if isinstance(body, dict):
96 |         for key in ("items", "data", "records", "results"):

```

```

97 |         value = body.get(key)
98 |         if isinstance(value, list):
99 |             return value
100 |     return []
101 |
102 |
103 | # -----
104 | # Envelope
105 | # -----
106 |
107 |
108 | def build_envelope(
109 |     *,
110 |     record: Mapping[str, Any],
111 |     run_id: str,
112 |     domain: str,
113 |     dataset: str,
114 |     source_application: str,
115 |     source_request_id: str,
116 |     business_date: str,
117 |     cursor_value: str | None,
118 |     schema_id: str,
119 |     schema_version: int | str,
120 | ) -> dict[str, Any]:
121 |     """Build the per-record Avro envelope.
122 |
123 |     Top-level fields are the source record keys (flat). ODS metadata
124 |     fields are stamped alongside so the JDBC sink can persist origin
125 |     correlation without nested types.
126 |     """
127 |     meta = _metadata.message_metadata(
128 |         run_id=run_id,
129 |         domain=domain,
130 |         dataset=dataset,
131 |         source_application=source_application,
132 |         source_request_id=source_request_id,
133 |     )
134 |     envelope: dict[str, Any] = {**dict(record)}
135 |     envelope["_ods_run_id"] = run_id
136 |     envelope["_ods_business_date"] = business_date
137 |     envelope["_ods_source_request_id"] = meta.get("_ods_source_request_id")
138 |     envelope["_ods_source_message_id"] = meta.get("_ods_source_message_id")
139 |     envelope["_ods_source_event_id"] = meta.get("_ods_source_event_id")
140 |     envelope["_ods_source_batch_id"] = meta.get("_ods_source_batch_id")
141 |     envelope["_ods_source_application"] = source_application
142 |     envelope["_ods_source_cursor"] = cursor_value
143 |     envelope["_ods_archive_s3_uri"] = None # written by the S3 sink later
144 |     envelope["_ods_domain"] = domain
145 |     envelope["_ods_dataset"] = dataset
146 |     envelope["_ods_file_id"] = None # synthetic file_id is the DAG's job
147 |     envelope["_ods_ingested_at"] = meta.get("_ods_ingested_at")
148 |     envelope["_ods_schema_id"] = schema_id
149 |     envelope["_ods_schema_version"] = (
150 |         int(schema_version) if isinstance(schema_version, (int, str)) and str(schema_version).isdigit()
151 |         else None
152 |     )
153 |     envelope["_ods_kafka_partition"] = None # filled in by delivery callback
154 |     envelope["_ods_kafka_offset"] = None
155 |     return envelope
156 |
157 |
158 | # -----
159 | # Producer
160 | # -----
161 |
162 |
163 | def build_avro_producer(
164 |     *,
165 |     kafka_bootstrap: str,
166 |     schema_registry_url: str,
167 |     schema_str: str,
168 |     transactional_id: str | None = None,
169 | ):
170 |     """Construct an idempotent Avro :class:`SerializingProducer`.
171 |
172 |     Lazy import so the runner module imports cleanly in environments
173 |     without confluent_kafka (unit tests use the injected ``producer``
174 |     parameter on :func:`run_once`).
175 |     """
176 |     from confluent_kafka import SerializingProducer
177 |     from confluent_kafka.schema_registry import SchemaRegistryClient
178 |     from confluent_kafka.schema_registry.avro import AvroSerializer
179 |     from confluent_kafka.serialization import StringSerializer
180 |
181 |     registry = SchemaRegistryClient({"url": schema_registry_url})
182 |     avro_serializer = AvroSerializer(registry, schema_str)
183 |     config = {
184 |         "bootstrap.servers": kafka_bootstrap,
185 |         "key.serializer": StringSerializer("utf_8"),
186 |         "value.serializer": avro_serializer,
187 |         "enable.idempotence": True,
188 |         "acks": "all",
189 |         "max.in.flight.requests.per.connection": 5,
190 |         "linger.ms": 5,
191 |     }
192 |     if transactional_id:
193 |         config["transactional.id"] = transactional_id
194 |     return SerializingProducer(config)
195 |
196 |

```

```

197 | # -----
198 | # run_once
199 | # -----
200 |
201 |
202 | def run_once(
203 |     *,
204 |     dataset_config: Mapping[str, Any],
205 |     kafka_bootstrap: str | None = None,
206 |     schema_registry_url: str | None = None,
207 |     schema_str: str | None = None,
208 |     committed_cursor_value: str | None,
209 |     run_id: str,
210 |     business_date: str,
211 |     session: requests.Session | None = None,
212 |     cursor: Cursor | None = None,
213 |     producer: Any = None,
214 |     env: Mapping[str, str] | None = None,
215 | ) -> PublishedBatch:
216 |     """Execute one poll/publish cycle.
217 |
218 |     Walks pages via the configured cursor, builds Avro envelopes per
219 |     record, produces them to the dataset's raw topic, and waits for
220 |     delivery. Returns the per-partition offset window and the new
221 |     cursor value. If the source returns no records, returns
222 |     ``no_changes=True`` and produces nothing.
223 |
224 |     Injectables (``session``, ``cursor``, ``producer``) keep the runner
225 |     unit-testable without HTTP / Kafka / Schema Registry.
226 |     """
227 |     domain = str(dataset_config["domain"])
228 |     dataset = str(dataset_config["dataset"])
229 |     source = dict(dataset_config.get("source") or {})
230 |     source_application = str(source.get("application", f"{domain}.{dataset}"))
231 |     target_topic = str(dataset_config["target_topic"])
232 |     schema_id = str(dataset_config.get("schema_id", f"{domain}.{dataset}"))
233 |     schema_version = dataset_config.get("schema_version", 1)
234 |     timeout_seconds = float(source.get("timeout_seconds", 30))
235 |     retries = int(source.get("retries", 3))
236 |
237 |     if session is None:
238 |         auth = build_auth(source.get("auth"), env=env)
239 |         session = _build_session(
240 |             auth=auth,
241 |             timeout_seconds=timeout_seconds,
242 |             retries=retries,
243 |         )
244 |     if cursor is None:
245 |         cursor = build_cursor(source, committed_value=committed_cursor_value)
246 |
247 |     source_request_id = str(uuid.uuid4())
248 |     all_records: list[Mapping[str, Any]] = []
249 |     request: CursorRequest | None = cursor.initial_request()
250 |     page_count = 0
251 |
252 |     while request is not None:
253 |         response = session.get(request.url, params=request.params or None)
254 |         page_count += 1
255 |         if response.status_code == 304:
256 |             break
257 |         response.raise_for_status()
258 |         body = response.json() if response.content else None
259 |         all_records.extend(_records_from_body(body))
260 |         request = cursor.next_request(response.headers, body)
261 |
262 |     new_cursor_value = cursor.advance(all_records)
263 |
264 |     if not all_records:
265 |         return PublishedBatch(
266 |             domain=domain,
267 |             dataset=dataset,
268 |             source_application=source_application,
269 |             run_id=run_id,
270 |             target_topic=target_topic,
271 |             record_count=0,
272 |             page_count=page_count,
273 |             old_cursor_value=committed_cursor_value,
274 |             new_cursor_value=None,
275 |             source_request_id=source_request_id,
276 |             no_changes=True,
277 |         )
278 |
279 |     if producer is None:
280 |         if not (kafka_bootstrap and schema_registry_url and schema_str):
281 |             raise ValueError(
282 |                 "producer must be supplied OR kafka_bootstrap + "
283 |                 "schema_registry_url + schema_str must be provided"
284 |             )
285 |         producer = build_avro_producer(
286 |             kafka_bootstrap=kafka_bootstrap,
287 |             schema_registry_url=schema_registry_url,
288 |             schema_str=schema_str,
289 |         )
290 |
291 |     delivered_offsets: dict[int, int] = {}
292 |     delivered_starts: dict[int, int] = {}
293 |     delivery_errors: list[str] = []
294 |
295 |     def _on_delivery(err, msg):
296 |         if err is not None:

```

```

297 |         delivery_errors.append(str(err))
298 |         return
299 |     partition = int(msg.partition())
300 |     offset = int(msg.offset())
301 |     delivered_starts.setdefault(partition, offset)
302 |     if partition not in delivered_offsets or delivered_offsets[partition] < offset:
303 |         delivered_offsets[partition] = offset
304 |
305 |     for record in all_records:
306 |         envelope = build_envelope(
307 |             record=record,
308 |             run_id=run_id,
309 |             domain=domain,
310 |             dataset=dataset,
311 |             source_application=source_application,
312 |             source_request_id=source_request_id,
313 |             business_date=business_date,
314 |             cursor_value=committed_cursor_value,
315 |             schema_id=schema_id,
316 |             schema_version=schema_version,
317 |         )
318 |         producer.produce(
319 |             topic=target_topic,
320 |             key=str(envelope["_ods_run_id"]),
321 |             value=envelope,
322 |             on_delivery=_on_delivery,
323 |         )
324 |
325 |     remaining = producer.flush(timeout=30)
326 |     if remaining:
327 |         raise RuntimeError(
328 |             f"{remaining} kafka message(s) not delivered for run_id={run_id}"
329 |         )
330 |     if delivery_errors:
331 |         raise RuntimeError(
332 |             f"kafka delivery errors for run_id={run_id}: {delivery_errors[:3]}"
333 |         )
334 |
335 |     # End-offset = max delivered offset + 1 (next-message convention).
336 |     offset_end = {p: o + 1 for p, o in delivered_offsets.items()}
337 |
338 |     return PublishedBatch(
339 |         domain=domain,
340 |         dataset=dataset,
341 |         source_application=source_application,
342 |         run_id=run_id,
343 |         target_topic=target_topic,
344 |         record_count=len(all_records),
345 |         page_count=page_count,
346 |         old_cursor_value=committed_cursor_value,
347 |         new_cursor_value=new_cursor_value,
348 |         source_request_id=source_request_id,
349 |         offset_start_by_partition=dict(delivered_starts),
350 |         offset_end_by_partition=offset_end,
351 |         no_changes=False,
352 |     )

```

19. ods_pipeline/lineage.py

46 lines

```
1 | """pipeline.lineage_edge operations."""
2 | from __future__ import annotations
3 |
4 |
5 | def write_edge(
6 |     conn,
7 |     *,
8 |     child_run_id: str,
9 |     edge_type: str,
10 |     parent_run_id: str | None = None,
11 |     parent_file_id: str | None = None,
12 |     source_ref: str | None = None,
13 |     target_ref: str | None = None,
14 |     record_count: int | None = None,
15 | ) -> None:
16 |     """Insert one row into ``pipeline.lineage_edge``.
17 |
18 |     Either ``parent_run_id`` or ``parent_file_id`` (or both) should be supplied.
19 |
20 |     Common *edge_type* values (use these constants in callers):
21 |     * ``"raw_to_curated"`` - S3 raw → S3 curated (written by ingestion job)
22 |     * ``"curated_to_kafka"`` - S3 curated → Kafka topic (written by publish job)
23 |     * ``"curated_to_postgres"`` - S3 curated → Postgres table
24 |     """
25 |     if parent_run_id is None and parent_file_id is None:
26 |         raise ValueError(
27 |             "write_edge requires at least one of parent_run_id or parent_file_id"
28 |         )
29 |     try:
30 |         with conn.cursor() as cur:
31 |             cur.execute(
32 |                 """
33 |                 INSERT INTO pipeline.lineage_edge
34 |                 (child_run_id, parent_run_id, parent_file_id,
35 |                  edge_type, source_ref, target_ref, record_count)
36 |                 VALUES (%s,%s,%s, %s,%s,%s,%s)
37 |                 """,
38 |                 (
39 |                     child_run_id, parent_run_id, parent_file_id,
40 |                     edge_type, source_ref, target_ref, record_count,
41 |                 ),
42 |             )
43 |         conn.commit()
44 |     except Exception:
45 |         conn.rollback()
46 |         raise
```

20. ods_pipeline/messages.py

308 lines

```
1 | """Control-plane helpers for message/API ingestion flows.
2 |
3 | Stateless contract (since 2026-05-07)
4 | -----
5 |
6 | Both :func:`start_run` and :func:`record_result` commit per write – the
7 | caller no longer wraps them in a transaction. The strict write order in
8 | ``record_result`` (stages → archive → reconciliation → run.status) means
9 | the dashboard never observes a ``status='succeeded'`` run without its
10 | proof rows already landed: succeeded is the LAST commit.
11 |
12 | For mid-flight failures the caller's own ``except`` block (or the
13 | :func:`ods_pipeline.stages.stage_scope` context manager) is responsible
14 | for writing the ``stage_failed`` row and calling
15 | :func:`ods_pipeline.runs.update` with ``status='failed'``. If even those
16 | handlers don't run (process killed mid-flight), the heartbeat-staleness
17 | janitor (:code:`airflow.dags.dag_run_janitor`) closes the orphan row.
18 | """
19 | from __future__ import annotations
20 |
21 | import json
22 | from collections.abc import Mapping
23 | from typing import Any
24 |
25 | from ods_pipeline import metadata, reconciliation, runs, stages
26 | from ods_pipeline.models import PATTERN_CORRELATION_FIELD, PatternType, Stage, StageEvent
27 |
28 |
29 | def _caller_must_rollback_on_exception() -> None:
30 |     """Caller-managed transaction contract.
31 |
32 |     ``record_result`` (below) calls ``runs.update``, ``stages.write``,
33 |     ``stages.finish``, and ``reconciliation.write_check`` with
34 |     ``commit=False``. The CALLER MUST wrap the invocation in a try/except
35 |     and call ``conn.rollback()`` on exception, otherwise the open
36 |     transaction stays open and partially-written rows remain in pg_locks.
37 |
38 |     Recommended pattern::
39 |
40 |         try:
41 |             messages.record_result(conn, ...)
42 |             conn.commit()
43 |         except Exception:
44 |             conn.rollback()
45 |             raise
46 |     """
47 |
48 |
49 | def correlate(
50 |     message: Mapping[str, Any],
51 |     context: Mapping[str, Any],
52 |     *,
53 |     pattern_type: str,
54 | ) -> bool:
55 |     """Return True if ``message`` belongs to the given ``context`` for ``pattern_type``.
56 |
57 |     Each ingestion pattern has its own correlation field (see
58 |     :code:`data: ods_pipeline.models.PATTERN_CORRELATION_FIELD`). The function:
59 |
60 |     - looks up the field for the pattern,
61 |     - returns True if both message and context carry the same non-empty value,
62 |     - returns True when context has no correlation set (broadcast / "all
63 |       messages of this pattern"),
64 |     - falls back to ``ods_run_id`` cross-check for the FILE pattern only,
65 |       preserving the legacy two-key match in
66 |       :code:`func:`glue.jobs.canonicalize.matches_context`.
67 |
68 |     Raises ``ValueError`` for unknown ``pattern_type``.
69 |     """
70 |     if pattern_type not in PatternType.ALL:
71 |         raise ValueError(
72 |             f"unknown pattern_type {pattern_type!r}; "
73 |             f"expected one of {sorted(PatternType.ALL)}"
74 |         )
75 |     field = PATTERN_CORRELATION_FIELD[pattern_type]
76 |     ctx_value = context.get(field) or context.get(field.removeprefix("_ods_"))
77 |     msg_value = message.get(field)
78 |
79 |     if pattern_type == PatternType.FILE:
80 |         # Legacy: file pattern allows correlation by run_id as a secondary key.
81 |         ctx_run = context.get("_ods_run_id") or context.get("run_id") or context.get("parent_run_id")
82 |         msg_run = message.get("_ods_run_id")
83 |         if ctx_value and msg_value and str(ctx_value) == str(msg_value):
84 |             return True
85 |         if ctx_run and msg_run and str(ctx_run) == str(msg_run):
86 |             return True
87 |         # No context = broadcast.
88 |         return not ctx_value and not ctx_run
89 |
90 |     if not ctx_value:
91 |         return True # broadcast / no correlation set
92 |     return msg_value is not None and str(ctx_value) == str(msg_value)
93 |
94 |
95 | def _correlation_metadata(
96 |     *,
```



```

97 |     run_id: str,
98 |     domain: str,
99 |     dataset: str,
100 |     source_application: str,
101 |     correlation: Mapping[str, Any],
102 | ) -> dict[str, Any]:
103 |     message_meta = metadata.message_metadata(
104 |         run_id=run_id,
105 |         domain=domain,
106 |         dataset=dataset,
107 |         source_application=source_application,
108 |         source_message_id=correlation.get("_ods_source_message_id")
109 |         or correlation.get("source_message_id"),
110 |         source_event_id=correlation.get("_ods_source_event_id")
111 |         or correlation.get("source_event_id"),
112 |         source_request_id=correlation.get("_ods_source_request_id")
113 |         or correlation.get("source_request_id"),
114 |         source_batch_id=correlation.get("_ods_source_batch_id")
115 |         or correlation.get("source_batch_id"),
116 |     )
117 |     return {
118 |         key: message_meta.get(key)
119 |         for key in metadata.MESSAGE_CORRELATION_FIELDS
120 |         if message_meta.get(key)
121 |     }
122 |
123 |
124 | def start_run(
125 |     conn,
126 |     *,
127 |     run_id: str,
128 |     domain: str,
129 |     dataset: str,
130 |     source_application: str,
131 |     correlation: Mapping[str, Any],
132 |     business_date: str | None = None,
133 |     kafka_topic: str | None = None,
134 |     expected_count: int | None = None,
135 |     parents: list[dict[str, Any]] | None = None,
136 | ) -> dict[str, Any]:
137 |     """Start a message/API run and open the receive stage.
138 |
139 |     ``correlation`` must include at least one of:
140 |     ``source_message_id``, ``source_event_id``, ``source_request_id``, or
141 |     ``source_batch_id``. The helper intentionally writes only existing
142 |     pipeline tables; payload storage remains S3/Kafka responsibility.
143 |     """
144 |     correlation_meta = _correlation_metadata(
145 |         run_id=run_id,
146 |         domain=domain,
147 |         dataset=dataset,
148 |         source_application=source_application,
149 |         correlation=correlation,
150 |     )
151 |     parent_payload = list(parents or [])
152 |     parent_payload.append({
153 |         "edge_type": "message_correlation",
154 |         "source_application": source_application,
155 |         **correlation_meta,
156 |     })
157 |     runs.start(
158 |         conn,
159 |         run_id=run_id,
160 |         pipeline_type="message_api",
161 |         domain=domain,
162 |         dataset=dataset,
163 |         business_date=business_date,
164 |         kafka_topic=kafka_topic,
165 |         parents=parent_payload,
166 |     )
167 |     stages.start(
168 |         conn,
169 |         run_id=run_id,
170 |         stage=Stage.MESSAGE_RECEIVE,
171 |         record_count_in=expected_count,
172 |         metrics={
173 |             "source_application": source_application,
174 |             "correlation": correlation_meta,
175 |         },
176 |     )
177 |     return correlation_meta
178 |
179 |
180 | def record_result(
181 |     conn,
182 |     *,
183 |     run_id: str,
184 |     domain: str,
185 |     dataset: str,
186 |     source_count: int,
187 |     published_count: int,
188 |     validation_fail_count: int = 0,
189 |     dlq_count: int = 0,
190 |     archive_count: int | None = None,
191 |     business_date: str | None = None,
192 |     kafka_topic: str | None = None,
193 |     dlq_ref: str | None = None,
194 |     archive_ref: str | None = None,
195 |     extra_detail: Mapping[str, Any] | None = None,
196 | ) -> str:

```

```

197 | """Close a message/API run with count reconciliation and stage facts.
198 |
199 | Stateless write order (2026-05-07): every helper call below commits
200 | independently. Order matters – recon row writes BEFORE the run flips
201 | to terminal status so the dashboard rule "succeeded ⇒ recon row
202 | present" always holds. If the process dies mid-call the next-minute
203 | janitor closes the still-running row; the partial stage rows that
204 | landed remain durable so operators can see exactly where the run
205 | stopped.
206 | """
207 | accepted_count = int(source_count) - int(validation_fail_count) - int(dlq_count)
208 | discrepancy = int(published_count) - accepted_count
209 | archive_discrepancy = (
210 |     None if archive_count is None else int(archive_count) - int(source_count)
211 | )
212 | ok = discrepancy == 0 and (archive_discrepancy in (None, 0))
213 | status = "succeeded" if ok else "failed"
214 |
215 | stages.finish(
216 |     conn,
217 |     run_id=run_id,
218 |     stage=Stage.MESSAGE_RECEIVE,
219 |     status="succeeded",
220 |     event_type=StageEvent.COMPLETED,
221 |     record_count_in=source_count,
222 |     record_count_out=source_count,
223 |     commit=True,
224 | )
225 | stages.write(
226 |     conn,
227 |     run_id=run_id,
228 |     stage=Stage.MESSAGE_VALIDATE,
229 |     status="warned" if validation_fail_count else "succeeded",
230 |     event_type=StageEvent.WARNED if validation_fail_count else StageEvent.COMPLETED,
231 |     record_count_in=source_count,
232 |     record_count_out=source_count - validation_fail_count,
233 |     metrics={"validation_fail_count": validation_fail_count},
234 |     commit=True,
235 | )
236 | if dlq_count:
237 |     stages.write(
238 |         conn,
239 |         run_id=run_id,
240 |         stage=Stage.DLQ_WRITE,
241 |         status="succeeded",
242 |         event_type=StageEvent.COMPLETED,
243 |         output_ref=dlq_ref,
244 |         record_count_in=dlq_count,
245 |         record_count_out=dlq_count,
246 |         commit=True,
247 |     )
248 | if archive_count is not None:
249 |     stages.write(
250 |         conn,
251 |         run_id=run_id,
252 |         stage=Stage.MESSAGE_ARCHIVE,
253 |         status="succeeded" if archive_discrepancy == 0 else "failed",
254 |         event_type=StageEvent.COMPLETED if archive_discrepancy == 0 else StageEvent.FAILED,
255 |         output_ref=archive_ref,
256 |         record_count_in=source_count,
257 |         record_count_out=archive_count,
258 |         error=None if archive_discrepancy == 0 else f"archive discrepancy={archive_discrepancy}",
259 |         commit=True,
260 |     )
261 |
262 | detail = {
263 |     "source_count": source_count,
264 |     "validation_fail_count": validation_fail_count,
265 |     "dlq_count": dlq_count,
266 |     "accepted_count": accepted_count,
267 |     "published_count": published_count,
268 |     "archive_count": archive_count,
269 |     "archive_discrepancy": archive_discrepancy,
270 |     **dict(extra_detail or {}),
271 | }
272 | reconciliation.write_check(
273 |     conn,
274 |     check_type="message_batch_count",
275 |     run_id=run_id,
276 |     domain=domain,
277 |     dataset=dataset,
278 |     business_date=business_date,
279 |     source_count=accepted_count,
280 |     kafka_count=published_count,
281 |     status="ok" if ok else "failed",
282 |     detail=json.dumps(detail, sort_keys=True),
283 |     commit=True,
284 | )
285 | stages.write(
286 |     conn,
287 |     run_id=run_id,
288 |     stage=Stage.RECON_MESSAGE,
289 |     status="succeeded" if ok else "failed",
290 |     event_type=StageEvent.COMPLETED if ok else StageEvent.FAILED,
291 |     record_count_in=accepted_count,
292 |     record_count_out=published_count,
293 |     metrics=detail,
294 |     error=None if ok else f"message reconciliation mismatch={discrepancy}",
295 |     commit=True,
296 | )

```

```
297 |     runs.update(  
298 |         conn,  
299 |         run_id,  
300 |         commit=True,  
301 |         status=status,  
302 |         record_count_source=source_count,  
303 |         record_count_dq_fail=validation_fail_count + dlq_count,  
304 |         record_count_published=published_count,  
305 |         kafka_topic=kafka_topic,  
306 |         error_summary=None if ok else "message/API reconciliation failed",  
307 |     )  
308 |     return status
```

21. ods_pipeline/metadata.py

267 lines

```
1 | """ODS metadata contracts for file, message, canonical, and archive records."""
2 | from __future__ import annotations
3 |
4 | from datetime import date, datetime, timezone
5 | from typing import Any, Mapping
6 |
7 | FILE_RECORD_FIELDS: tuple[str, ...] = (
8 |     "_ods_file_id",
9 |     "_ods_run_id",
10 |     "_ods_domain",
11 |     "_ods_dataset",
12 |     "_ods_business_date",
13 |     "_ods_source_application",
14 |     "_ods_ingested_at",
15 | )
16 |
17 | MESSAGE_CORRELATION_FIELDS: tuple[str, ...] = (
18 |     "_ods_source_message_id",
19 |     "_ods_source_event_id",
20 |     "_ods_source_request_id",
21 |     "_ods_source_batch_id",
22 | )
23 |
24 | MESSAGE_RECORD_FIELDS: tuple[str, ...] = (
25 |     *MESSAGE_CORRELATION_FIELDS,
26 |     "_ods_run_id",
27 |     "_ods_domain",
28 |     "_ods_dataset",
29 |     "_ods_source_application",
30 |     "_ods_ingested_at",
31 | )
32 |
33 | CANONICAL_FILE_RECORD_FIELDS: tuple[str, ...] = (
34 |     "_ods_file_id",
35 |     "_ods_raw_run_id",
36 |     "_ods_canonicalize_run_id",
37 |     "_ods_domain",
38 |     "_ods_dataset",
39 |     "_ods_business_date",
40 |     "_ods_source_application",
41 |     "_ods_ingested_at",
42 | )
43 |
44 | CANONICAL_MESSAGE_RECORD_FIELDS: tuple[str, ...] = (
45 |     *MESSAGE_CORRELATION_FIELDS,
46 |     "_ods_raw_run_id",
47 |     "_ods_canonicalize_run_id",
48 |     "_ods_domain",
49 |     "_ods_dataset",
50 |     "_ods_source_application",
51 |     "_ods_ingested_at",
52 | )
53 |
54 | HISTORY_TABLE_FIELDS: tuple[str, ...] = (
55 |     "_ods_run_id",
56 |     "_ods_business_date",
57 |     "_ods_ingested_at",
58 | )
59 |
60 | FILE_HISTORY_TABLE_FIELDS: tuple[str, ...] = (
61 |     *HISTORY_TABLE_FIELDS,
62 |     "_ods_file_id",
63 | )
64 |
65 | MESSAGE_HISTORY_TABLE_FIELDS: tuple[str, ...] = (
66 |     *HISTORY_TABLE_FIELDS,
67 |     *MESSAGE_CORRELATION_FIELDS,
68 | )
69 |
70 | ARCHIVE_ENVELOPE_FIELDS: tuple[str, ...] = (
71 |     *MESSAGE_CORRELATION_FIELDS,
72 |     "_ods_source_application",
73 |     "_ods_domain",
74 |     "_ods_dataset",
75 |     "_ods_ingested_at",
76 |     "_ods_schema_id",
77 |     "_ods_schema_version",
78 |     "_ods_run_id",
79 |     "payload",
80 | )
81 |
82 |
83 | def utc_now_iso() -> str:
84 |     """Return an ISO-8601 UTC timestamp for ODS metadata fields."""
85 |     return datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")
86 |
87 |
88 | def normalise_value(value: Any) -> Any:
89 |     """Normalise common Python values into JSON/Avro friendly metadata values."""
90 |     if isinstance(value, datetime):
91 |         return value.astimezone(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")
92 |     if isinstance(value, date):
93 |         return value.isoformat()
94 |     return value
95 |
96 |
```

```

97 | def missing_fields(record: Mapping[str, Any], required_fields: tuple[str, ...]) -> list[str]:
98 |     """Return required fields that are absent, ``None``, or blank strings."""
99 |     missing: list[str] = []
100 |     for field in required_fields:
101 |         value = record.get(field)
102 |         if value is None or value == "":
103 |             missing.append(field)
104 |     return missing
105 |
106 |
107 | def require_fields(
108 |     record: Mapping[str, Any],
109 |     required_fields: tuple[str, ...],
110 |     *,
111 |     context: str = "record",
112 | ) -> None:
113 |     """Raise ``ValueError`` if *record* misses any required ODS metadata fields."""
114 |     missing = missing_fields(record, required_fields)
115 |     if missing:
116 |         raise ValueError(f"{context} missing required ODS metadata fields: {missing}")
117 |
118 |
119 | def require_message_correlation(record: Mapping[str, Any], *, context: str = "message") -> None:
120 |     """Require at least one stable source correlation key for message/API records."""
121 |     if not any(record.get(field) for field in MESSAGE_CORRELATION_FIELDS):
122 |         raise ValueError(
123 |             f"{context} must include at least one source correlation key: "
124 |             f"{list(MESSAGE_CORRELATION_FIELDS)}"
125 |         )
126 |
127 |
128 | def file_metadata(
129 |     *,
130 |     file_id: str,
131 |     run_id: str,
132 |     domain: str,
133 |     dataset: str,
134 |     business_date: str | date,
135 |     source_application: str,
136 |     ingested_at: str | datetime | None = None,
137 | ) -> dict[str, Any]:
138 |     """Build the standard metadata block for file based records."""
139 |     metadata = {
140 |         "_ods_file_id": file_id,
141 |         "_ods_run_id": run_id,
142 |         "_ods_domain": domain,
143 |         "_ods_dataset": dataset,
144 |         "_ods_business_date": normalise_value(business_date),
145 |         "_ods_source_application": source_application,
146 |         "_ods_ingested_at": normalise_value(ingested_at) if ingested_at else utc_now_iso(),
147 |     }
148 |     require_fields(metadata, FILE_RECORD_FIELDS, context="file metadata")
149 |     return metadata
150 |
151 |
152 | def message_metadata(
153 |     *,
154 |     run_id: str,
155 |     domain: str,
156 |     dataset: str,
157 |     source_application: str,
158 |     source_message_id: str | None = None,
159 |     source_event_id: str | None = None,
160 |     source_request_id: str | None = None,
161 |     source_batch_id: str | None = None,
162 |     ingested_at: str | datetime | None = None,
163 | ) -> dict[str, Any]:
164 |     """Build the standard metadata block for message/API records."""
165 |     metadata = {
166 |         "_ods_source_message_id": source_message_id,
167 |         "_ods_source_event_id": source_event_id,
168 |         "_ods_source_request_id": source_request_id,
169 |         "_ods_source_batch_id": source_batch_id,
170 |         "_ods_run_id": run_id,
171 |         "_ods_domain": domain,
172 |         "_ods_dataset": dataset,
173 |         "_ods_source_application": source_application,
174 |         "_ods_ingested_at": normalise_value(ingested_at) if ingested_at else utc_now_iso(),
175 |     }
176 |     require_fields(
177 |         metadata,
178 |         ("_ods_run_id", "_ods_domain", "_ods_dataset", "_ods_source_application", "_ods_ingested_at"),
179 |         context="message metadata",
180 |     )
181 |     require_message_correlation(metadata)
182 |     return metadata
183 |
184 |
185 | def canonical_metadata(
186 |     source_metadata: Mapping[str, Any],
187 |     *,
188 |     canonicalize_run_id: str,
189 |     raw_run_id: str | None = None,
190 | ) -> dict[str, Any]:
191 |     """Preserve source correlation metadata and add canonicalization run metadata."""
192 |     metadata = {
193 |         key: source_metadata.get(key)
194 |         for key in (
195 |             "_ods_file_id",
196 |             *MESSAGE_CORRELATION_FIELDS,

```

```

197 |         "_ods_domain",
198 |         "_ods_dataset",
199 |         "_ods_business_date",
200 |         "_ods_source_application",
201 |         "_ods_ingested_at",
202 |     )
203 |     if source_metadata.get(key) is not None
204 | }
205 | metadata["_ods_raw_run_id"] = raw_run_id or source_metadata.get("_ods_run_id")
206 | metadata["_ods_canonicalize_run_id"] = canonicalize_run_id
207 | if metadata.get("_ods_file_id"):
208 |     require_fields(metadata, CANONICAL_FILE_RECORD_FIELDS, context="canonical file metadata")
209 | else:
210 |     require_message_correlation(metadata, context="canonical message metadata")
211 |     require_fields(
212 |         metadata,
213 |         (
214 |             "_ods_raw_run_id",
215 |             "_ods_canonicalize_run_id",
216 |             "_ods_domain",
217 |             "_ods_dataset",
218 |             "_ods_source_application",
219 |             "_ods_ingested_at",
220 |         ),
221 |         context="canonical message metadata",
222 |     )
223 | return metadata
224 |
225 |
226 | def archive_envelope(
227 |     *,
228 |     payload: Mapping[str, Any],
229 |     metadata: Mapping[str, Any],
230 |     schema_id: str,
231 |     schema_version: int | str,
232 |     archive_s3_uri: str | None = None,
233 | ) -> dict[str, Any]:
234 |     """Build a JSONL-friendly S3 archive envelope for complex events."""
235 |     envelope = {
236 |         key: metadata.get(key)
237 |         for key in (
238 |             *MESSAGE_CORRELATION_FIELDS,
239 |             "_ods_source_application",
240 |             "_ods_domain",
241 |             "_ods_dataset",
242 |             "_ods_ingested_at",
243 |             "_ods_run_id",
244 |         )
245 |         if metadata.get(key) is not None
246 |     }
247 |     envelope["_ods_schema_id"] = schema_id
248 |     envelope["_ods_schema_version"] = schema_version
249 |     if archive_s3_uri is not None:
250 |         envelope["_ods_archive_s3_uri"] = archive_s3_uri
251 |     envelope["payload"] = dict(payload)
252 |     require_message_correlation(envelope, context="archive envelope")
253 |     require_fields(
254 |         envelope,
255 |         (
256 |             "_ods_source_application",
257 |             "_ods_domain",
258 |             "_ods_dataset",
259 |             "_ods_ingested_at",
260 |             "_ods_schema_id",
261 |             "_ods_schema_version",
262 |             "_ods_run_id",
263 |             "payload",
264 |         ),
265 |         context="archive envelope",
266 |     )
267 |     return envelope

```

22. ods_pipeline/models.py

131 lines

```
1 | """Canonical constants for the pipeline control-plane schema."""
2 | from __future__ import annotations
3 |
4 |
5 | class PatternType:
6 |     """Ingestion pattern types. Each has its own correlation field.
7 |
8 |     Used by ``ods_pipeline.messages.correlate`` and the future
9 |     ``ods_pipeline.patterns.IngestionPattern`` registry (T12).
10 |    """
11 |
12 |    FILE = "file" # raw file ingestion -> correlate by _ods_file_id
13 |    CDC = "cdc" # change data capture -> correlate by _ods_change_lsn
14 |    API = "api" # synchronous request -> correlate by _ods_source_request_id
15 |    EVENT = "event" # async event stream -> correlate by _ods_source_event_id
16 |
17 |    ALL: frozenset[str] = frozenset({"file", "cdc", "api", "event"})
18 |
19 |
20 | #: Maps pattern type to the canonical correlation field on the message envelope.
21 | PATTERN_CORRELATION_FIELD: dict[str, str] = {
22 |     PatternType.FILE: "_ods_file_id",
23 |     PatternType.CDC: "_ods_change_lsn",
24 |     PatternType.API: "_ods_source_request_id",
25 |     PatternType.EVENT: "_ods_source_event_id",
26 | }
27 |
28 |
29 | class Stage:
30 |     """Valid values for ``pipeline.run_stage_log.stage``."""
31 |
32 |     RAW_READ = "raw_read" # ingestion: read CSV/file from S3 raw
33 |     RAW_POLL = "raw_poll" # api_pull: HTTP poll source API for records
34 |     SCHEMA_VALIDATE = "schema_validate" # validate columns against schema registry
35 |     DQ_CHECK = "dq_check" # data quality rules evaluation
36 |     CURATED_WRITE = "curated_write" # write Parquet to S3 curated
37 |     CURATED_READ = "curated_read" # publish: read curated Parquet
38 |     KAFKA_CONSUME = "kafka_consume" # canonicalize: bounded raw-topic consume
39 |     CANONICAL_TRANSFORM = "canonical_transform" # raw-shape -> canonical-shape
40 |     KAFKA_PUBLISH = "kafka_publish" # produce Avro messages to Kafka topic
41 |     MESSAGE_RECEIVE = "message_receive" # message/API: receive request/batch/window
42 |     MESSAGE_VALIDATE = "message_validate" # message/API: validate source events
43 |     MESSAGE_ARCHIVE = "message_archive" # message/API: write S3 archive envelopes
44 |     DLQ_WRITE = "dlq_write" # write failed records/events to DLQ
45 |     RECON_T0 = "recon_t0" # T0 offset reconciliation check
46 |     RECON_T1 = "recon_t1" # raw-topic -> canonical-topic reconciliation
47 |     RECON_MESSAGE = "recon_message" # message/API count reconciliation
48 |     SINK_PG_WAIT = "sink_pg_wait" # wait for JDBC sink to consume offsets
49 |     SINK_S3_WAIT = "sink_s3_wait" # wait for S3 sink to consume offsets
50 |     FINALISE = "finalise" # DAG finalise: mark run succeeded
51 |
52 |     @classmethod
53 |     def all_values(cls) -> frozenset[str]:
54 |         return frozenset(
55 |             v for k, v in vars(cls).items()
56 |             if not k.startswith("_") and isinstance(v, str)
57 |         )
58 |
59 |
60 | class StageEvent:
61 |     """Valid values for ``pipeline.run_stage_log.event_type``.
62 |
63 |     Filtering convention:
64 |     terminal events = stage_completed | stage_failed | stage_skipped | stage_warned
65 |     in-progress = stage_started
66 |    """
67 |
68 |    STARTED = "stage_started"
69 |    COMPLETED = "stage_completed"
70 |    FAILED = "stage_failed"
71 |    SKIPPED = "stage_skipped"
72 |    WARNED = "stage_warned" # completed with warnings (e.g. DQ soft blocks)
73 |    HEARTBEAT = "stage_heartbeat" # long-running stage liveness ping (R8)
74 |
75 |    TERMINAL: frozenset[str] = frozenset(
76 |        {"stage_completed", "stage_failed", "stage_skipped", "stage_warned"}
77 |    )
78 |
79 |
80 | class RunStatus:
81 |     """Valid values for ``pipeline.run_log.status``."""
82 |
83 |     RUNNING = "running"
84 |     SUCCEEDED = "succeeded"
85 |     FAILED = "failed"
86 |     PARTIAL = "partial"
87 |
88 |
89 | #: Statuses that close a run (set ``ended_at``).
90 | TERMINAL_STATUSES: frozenset[str] = frozenset(
91 |     {RunStatus.SUCCEEDED, RunStatus.FAILED, RunStatus.PARTIAL}
92 | )
93 |
94 | #: Fields that may be inserted into ``pipeline.glue_job_log`` by
95 | #: ``glue.jobs.utils.write_job_log``. Any caller-supplied key outside this set
96 | #: is rejected before SQL composition to prevent identifier injection.
```

```

97 | ALLOWED_JOB_LOG_FIELDS: frozenset[str] = frozenset({
98 |     "run_id",
99 |     "job_name",
100 |     "pipeline_type",
101 |     "domain",
102 |     "dataset",
103 |     "source_path",
104 |     "target_path",
105 |     "business_date",
106 |     "status",
107 |     "record_count",
108 |     "error_reason",
109 |     "error_detail",
110 |     "config_version",
111 |     "config_snapshot",
112 | })
113 |
114 |
115 | #: Fields that may be updated on ``pipeline.run_log``.
116 | ALLOWED_RUN_FIELDS: frozenset[str] = frozenset({
117 |     "status",
118 |     "record_count_source",
119 |     "record_count_dq_pass",
120 |     "record_count_dq_fail",
121 |     "record_count_published",
122 |     "kafka_topic",
123 |     "kafka_offset_start",
124 |     "kafka_offset_end",
125 |     "config_version_id",
126 |     "schema_version_id",
127 |     "parents",
128 |     "error_summary",
129 |     "file_id",
130 |     "business_date",
131 | })

```


23. ods_pipeline/offsets.py

250 lines

```
1 | """Kafka offset helpers for partition-safe reconciliation and sink waits."""
2 | from __future__ import annotations
3 |
4 | import json
5 | from collections.abc import Mapping
6 | from typing import Any
7 |
8 | OffsetMap = dict[int, int]
9 | OffsetRangeMap = dict[int, tuple[int, int]]
10 |
11 |
12 | def persist_ranges(
13 |     conn,
14 |     *,
15 |     run_id: str,
16 |     stage: str,
17 |     topic: str,
18 |     ranges: Mapping[int, tuple[int, int]],
19 |     commit: bool = True,
20 | ) -> int:
21 |     """Write per-partition offset ranges to ``pipeline.run_kafka_offsets``.
22 |
23 |     Returns the number of rows inserted/updated.
24 |
25 |     Designed to be called inside the SAME transaction as the run-status update
26 |     so a crash between Kafka transaction commit and Postgres commit leaves a
27 |     detectable inconsistency (no offset rows + run still 'running').
28 |
29 |     ``commit=True`` (default) preserves single-call ergonomics; ``commit=False``
30 |     lets a caller (B8 exactly-once flow) own the transaction boundary and
31 |     commit alongside ``runs.update``.
32 |     """
33 |     if not ranges:
34 |         if commit:
35 |             conn.commit()
36 |             return 0
37 |     rows = [
38 |         (run_id, stage, topic, int(partition), int(start), int(end))
39 |         for partition, (start, end) in ranges.items()
40 |     ]
41 |     try:
42 |         with conn.cursor() as cur:
43 |             cur.executemany(
44 |                 """
45 |                 INSERT INTO pipeline.run_kafka_offsets
46 |                 (run_id, stage, topic, partition, offset_start, offset_end)
47 |                 VALUES (%s, %s, %s, %s, %s, %s)
48 |                 ON CONFLICT (run_id, stage, topic, partition)
49 |                 DO UPDATE SET
50 |                     offset_start = EXCLUDED.offset_start,
51 |                     offset_end   = EXCLUDED.offset_end,
52 |                     recorded_at  = now()
53 |                 """,
54 |                 rows,
55 |             )
56 |         if commit:
57 |             conn.commit()
58 |         return len(rows)
59 |     except Exception:
60 |         if commit:
61 |             conn.rollback()
62 |         raise
63 |
64 |
65 | def read_ranges(conn, *, run_id: str, stage: str | None = None) -> dict[str, OffsetRangeMap]:
66 |     """Return ``{topic: {partition: (start, end)}`` for a run (optionally a stage).
67 |
68 |     Empty dict if no offsets recorded. Used by B8 resume-time idempotency
69 |     check and T8 dashboards.
70 |     """
71 |     where = "run_id = %s"
72 |     params: tuple[Any, ...] = (run_id,)
73 |     if stage is not None:
74 |         where += " AND stage = %s"
75 |         params = (run_id, stage)
76 |     out: dict[str, OffsetRangeMap] = {}
77 |     with conn.cursor() as cur:
78 |         cur.execute(
79 |             f"""
80 |             SELECT topic, partition, offset_start, offset_end
81 |             FROM pipeline.run_kafka_offsets
82 |             WHERE {where}
83 |             """,
84 |             params,
85 |         )
86 |         for topic, partition, offset_start, offset_end in cur.fetchall():
87 |             out.setdefault(topic, {})[int(partition)] = (int(offset_start), int(offset_end))
88 |     return out
89 |
90 |
91 | def has_recorded_offsets(conn, *, run_id: str, stage: str) -> bool:
92 |     """Cheap existence check used by B8 resume-time idempotency in ``runs.start``.
93 |
94 |     Returns True if any ``run_kafka_offsets`` row exists for the
95 |     ``(run_id, stage)`` pair — i.e. a prior attempt's Kafka transaction
96 |     committed AND its offsets were persisted to Postgres in the same tx.
```

```

97 |     Caller should treat this as 'republish would duplicate; skip'.
98 |     """
99 |     with conn.cursor() as cur:
100 |         cur.execute(
101 |             """
102 |             SELECT 1 FROM pipeline.run_kafka_offsets
103 |             WHERE run_id = %s AND stage = %s
104 |             LIMIT 1
105 |             """,
106 |             (run_id, stage),
107 |         )
108 |         return cur.fetchone() is not None
109 |
110 |
111 | def normalise_offset_map(raw: Mapping[Any, Any] | str | None) -> OffsetMap:
112 |     """Return ``{partition: offset}`` with integer keys and values."""
113 |     if raw is None:
114 |         return {}
115 |     if isinstance(raw, str):
116 |         raw = json.loads(raw)
117 |     return {int(partition): int(offset) for partition, offset in raw.items()}
118 |
119 |
120 | def normalise_range_map(raw: Mapping[Any, Any] | str | None) -> OffsetRangeMap:
121 |     """Return ``{partition: (start, end)}`` from dict/list JSON shapes."""
122 |     if raw is None:
123 |         return {}
124 |     if isinstance(raw, str):
125 |         raw = json.loads(raw)
126 |     ranges: OffsetRangeMap = {}
127 |     for partition, value in raw.items():
128 |         if isinstance(value, Mapping):
129 |             start, end = value["start"], value["end"]
130 |         else:
131 |             start, end = value[0], value[1]
132 |         ranges[int(partition)] = (int(start), int(end))
133 |     return ranges
134 |
135 |
136 | def range_count(ranges: Mapping[Any, Any] | str | None) -> int:
137 |     """Count records represented by per-partition offset ranges."""
138 |     return sum(
139 |         max(0, end - start)
140 |         for start, end in normalise_range_map(ranges).values()
141 |     )
142 |
143 |
144 | def delta_count(start_offsets: Mapping[Any, Any], end_offsets: Mapping[Any, Any]) -> int:
145 |     """Count records between two per-partition high-watermark snapshots."""
146 |     starts = normalise_offset_map(start_offsets)
147 |     ends = normalise_offset_map(end_offsets)
148 |     return sum(
149 |         max(0, ends.get(partition, 0) - starts.get(partition, 0))
150 |         for partition in set(starts) | set(ends)
151 |     )
152 |
153 |
154 | def ranges_from_offsets(start_offsets: Mapping[Any, Any], end_offsets: Mapping[Any, Any]) -> OffsetRangeMap:
155 |     """Build per-partition ranges from two offset snapshots."""
156 |     starts = normalise_offset_map(start_offsets)
157 |     ends = normalise_offset_map(end_offsets)
158 |     return {
159 |         partition: (starts.get(partition, 0), ends.get(partition, 0))
160 |         for partition in sorted(set(starts) | set(ends))
161 |     }
162 |
163 |
164 | def jsonable_offset_map(offsets: Mapping[Any, Any] | None) -> dict[str, int]:
165 |     """Return an offset map that serializes cleanly to JSON."""
166 |     return {
167 |         str(partition): offset
168 |         for partition, offset in sorted(normalise_offset_map(offsets).items())
169 |     }
170 |
171 |
172 | def jsonable_range_map(ranges: Mapping[Any, Any] | None) -> dict[str, dict[str, int]]:
173 |     """Return a range map that serializes cleanly to JSON."""
174 |     return {
175 |         str(partition): {"start": start, "end": end}
176 |         for partition, (start, end) in sorted(normalise_range_map(ranges).items())
177 |     }
178 |
179 |
180 | def partitions_consumed(committed_offsets: Mapping[Any, Any], target_offsets: Mapping[Any, Any]) -> bool:
181 |     """True only when every target partition has reached its target offset."""
182 |     committed = normalise_offset_map(committed_offsets)
183 |     targets = normalise_offset_map(target_offsets)
184 |     if not targets:
185 |         return False
186 |     return all(committed.get(partition, -1) >= target for partition, target in targets.items())
187 |
188 |
189 | class OffsetTracker:
190 |     """Capture broker-confirmed ``(partition, offset)`` per delivered Kafka message.
191 |
192 |     Designed for use with confluent_kafka's transactional producer (B5/B6).
193 |     The producer's per-message ``on_delivery`` callback runs on the librdkafka
194 |     poll thread, so the callback MUST NOT raise. Errors are accumulated in
195 |     ``errors`` for the caller to inspect after ``producer.flush()``.
196 |

```

```

197 | Recon contract: ``len(rows_attempted) == tracker.delivered_count`` proves
198 | every queued ``produce()`` resulted in a broker-acknowledged write inside
199 | the open transaction. Safer than ``offset_end - offset_start`` because it
200 | is immune to other producers writing to the same topic concurrently.
201 |
202 | Usage:
203 |
204 |     tracker = OffsetTracker()
205 |     for row in rows:
206 |         producer.produce(topic, key=k, value=v, on_delivery=tracker.on_delivery)
207 |     producer.flush()
208 |     if tracker.errors:
209 |         raise RuntimeError(f"{len(tracker.errors)} delivery failures")
210 |     assert tracker.delivered_count == len(rows)
211 | """
212 |
213 | def __init__(self) -> None:
214 |     # {partition: [offset, ...]} - order is delivery order, not produce order.
215 |     self._delivered: dict[int, list[int]] = {}
216 |     self.errors: list[str] = []
217 |
218 | def on_delivery(self, err, msg) -> None: # noqa: D401 - librdkafka callback shape
219 |     """confluent_kafka per-message delivery callback. Must not raise."""
220 |     if err is not None:
221 |         self.errors.append(str(err))
222 |         return
223 |     try:
224 |         partition = int(msg.partition())
225 |         offset = int(msg.offset())
226 |     except Exception as exc: # pragma: no cover - defensive
227 |         self.errors.append(f"unparseable delivery report: {exc}")
228 |         return
229 |     self._delivered.setdefault(partition, []).append(offset)
230 |
231 | @property
232 | def delivered_count(self) -> int:
233 |     """Total number of broker-acknowledged messages across all partitions."""
234 |     return sum(len(offsets) for offsets in self._delivered.values())
235 |
236 | def per_partition_counts(self) -> dict[int, int]:
237 |     """``{partition: delivered_count}``."""
238 |     return {partition: len(offsets) for partition, offsets in self._delivered.items()}
239 |
240 | def per_partition_ranges(self) -> OffsetRangeMap:
241 |     """``{partition: (min_offset, max_offset+1)}`` for delivered messages.
242 |
243 |     ``end`` is exclusive (matches the rest of this module's range
244 |     convention). Empty partitions are omitted.
245 |     """
246 |     return {
247 |         partition: (min(offsets), max(offsets) + 1)
248 |         for partition, offsets in self._delivered.items()
249 |         if offsets
250 |     }

```

24. ods_pipeline/ops/___init___.py

1 lines

```
1 | """Operator-facing CLI: ``python -m ods_pipeline.ops <subcmd>``,"""
```

25. ods_pipeline/ops/__main__.py

34 lines

```
1 | """Entry point: ``python -m ods_pipeline.ops``.
2 |
3 | Dispatches to subcommand modules. Argparse-only – no external deps.
4 | """
5 | from __future__ import annotations
6 |
7 | import argparse
8 | import sys
9 |
10 | from ods_pipeline.ops import dlq as dlq_cmd
11 | from ods_pipeline.ops import runs as runs_cmd
12 |
13 |
14 | def _build_parser() -> argparse.ArgumentParser:
15 |     parser = argparse.ArgumentParser(prog="python -m ods_pipeline.ops")
16 |     sub = parser.add_subparsers(dest="cmd", required=True)
17 |     dlq_cmd.register(sub.add_parser("dlq", help="DLQ inspection + replay"))
18 |     runs_cmd.register(sub.add_parser("runs", help="replay / rerun runs"))
19 |     return parser
20 |
21 |
22 | def main(argv: list[str] | None = None) -> int:
23 |     parser = _build_parser()
24 |     args = parser.parse_args(argv)
25 |     if args.cmd == "dlq":
26 |         return dlq_cmd.dispatch(args)
27 |     if args.cmd == "runs":
28 |         return runs_cmd.dispatch(args)
29 |     parser.print_help()
30 |     return 2
31 |
32 |
33 | if __name__ == "__main__":
34 |     sys.exit(main())
```

26. ods_pipeline/ops/dlq.py

215 lines

```
1 | """DLQ subcommand: list / show / replay.
2 |
3 | Operators inspect S3 DLQ records and replay specific envelopes back through
4 | the canonical pipeline. Replay creates a new ``run_log`` row linked via
5 | ``lineage_edge`` (``edge_type='replay'``, ``parent_run_id=<original failed
6 | run>``) so the original evidence is preserved.
7 |
8 | Designed to be unit-testable: side-effects (S3, Kafka, Postgres) are passed
9 | in as already-constructed clients via the dependency injection in
10 | ``_DlqOps``. ``dispatch()`` builds default clients from env when invoked
11 | from the CLI; tests construct ``_DlqOps`` directly.
12 | """
13 | from __future__ import annotations
14 |
15 | import argparse
16 | import json
17 | import sys
18 | import uuid
19 | from collections import Counter
20 | from collections.abc import Iterable
21 | from typing import Any
22 |
23 | # -----
24 | # argparse wiring
25 | # -----
26 |
27 | def register(parser: argparse.ArgumentParser) -> None:
28 |     sub = parser.add_subparsers(dest="dlq_cmd", required=True)
29 |
30 |     p_list = sub.add_parser("list", help="counts by domain/dataset/run/stage")
31 |     p_list.add_argument("--domain")
32 |     p_list.add_argument("--dataset")
33 |     p_list.add_argument("--prefix", help="restrict to specific S3 prefix")
34 |
35 |     p_show = sub.add_parser("show", help="dump envelope JSON for a key")
36 |     p_show.add_argument("s3_uri")
37 |
38 |     p_replay = sub.add_parser("replay", help="re-publish a DLQ record")
39 |     p_replay.add_argument("s3_uri")
40 |     p_replay.add_argument("--target-topic", required=True,
41 |                           help="Kafka topic to republish into")
42 |     p_replay.add_argument("--dry-run", action="store_true",
43 |                           help="print intended action; do not produce/write")
44 |
45 |
46 | def dispatch(args) -> int: # pragma: no cover – thin glue
47 |     import boto3
48 |
49 |     s3 = boto3.client("s3")
50 |     pg = _connect_pg()
51 |     producer = None # lazily init only on replay
52 |     bucket = _default_bucket()
53 |     ops = _DlqOps(s3=s3, pg_conn=pg, bucket=bucket,
54 |                  producer_factory=lambda: _make_producer())
55 |     if args.dlq_cmd == "list":
56 |         for line in ops.list(domain=args.domain, dataset=args.dataset,
57 |                               prefix=args.prefix):
58 |             print(line)
59 |         return 0
60 |     if args.dlq_cmd == "show":
61 |         print(json.dumps(ops.show(args.s3_uri), indent=2, default=str))
62 |         return 0
63 |     if args.dlq_cmd == "replay":
64 |         result = ops.replay(args.s3_uri, target_topic=args.target_topic,
65 |                              dry_run=args.dry_run)
66 |         print(json.dumps(result, indent=2, default=str))
67 |         return 0
68 |     print(f"unknown subcommand {args.dlq_cmd!r}", file=sys.stderr)
69 |     return 2
70 |
71 |
72 | # -----
73 | # Core logic – DI-friendly
74 | # -----
75 |
76 | class _DlqOps:
77 |     """Stateless wrapper. All side-effect collaborators are injected."""
78 |
79 |     def __init__(self, *, s3, pg_conn, bucket: str,
80 |                  producer_factory=None):
81 |         self._s3 = s3
82 |         self._pg = pg_conn
83 |         self._bucket = bucket
84 |         self._producer_factory = producer_factory
85 |
86 |     # --- list -----
87 |
88 |     def list(self, *, domain: str | None = None, dataset: str | None = None,
89 |              prefix: str | None = None) -> Iterable[str]:
90 |         keys = list(self._iter_keys(domain=domain, dataset=dataset,
91 |                                     prefix=prefix))
92 |         if not keys:
93 |             yield "(no DLQ records found)"
94 |             return
95 |         breakdown: Counter = Counter()
96 |         for key in keys:
```

```

97 |         parts = key.split("/")
98 |         # layout: <domain>/<dataset>/<stage>/date=.../run_id=.../<n>.json
99 |         if len(parts) >= 5:
100 |             d, ds, stage = parts[0], parts[1], parts[2]
101 |             run_id = next((p.split("=", 1)[1] for p in parts
102 |                           if p.startswith("run_id=")), "?")
103 |             breakdown[(d, ds, stage, run_id)] += 1
104 |         yield f"total: {len(keys)} record(s)"
105 |         yield "domain/dataset/stage/run_id\tcount"
106 |         for (d, ds, stage, run_id), count in sorted(breakdown.items()):
107 |             yield f"{d}/{ds}/{stage}/{run_id}\t{count}"
108 |
109 |     # --- show -----
110 |
111 |     def show(self, s3_uri: str) -> dict[str, Any]:
112 |         bucket, key = _split_s3_uri(s3_uri)
113 |         obj = self._s3.get_object(Bucket=bucket, Key=key)
114 |         return json.loads(obj["Body"].read())
115 |
116 |     # --- replay -----
117 |
118 |     def replay(self, s3_uri: str, *, target_topic: str,
119 |                dry_run: bool = False) -> dict[str, Any]:
120 |         envelope = self.show(s3_uri)
121 |         original_run = envelope.get("_ods_run_id")
122 |         domain = envelope.get("_ods_domain")
123 |         dataset = envelope.get("_ods_dataset")
124 |         payload = envelope.get("payload", {})
125 |         replay_run_id = str(uuid.uuid4())
126 |         result = {
127 |             "replay_run_id": replay_run_id,
128 |             "original_run_id": original_run,
129 |             "target_topic": target_topic,
130 |             "domain": domain,
131 |             "dataset": dataset,
132 |             "dry_run": dry_run,
133 |         }
134 |         if dry_run:
135 |             result["status"] = "dry-run"
136 |             return result
137 |         # Real replay: open new run, link via lineage, produce, commit
138 |         from ods_pipeline import lineage, runs
139 |         runs.start(self._pg, run_id=replay_run_id, pipeline_type="dlq_replay",
140 |                   domain=domain or "unknown",
141 |                   dataset=dataset or "unknown",
142 |                   business_date=envelope.get("_ods_business_date"),
143 |                   parents=[original_run] if original_run else None)
144 |         if original_run:
145 |             lineage.write_edge(self._pg, child_run_id=replay_run_id,
146 |                               parent_run_id=original_run,
147 |                               edge_type="replay",
148 |                               source_ref=s3_uri,
149 |                               target_ref=f"kafka://{target_topic}",
150 |                               record_count=1)
151 |         producer = self._producer_factory()
152 |         producer.produce(topic=target_topic,
153 |                          value=json.dumps(payload).encode("utf-8"))
154 |         producer.flush()
155 |         result["status"] = "replayed"
156 |         return result
157 |
158 |     # --- helpers -----
159 |
160 |     def _iter_keys(self, *, domain: str | None, dataset: str | None,
161 |                   prefix: str | None):
162 |         if prefix is not None:
163 |             search = prefix
164 |         else:
165 |             parts = []
166 |             if domain:
167 |                 parts.append(domain)
168 |             if dataset:
169 |                 parts.append(dataset)
170 |             search = "/".join(parts) + "/" if parts else ""
171 |         paginator = self._s3.get_paginator("list_objects_v2")
172 |         for page in paginator.paginate(Bucket=self._bucket, Prefix=search):
173 |             for obj in page.get("Contents", []):
174 |                 yield obj["Key"]
175 |
176 |
177 |     # -----
178 |     # helpers (also used by CLI builder)
179 |     # -----
180 |
181 |     def _split_s3_uri(uri: str) -> tuple[str, str]:
182 |         if not uri.startswith("s3://"):
183 |             raise ValueError(f"not an s3 URI: {uri!r}")
184 |         rest = uri[len("s3://"):]
185 |         bucket, _, key = rest.partition("/")
186 |         if not bucket or not key:
187 |             raise ValueError(f"malformed s3 URI: {uri!r}")
188 |         return bucket, key
189 |
190 |
191 |     def _default_bucket() -> str: # pragma: no cover
192 |         import os
193 |         env = os.environ.get("ENV", "local")
194 |         return f"ods-dlq-{env}"
195 |
196 |

```

```

197 | def _connect_pg(): # pragma: no cover
198 |     import os
199 |
200 |     import psycopg2
201 |     return psycopg2.connect(
202 |         host=os.environ.get("PG_HOST", "127.0.0.1"),
203 |         port=int(os.environ.get("PG_PORT", "5440")),
204 |         dbname=os.environ.get("PG_DB", "ods_dev"),
205 |         user=os.environ.get("PG_USER", "ods"),
206 |         password=os.environ.get("PG_PASSWORD", "ods"),
207 |     )
208 |
209 |
210 | def _make_producer(): # pragma: no cover
211 |     import os
212 |
213 |     from confluent_kafka import Producer
214 |     return Producer({"bootstrap.servers": os.environ.get("KAFKA_BOOTSTRAP",
215 |                                                         "localhost:9092")})

```


27. ods_pipeline/ops/runs.py

206 lines

```
1 | """Replay / rerun subcommand for python -m ods_pipeline.ops.
2 |
3 | Two operations:
4 | - replay --file-id <uuid>   restart all runs that touched a file
5 | - rerun --run-id <uuid>    restart one specific run
6 |
7 | Both operations:
8 | 1. Look up the original run + dataset metadata from run_log
9 | 2. Allocate a replay_request_id for operator/audit correlation
10 | 3. Trigger the appropriate Airflow DAG via REST (or print intended action
11 |    in --dry-run)
12 | 4. The DAG creates the actual run rows and links lineage using
13 |    replay_of_run_id from conf. Old run + its evidence are NEVER mutated.
14 | """
15 | from __future__ import annotations
16 |
17 | import argparse
18 | import json
19 | import sys
20 | import uuid
21 | from typing import Any
22 |
23 |
24 | def register(parser: argparse.ArgumentParser) -> None:
25 |     sub = parser.add_subparsers(dest="runs_cmd", required=True)
26 |     p_replay = sub.add_parser("replay", help="restart runs for a file")
27 |     p_replay.add_argument("--file-id", required=True)
28 |     p_replay.add_argument("--dry-run", action="store_true")
29 |
30 |     p_rerun = sub.add_parser("rerun", help="restart a specific run")
31 |     p_rerun.add_argument("--run-id", required=True)
32 |     p_rerun.add_argument("--dry-run", action="store_true")
33 |
34 |
35 | def dispatch(args) -> int: # pragma: no cover – thin glue
36 |     pg = _connect_pg()
37 |     airflow = _make_airflow_client()
38 |     ops = _RunsOps(pg_conn=pg, airflow=airflow)
39 |     if args.runs_cmd == "replay":
40 |         out = ops.replay_file(args.file_id, dry_run=args.dry_run)
41 |     elif args.runs_cmd == "rerun":
42 |         out = ops.rerun(args.run_id, dry_run=args.dry_run)
43 |     else:
44 |         print(f"unknown subcommand {args.runs_cmd!r}", file=sys.stderr)
45 |         return 2
46 |     print(json.dumps(out, indent=2, default=str))
47 |     return 0
48 |
49 |
50 | class _RunsOps:
51 |     """Stateless wrapper. Postgres + Airflow client injected for testability."""
52 |
53 |     def __init__(self, *, pg_conn, airflow):
54 |         self._pg = pg_conn
55 |         self._airflow = airflow
56 |
57 |     # -----
58 |     # rerun: keyed by run_id
59 |     # -----
60 |     def rerun(self, run_id: str, *, dry_run: bool = False) -> dict[str, Any]:
61 |         original = self._lookup_run(run_id)
62 |         if original is None:
63 |             raise LookupError(f"run_id not found: {run_id}")
64 |         return self._launch_replay(original, source="rerun", dry_run=dry_run)
65 |
66 |     # -----
67 |     # replay: keyed by file_id (one or more original runs)
68 |     # -----
69 |     def replay_file(self, file_id: str, *, dry_run: bool = False) -> dict[str, Any]:
70 |         original = self._lookup_replay_candidate_by_file(file_id)
71 |         if not original:
72 |             raise LookupError(f"no runs found for file_id={file_id}")
73 |         return {
74 |             "file_id": file_id,
75 |             "replay": self._launch_replay(original, source="replay", dry_run=dry_run),
76 |         }
77 |
78 |     # -----
79 |     # internals
80 |     # -----
81 |     def _lookup_run(self, run_id: str):
82 |         with self._pg.cursor() as cur:
83 |             cur.execute(
84 |                 """
85 |                 SELECT run_id::text, pipeline_type, domain, dataset,
86 |                       business_date::text, file_id::text, kafka_topic
87 |                 FROM pipeline.run_log WHERE run_id = %s
88 |                 """,
89 |                 (run_id,),
90 |             )
91 |             row = cur.fetchone()
92 |             if not row:
93 |                 return None
94 |             keys = ("run_id", "pipeline_type", "domain", "dataset",
95 |                    "business_date", "file_id", "kafka_topic")
96 |             return dict(zip(keys, row))
```

```

97 |
98 | def _lookup_replay_candidate_by_file(self, file_id: str):
99 |     with self._pg.cursor() as cur:
100 |         cur.execute(
101 |             """
102 |             SELECT run_id::text, pipeline_type, domain, dataset,
103 |                   business_date::text, file_id::text, kafka_topic
104 |             FROM pipeline.run_log
105 |             WHERE file_id = %s::uuid
106 |             ORDER BY CASE pipeline_type
107 |                 WHEN 's3_batch' THEN 0
108 |                 WHEN 'file' THEN 1
109 |                 WHEN 'ingestion' THEN 2
110 |                 WHEN 'publish_raw' THEN 3
111 |                 WHEN 'publish' THEN 4
112 |                 WHEN 'canonicalize' THEN 5
113 |                 ELSE 9
114 |             END,
115 |                 started_at DESC
116 |             LIMIT 1
117 |             """,
118 |             (file_id,),
119 |         )
120 |         row = cur.fetchone()
121 |         if not row:
122 |             return None
123 |         keys = ("run_id", "pipeline_type", "domain", "dataset",
124 |                "business_date", "file_id", "kafka_topic")
125 |         return dict(zip(keys, row))
126 |
127 | def _launch_replay(self, original, *, source: str,
128 |                   dry_run: bool) -> dict[str, Any]:
129 |     replay_request_id = str(uuid.uuid4())
130 |     result = {
131 |         "source": source,
132 |         "replay_request_id": replay_request_id,
133 |         "original_run_id": original["run_id"],
134 |         "pipeline_type": original["pipeline_type"],
135 |         "domain": original["domain"],
136 |         "dataset": original["dataset"],
137 |         "business_date": original["business_date"],
138 |         "file_id": original["file_id"],
139 |         "dry_run": dry_run,
140 |     }
141 |     if dry_run:
142 |         result["status"] = "dry-run"
143 |         return result
144 |     dag_id = _dag_for(original["pipeline_type"])
145 |     self._airflow.trigger_dag(
146 |         dag_id=dag_id,
147 |         conf={"replay_request_id": replay_request_id,
148 |              "replay_of_run_id": original["run_id"],
149 |              "file_id": original["file_id"],
150 |              "domain": original["domain"],
151 |              "dataset": original["dataset"],
152 |              "business_date": original["business_date"]},
153 |     )
154 |     result["status"] = "triggered"
155 |     result["dag_id"] = dag_id
156 |     return result
157 |
158 |
159 | def _dag_for(pipeline_type: str) -> str:
160 |     return {
161 |         "file": "dag_ingest",
162 |         "message_api": "dag_event_api",
163 |         "dlq_replay": "dag_dlq_replay",
164 |     }.get(pipeline_type, "dag_ingest")
165 |
166 |
167 | # -----
168 | # CLI helpers (production-only paths)
169 | # -----
170 |
171 | def _connect_pg(): # pragma: no cover
172 |     import os
173 |
174 |     import psycopg2
175 |     return psycopg2.connect(
176 |         host=os.environ.get("PG_HOST", "127.0.0.1"),
177 |         port=int(os.environ.get("PG_PORT", "5440")),
178 |         dbname=os.environ.get("PG_DB", "ods_dev"),
179 |         user=os.environ.get("PG_USER", "ods"),
180 |         password=os.environ.get("PG_PASSWORD", "ods"),
181 |     )
182 |
183 |
184 | def _make_airflow_client(): # pragma: no cover
185 |     import os
186 |
187 |     import requests
188 |
189 |     base = os.environ.get("AIRFLOW_BASE_URL", "http://localhost:8080")
190 |     auth = (
191 |         os.environ.get("AIRFLOW_USER", "airflow"),
192 |         os.environ.get("AIRFLOW_PASSWORD", "airflow"),
193 |     )
194 |
195 |     class _AirflowClient:
196 |         def trigger_dag(self, *, dag_id: str, conf: dict[str, Any]) -> dict:

```

```
197 |         resp = requests.post(  
198 |             f"{base}/api/v1/dags/{dag_id}/dagRuns",  
199 |             json={"conf": conf},  
200 |             auth=auth,  
201 |             timeout=30,  
202 |         )  
203 |         resp.raise_for_status()  
204 |         return resp.json()  
205 |  
206 |     return _AirflowClient()
```

28. ods_pipeline/patterns/__init__.py

18 lines

```
1 | """Ingestion pattern registry (A4 / T12).
2 |
3 | Each pattern type (file, cdc, api, event) has a concrete `IngestionPattern`
4 | declaring its stages, topics, sinks, recon checks and correlation field.
5 | Consumers (DAG factory, Glue entrypoints, event_api) look up patterns
6 | by name from the registry to drive execution.
7 | """
8 | from ods_pipeline.patterns import api_pull as _api_pull # noqa: F401 – registers patterns
9 | from ods_pipeline.patterns import event as _event # noqa: F401 – registers patterns
10 | from ods_pipeline.patterns import file as _file # noqa: F401 – registers patterns
11 | from ods_pipeline.patterns.base import (
12 |     PATTERNS,
13 |     IngestionPattern,
14 |     get,
15 |     register,
16 | )
17 |
18 | __all__ = ["IngestionPattern", "PATTERNS", "get", "register"]
```

29. ods_pipeline/patterns/api_pull.py

40 lines

```
1 | """API pull ingestion pattern.
2 |
3 | Airflow polls an external HTTP API on a schedule, archives the response
4 | as gzipped JSONL to S3, registers the archive in pipeline.file_catalogue,
5 | and triggers the existing dag_ingest pipeline (raw_read -> Kafka publish
6 | -> canonicalize -> JDBC sink). Cursor commit follows a two-phase
7 | pending/committed split so downstream failure does not lose data.
8 |
9 | The registration mirrors the file/event patterns: a declarative
10 | IngestionPattern is the single source of truth for stages, topics,
11 | sinks, recon checks and YAML config.
12 | """
13 | from __future__ import annotations
14 |
15 | from ods_pipeline.models import PatternType, Stage
16 | from ods_pipeline.patterns.base import IngestionPattern, register
17 |
18 | # Stages emitted by the api_pull control-plane (poller + dag_api_pull).
19 | # Downstream stages (raw_read, schema_validate, ...) are emitted by the
20 | # triggered dag_ingest run and are not duplicated here.
21 | _API_PULL_STAGES = (
22 |     Stage.RAW_POLL,
23 |     Stage.MESSAGE_ARCHIVE,
24 |     Stage.RECON_MESSAGE,
25 |     Stage.FINALISE,
26 | )
27 |
28 |
29 | API_PULL_DEMO = register(IngestionPattern(
30 |     name="insurance.api_pull_demo",
31 |     pattern_type=PatternType.API,
32 |     stages=_API_PULL_STAGES,
33 |     topics=(
34 |         "ods.insurance.api_pull_demo",
35 |         "ods.insurance.api_pull_demo.canonical",
36 |     ),
37 |     sinks=("jdbc-sink-api-pull-demo",),
38 |     recon_checks=("api_pull_archive_count",),
39 |     yaml_config="patterns/insurance/api_pull_demo.yaml",
40 | ))
```

30. ods_pipeline/patterns/base.py

64 lines

```
1 | """IngestionPattern dataclass + registry."""
2 | from __future__ import annotations
3 |
4 | from dataclasses import dataclass
5 |
6 | from ods_pipeline.models import PATTERN_CORRELATION_FIELD, PatternType, Stage
7 |
8 |
9 | @dataclass(frozen=True)
10 | class IngestionPattern:
11 |     """Declarative spec of an ingestion flow.
12 |
13 |     Pattern types:
14 |     - ``file`` - SFTP/S3 file -> Glue ingestion -> raw Kafka -> canonical Kafka -> sink
15 |     - ``cdc`` - DB CDC stream -> raw Kafka -> canonical Kafka -> sink
16 |     - ``api`` - synchronous request -> S3 archive + raw Kafka -> canonical -> sink
17 |     - ``event`` - async event stream -> raw Kafka -> canonical -> sink
18 |
19 |     The ``correlation_field`` is derived from ``pattern_type`` so callers
20 |     cannot accidentally desync them.
21 |     """
22 |
23 |     name: str
24 |     pattern_type: str
25 |     stages: tuple[str, ...]
26 |     topics: tuple[str, ...]
27 |     sinks: tuple[str, ...]
28 |     recon_checks: tuple[str, ...] = ()
29 |     yaml_config: str | None = None
30 |
31 |     def __post_init__(self):
32 |         if self.pattern_type not in PatternType.ALL:
33 |             raise ValueError(
34 |                 f"unknown pattern_type {self.pattern_type!r}; "
35 |                 f"must be one of {sorted(PatternType.ALL)}"
36 |             )
37 |         unknown_stages = set(self.stages) - Stage.all_values()
38 |         if unknown_stages:
39 |             raise ValueError(f"unknown stages: {sorted(unknown_stages)}")
40 |
41 |     @property
42 |     def correlation_field(self) -> str:
43 |         """Canonical envelope correlation field for this pattern type."""
44 |         return PATTERN_CORRELATION_FIELD[self.pattern_type]
45 |
46 |
47 | # Module-level registry. Patterns register themselves at import time.
48 | PATTERNS: dict[str, IngestionPattern] = {}
49 |
50 |
51 | def register(pattern: IngestionPattern) -> IngestionPattern:
52 |     """Register ``pattern`` in the global registry. Re-register replaces."""
53 |     PATTERNS[pattern.name] = pattern
54 |     return pattern
55 |
56 |
57 | def get(name: str) -> IngestionPattern:
58 |     """Return pattern by name. Raises KeyError if unknown."""
59 |     if name not in PATTERNS:
60 |         raise KeyError(
61 |             f"no pattern registered named {name!r}; "
62 |             f"known: {sorted(PATTERNS)}"
63 |         )
64 |     return PATTERNS[name]
```

31. ods_pipeline/patterns/event.py

29 lines

```
1 | """Event-pattern demo registration (T16).
2 |
3 | First non-file IngestionPattern. Proves the IngestionPattern abstraction
4 | generalises beyond the file pattern by composing the same control-plane
5 | primitives (runs, stages, lineage, recon) for an HTTP-trigger source.
6 | """
7 | from __future__ import annotations
8 |
9 | from ods_pipeline.models import PatternType, Stage
10 | from ods_pipeline.patterns.base import IngestionPattern, register
11 |
12 | EVENT_DEMO = register(IngestionPattern(
13 |     name="insurance.event_demo",
14 |     pattern_type=PatternType.EVENT,
15 |     stages=(
16 |         Stage.MESSAGE_RECEIVE,
17 |         Stage.MESSAGE_VALIDATE,
18 |         Stage.MESSAGE_ARCHIVE,
19 |         Stage.KAFKA_PUBLISH,
20 |         Stage.CANONICAL_TRANSFORM,
21 |         Stage.RECON_MESSAGE,
22 |         Stage.SINK_PG_WAIT,
23 |         Stage.FINALISE,
24 |     ),
25 |     topics=("ods.insurance.events", "ods.insurance.events.canonical"),
26 |     sinks=("jdbc-sink-events",),
27 |     recon_checks=("recon_message_count",),
28 |     yaml_config="patterns/insurance/event_demo.yaml",
29 | ))
```

32. ods_pipeline/patterns/file.py

51 lines

```
1 | """File-based ingestion patterns (risk, policies).
2 |
3 | Both flow: SFTP -> S3 raw -> Glue ingestion (CSV -> Parquet, DQ) ->
4 | publish raw Kafka -> canonicalize -> publish canonical Kafka -> JDBC
5 | sink(s) -> recon.
6 |
7 | These declarations replace the implicit "Glue + DAG knows what to do"
8 | convention with explicit metadata that DAG factory + Glue entrypoint
9 | can consume.
10 | """
11 | from __future__ import annotations
12 |
13 | from ods_pipeline.models import PatternType, Stage
14 | from ods_pipeline.patterns.base import IngestionPattern, register
15 |
16 | _FILE_STAGES = (
17 |     Stage.RAW_READ,
18 |     Stage.SCHEMA_VALIDATE,
19 |     Stage.DQ_CHECK,
20 |     Stage.CURATED_WRITE,
21 |     Stage.CURATED_READ,
22 |     Stage.KAFKA_PUBLISH,
23 |     Stage.KAFKA_CONSUME,
24 |     Stage.CANONICAL_TRANSFORM,
25 |     Stage.RECON_T0,
26 |     Stage.RECON_T1,
27 |     Stage.SINK_PG_WAIT,
28 |     Stage.FINALISE,
29 | )
30 |
31 |
32 | POLICIES = register(IngestionPattern(
33 |     name="insurance.policies",
34 |     pattern_type=PatternType.FILE,
35 |     stages=_FILE_STAGES,
36 |     topics=("ods.insurance.policies",),
37 |     sinks=("jdbc-sink-policies", "jdbc-sink-policy-history"),
38 |     recon_checks=("t0_publish_count", "t1_canonical_count", "dual_sink_parity"),
39 |     yaml_config="patterns/insurance/policies.yaml",
40 | ))
41 |
42 |
43 | RISK = register(IngestionPattern(
44 |     name="insurance.risk",
45 |     pattern_type=PatternType.FILE,
46 |     stages=_FILE_STAGES,
47 |     topics=("ods.insurance.risk", "ods.insurance.risk-canonical"),
48 |     sinks=("jdbc-sink-risk",),
49 |     recon_checks=("t0_publish_count", "t1_canonical_count"),
50 |     yaml_config="patterns/insurance/risk.yaml",
51 | ))
```


33. ods_pipeline/publish.py

90 lines

```
1 | """Reusable Kafka transactional-publish lifecycle (B5/B6).
2 |
3 | Extracted from ``glue/jobs/ods_s3_publish.py`` so the same lifecycle can be
4 | exercised by:
5 |
6 | * ``ods_s3_publish.py`` – current production caller.
7 | * ``ods_pipeline/messages.py`` callers – once T6 / T16 land message-pattern
8 |   publishing.
9 | * The unit test ``tests/unit/test_publish_transactional.py`` – exercises the
10 |  exact production code path, so reverting the Glue caller would fail the
11 |  test suite (no tautology – the helper is load-bearing).
12 |
13 | Design contract:
14 |
15 | * The caller owns ``producer = Producer(config)`` and ``producer.close()``
16 |   (typically inside its own ``finally``). This helper does NOT close the
17 |   producer because the caller may want to reuse the same Producer across
18 |   multiple invocations or wrap close ordering with other cleanup.
19 | * The caller passes a fresh ``OffsetTracker``; on success the tracker is
20 |   populated with broker-acknowledged ``(partition, offset)`` per delivered
21 |   message; ``tracker.delivered_count`` is the recon source-of-truth.
22 | * The caller passes ``payloads`` – any iterable; each item is forwarded to
23 |   ``on_send(producer, payload, on_delivery=tracker.on_delivery)``. The
24 |   ``on_send`` callback is the integration point: production callers serialise
25 |   Avro and call ``producer.produce(...)``; tests use a one-line lambda.
26 |
27 | Lifecycle (single transaction):
28 |   init_transactions → begin_transaction
29 |                       → for payload: on_send(...)
30 |                       → flush
31 |                       → if tracker.errors: raise
32 |                       → commit_transaction
33 |   Any exception in the produce loop or in flush → abort_transaction
34 |                                           → raise
35 |
36 | from __future__ import annotations
37 |
38 | from collections.abc import Callable, Iterable
39 | from typing import Any
40 |
41 | from ods_pipeline.offsets import OffsetTracker
42 |
43 |
44 | def _default_on_send(producer, payload, *, on_delivery):
45 |     """Default per-payload sender used when caller doesn't supply one.
46 |
47 |     Expects ``payload`` to be a mapping with ``topic``, ``key``, ``value`` –
48 |     matches the lightweight test fixture shape. Production callers ALWAYS
49 |     pass their own ``on_send`` because they need to serialise Avro and
50 |     compute message keys.
51 |     """
52 |     producer.produce(
53 |         topic=payload["topic"],
54 |         key=payload.get("key"),
55 |         value=payload.get("value"),
56 |         on_delivery=on_delivery,
57 |     )
58 |
59 |
60 | def publish_with_transaction(
61 |     producer,
62 |     payloads: Iterable[Any],
63 |     tracker: OffsetTracker,
64 |     *,
65 |     on_send: Callable[..., None] | None = None,
66 | ) -> None:
67 |     """Run one transactional publish: init → begin → produce* → commit/abort.
68 |
69 |     Caller owns ``producer.close()`` (typically in their own ``finally``).
70 |     Raises whatever the produce loop raises, after first calling
71 |     ``producer.abort_transaction()``. If ``tracker.errors`` is non-empty
72 |     after ``producer.flush()`` , raises ``RuntimeError`` describing the first
73 |     failure (and aborts).
74 |     """
75 |     sender = on_send or _default_on_send
76 |     producer.init_transactions()
77 |     producer.begin_transaction()
78 |     try:
79 |         for payload in payloads:
80 |             sender(producer, payload, on_delivery=tracker.on_delivery)
81 |         producer.flush()
82 |         if tracker.errors:
83 |             raise RuntimeError(
84 |                 f"{len(tracker.errors)} kafka delivery failures; "
85 |                 f"first: {tracker.errors[0]}"
86 |             )
87 |         producer.commit_transaction()
88 |     except Exception:
89 |         producer.abort_transaction()
90 |         raise
```

34. ods_pipeline/reconciliation.py

378 lines

```
1 | """pipeline.reconciliation_log operations."""
2 | from __future__ import annotations
3 |
4 | import json
5 | from collections.abc import Mapping
6 | from typing import Any
7 |
8 | from psycopg2 import sql
9 |
10 |
11 | def write_check(
12 |     conn,
13 |     *,
14 |     check_type: str,
15 |     run_id: str,
16 |     domain: str,
17 |     dataset: str,
18 |     business_date: str,
19 |     source_count: int | None = None,
20 |     kafka_count: int | None = None,
21 |     postgres_count: int | None = None,
22 |     status: str,
23 |     detail: str | None = None,
24 |     window_start=None,
25 |     window_end=None,
26 |     commit: bool = True,
27 | ) -> None:
28 |     """Insert a row into ``pipeline.reconciliation_log``.
29 |
30 |     Computes ``discrepancy_count`` and ``discrepancy_pct`` automatically:
31 |     * If *source_count* and *kafka_count* both supplied:
32 |       ``discrepancy = kafka_count - source_count``
33 |     * If *kafka_count* and *postgres_count* both supplied:
34 |       ``discrepancy = postgres_count - kafka_count``
35 |
36 |     ``commit``: when True (default), the helper commits its own transaction.
37 |     When False, the caller owns the surrounding tx (used by atomic
38 |     ``record_result`` flow - B4).
39 |     """
40 |     discrepancy: int | None = None
41 |     if source_count is not None and kafka_count is not None:
42 |         discrepancy = (kafka_count or 0) - (source_count or 0)
43 |     elif kafka_count is not None and postgres_count is not None:
44 |         discrepancy = (postgres_count or 0) - (kafka_count or 0)
45 |
46 |     pct: float | None = None
47 |     if discrepancy is not None and source_count:
48 |         pct = round(100.0 * discrepancy / source_count, 4)
49 |
50 |     try:
51 |         with conn.cursor() as cur:
52 |             cur.execute(
53 |                 """
54 |                 INSERT INTO pipeline.reconciliation_log
55 |                 (check_type, run_id, domain, dataset, business_date,
56 |                  window_start, window_end,
57 |                  source_count, kafka_count, postgres_count,
58 |                  discrepancy_count, discrepancy_pct, status, detail)
59 |                 VALUES (%s,%s,%s,%s,%s, %s,%s, %s,%s,%s, %s,%s,%s,%s)
60 |                 """
61 |                 (
62 |                     check_type, run_id, domain, dataset,
63 |                     None if business_date is None else str(business_date),
64 |                     window_start, window_end,
65 |                     source_count, kafka_count, postgres_count,
66 |                     discrepancy, pct, status, detail,
67 |                 ),
68 |             )
69 |             if commit:
70 |                 conn.commit()
71 |     except Exception:
72 |         if commit:
73 |             conn.rollback()
74 |         raise
75 |
76 |
77 | # Default dual-sink table pairs per dataset.
78 | # Each entry: dataset -> (current_table, history_table, run_id_column).
79 | DUAL_SINK_TABLES: dict[str, tuple[str, str, str]] = {
80 |     "policies": ("ods.insurance_policy",
81 |                  "ods.insurance_policy_history",
82 |                  "_ods_run_id"),
83 | }
84 |
85 |
86 | # dataset -> (current_table, history_table, key_columns, compare_columns, order_column)
87 | CURRENT_HISTORY_TABLES: dict[str, tuple[str, str, tuple[str, ...], tuple[str, ...], str]] = {
88 |     "policies": (
89 |         "ods.insurance_policy",
90 |         "ods.insurance_policy_history",
91 |         ("policy_id",),
92 |         ("status", "premium", "effective_date", "_ods_file_id", "_ods_run_id"),
93 |         "_ods_ingested_at",
94 |     ),
95 | }
96 |
```

```

97 |
98 | def _split_table_ref(table_ref: str) -> tuple[str, str]:
99 |     schema, sep, table = table_ref.partition(".")
100 |     if not sep or not schema.isidentifier() or not table.isidentifier():
101 |         raise ValueError(f"table reference must be schema.table: {table_ref!r}")
102 |     return schema, table
103 |
104 |
105 | def _safe_column(column: str) -> str:
106 |     if not column.isidentifier():
107 |         raise ValueError(f"illegal column name: {column!r}")
108 |     return column
109 |
110 |
111 | def check_dual_sink_parity(
112 |     conn,
113 |     *,
114 |     run_id: str,
115 |     domain: str,
116 |     dataset: str,
117 |     business_date: str | None,
118 |     pairs: Mapping[str, tuple[str, str, str]] | None = None,
119 |     commit: bool = True,
120 | ) -> dict[str, Any]:
121 |     """Compare row counts between current and history sinks for ``run_id``.
122 |
123 |     Returns the reconciliation summary and writes a corresponding row to
124 |     ``pipeline.reconciliation_log`` with ``check_type='dual_sink_parity'``.
125 |
126 |     History sink lag is the most common source of divergence between the
127 |     upsert (current) and append (history) JDBC sinks reading from the same
128 |     canonical topic. If the history connector is paused, restart-lagging,
129 |     or schema-evolution-blocked, the current state can advance without a
130 |     matching history row, leaving the audit trail incomplete.
131 |
132 |     Status:
133 |     - ``ok`` - counts match (delta == 0)
134 |     - ``pending`` - at least one sink has no rows yet (likely lag)
135 |     - ``failed`` - both sinks have rows but counts diverge
136 |
137 |     """
138 |     pairs_to_check = pairs or DUAL_SINK_TABLES
139 |     table_pair = pairs_to_check.get(dataset)
140 |     if not table_pair:
141 |         return {"status": "skipped", "reason": f"no dual-sink pair registered for {dataset}"}
142 |     current_table, history_table, run_col = table_pair
143 |     current_schema, current_name = _split_table_ref(current_table)
144 |     history_schema, history_name = _split_table_ref(history_table)
145 |     run_col = _safe_column(run_col)
146 |
147 |     with conn.cursor() as cur:
148 |         cur.execute(
149 |             sql.SQL("SELECT COUNT(*) FROM {}.{} WHERE {} = %s").format(
150 |                 sql.Identifier(current_schema),
151 |                 sql.Identifier(current_name),
152 |                 sql.Identifier(run_col),
153 |             ),
154 |             (run_id,),
155 |         )
156 |         current_count = int(cur.fetchone()[0])
157 |         cur.execute(
158 |             sql.SQL("SELECT COUNT(*) FROM {}.{} WHERE {} = %s").format(
159 |                 sql.Identifier(history_schema),
160 |                 sql.Identifier(history_name),
161 |                 sql.Identifier(run_col),
162 |             ),
163 |             (run_id,),
164 |         )
165 |         history_count = int(cur.fetchone()[0])
166 |
167 |     delta = history_count - current_count
168 |     if current_count == 0 or history_count == 0:
169 |         status = "pending"
170 |         detail = (f"current={current_count}, history={history_count}; "
171 |                  "one or both sinks empty – likely consumer lag")
172 |     elif delta == 0:
173 |         status = "ok"
174 |         detail = f"current={current_count}, history={history_count}"
175 |     else:
176 |         status = "failed"
177 |         detail = (f"current={current_count}, history={history_count}, "
178 |                  f"delta={delta}")
179 |
180 |     write_check(
181 |         conn,
182 |         check_type="dual_sink_parity",
183 |         run_id=run_id,
184 |         domain=domain,
185 |         dataset=dataset,
186 |         business_date=business_date,
187 |         source_count=current_count,
188 |         postgres_count=history_count,
189 |         status=status,
190 |         detail=detail,
191 |         commit=commit,
192 |     )
193 |     return {
194 |         "status": status,
195 |         "current_count": current_count,
196 |         "history_count": history_count,
197 |         "delta": delta,

```

```

197 |         "detail": detail,
198 |     }
199 |
200 |
201 | def compare_history_vs_current(
202 |     conn,
203 |     *,
204 |     run_id: str,
205 |     domain: str,
206 |     dataset: str,
207 |     business_date: str | None,
208 |     tables: Mapping[str, tuple[str, str, tuple[str, ...], tuple[str, ...], str]] | None = None,
209 |     sample_limit: int = 20,
210 |     commit: bool = True,
211 | ) -> dict[str, Any]:
212 |     """Compare current-state rows to latest matching history rows.
213 |
214 |     Count parity can prove both sinks received the same number of rows, but it
215 |     cannot prove the current table holds the same values as the append history
216 |     table's latest row per business key. This check builds a latest-history
217 |     view with ``row_number()`` and compares configured columns using
218 |     ``IS DISTINCT FROM`` so NULLs are handled correctly.
219 |     """
220 |     registry = tables or CURRENT_HISTORY_TABLES
221 |     table_config = registry.get(dataset)
222 |     if not table_config:
223 |         return {"status": "skipped", "reason": f"no current/history pair registered for {dataset}"}
224 |
225 |     current_table, history_table, key_columns, compare_columns, order_column = table_config
226 |     current_schema, current_name = _split_table_ref(current_table)
227 |     history_schema, history_name = _split_table_ref(history_table)
228 |     key_columns = tuple(_safe_column(column) for column in key_columns)
229 |     compare_columns = tuple(_safe_column(column) for column in compare_columns)
230 |     order_column = _safe_column(order_column)
231 |     if not key_columns:
232 |         raise ValueError("at least one key column is required")
233 |     if not compare_columns:
234 |         raise ValueError("at least one compare column is required")
235 |
236 |     key_join = sql.SQL(" AND ").join(
237 |         sql.SQL("c.{col} = h.{col}").format(col=sql.Identifier(column))
238 |         for column in key_columns
239 |     )
240 |     diff_predicate = sql.SQL(" OR ").join(
241 |         sql.SQL("c.{col} IS DISTINCT FROM h.{col}").format(col=sql.Identifier(column))
242 |         for column in compare_columns
243 |     )
244 |     key_json = sql.SQL(", ").join(
245 |         sql.SQL("{literal}, c.{col}").format(
246 |             literal=sql.Literal(column),
247 |             col=sql.Identifier(column),
248 |         )
249 |         for column in key_columns
250 |     )
251 |     value_json = sql.SQL(", ").join(
252 |         sql.SQL("{literal}, jsonb_build_object('current', c.{col}, 'history', h.{col})").format(
253 |             literal=sql.Literal(column),
254 |             col=sql.Identifier(column),
255 |         )
256 |         for column in compare_columns
257 |     )
258 |     partition_by = sql.SQL(", ").join(sql.Identifier(column) for column in key_columns)
259 |     history_filters = [sql.SQL("{} = %s").format(sql.Identifier("_ods_business_date"))]
260 |     current_filters = [sql.SQL("{} = %s").format(sql.Identifier("_ods_business_date"))]
261 |     params: list[Any] = [business_date, business_date, sample_limit]
262 |     if business_date is None:
263 |         history_filters = [sql.SQL("%s IS NULL")]
264 |         current_filters = [sql.SQL("%s IS NULL")]
265 |
266 |     query = sql.SQL(
267 |         """
268 |         WITH latest_history AS (
269 |             SELECT *
270 |             FROM (
271 |                 SELECT h.*,
272 |                        row_number() OVER (
273 |                            PARTITION BY {partition_by}
274 |                            ORDER BY {order_col} DESC NULLS LAST
275 |                        ) AS rn
276 |                 FROM {history_schema}.{history_table} h
277 |                 WHERE {history_where}
278 |             ) ranked
279 |             WHERE rn = 1
280 |         ),
281 |         current_rows AS (
282 |             SELECT *
283 |             FROM {current_schema}.{current_table} c
284 |             WHERE {current_where}
285 |         ),
286 |         mismatches AS (
287 |             SELECT jsonb_build_object({key_json}) AS key,
288 |                    jsonb_build_object({value_json}) AS differences
289 |             FROM current_rows c
290 |             JOIN latest_history h ON {key_join}
291 |             WHERE {diff_predicate}
292 |         ),
293 |         missing_history AS (
294 |             SELECT jsonb_build_object({key_json}) AS key
295 |             FROM current_rows c
296 |             LEFT JOIN latest_history h ON {key_join}

```

```

297 |         WHERE h.{first_key} IS NULL
298 |     ),
299 |     counts AS (
300 |         SELECT
301 |             (SELECT COUNT(*) FROM current_rows) AS current_count,
302 |             (SELECT COUNT(*) FROM latest_history) AS history_count,
303 |             (SELECT COUNT(*) FROM mismatches) AS mismatch_count,
304 |             (SELECT COUNT(*) FROM missing_history) AS missing_history_count
305 |         )
306 |     SELECT counts.current_count,
307 |            counts.history_count,
308 |            counts.mismatch_count,
309 |            counts.missing_history_count,
310 |            COALESCE(
311 |                SELECT jsonb_agg(sample)
312 |                FROM (
313 |                    SELECT jsonb_build_object(
314 |                        'type', 'mismatch',
315 |                        'key', key,
316 |                        'differences', differences
317 |                    ) AS sample
318 |                    FROM mismatches
319 |                    LIMIT %s
320 |                ) s
321 |            ), '['::jsonb) ||
322 |            COALESCE(
323 |                SELECT jsonb_agg(sample)
324 |                FROM (
325 |                    SELECT jsonb_build_object(
326 |                        'type', 'missing_history',
327 |                        'key', key
328 |                    ) AS sample
329 |                    FROM missing_history
330 |                    LIMIT %s
331 |                ) s
332 |            ), '['::jsonb) AS samples
333 |     FROM counts
334 |     """
335 | ).format(
336 |     partition_by=partition_by,
337 |     order_col=sql.Identifier(order_column),
338 |     history_schema=sql.Identifier(history_schema),
339 |     history_table=sql.Identifier(history_name),
340 |     current_schema=sql.Identifier(current_schema),
341 |     current_table=sql.Identifier(current_name),
342 |     history_where=sql.SQL(" AND ").join(history_filters),
343 |     current_where=sql.SQL(" AND ").join(current_filters),
344 |     key_join=key_join,
345 |     diff_predicate=diff_predicate,
346 |     key_json=key_json,
347 |     value_json=value_json,
348 |     first_key=sql.Identifier(key_columns[0]),
349 | )
350 | params.append(sample_limit)
351 |
352 | with conn.cursor() as cur:
353 |     cur.execute(query, params)
354 |     current_count, history_count, mismatch_count, missing_history_count, samples = cur.fetchone()
355 |
356 | issue_count = int(mismatch_count) + int(missing_history_count)
357 | status = "ok" if issue_count == 0 else "failed"
358 | detail_payload = {
359 |     "current_count": int(current_count),
360 |     "history_count": int(history_count),
361 |     "mismatch_count": int(mismatch_count),
362 |     "missing_history_count": int(missing_history_count),
363 |     "samples": samples,
364 | }
365 | write_check(
366 |     conn,
367 |     check_type="current_history_row_value",
368 |     run_id=run_id,
369 |     domain=domain,
370 |     dataset=dataset,
371 |     business_date=business_date,
372 |     source_count=int(history_count),
373 |     postgres_count=int(current_count),
374 |     status=status,
375 |     detail=json.dumps(detail_payload, default=str, sort_keys=True),
376 |     commit=commit,
377 | )
378 | return {"status": status, **detail_payload}

```

35. ods_pipeline/runs.py

238 lines

```
1 | """pipeline.run_log operations."""
2 | from __future__ import annotations
3 |
4 | import json
5 |
6 | from psycopg2 import sql
7 |
8 | from ods_pipeline.models import ALLOWED_RUN_FIELDS, TERMINAL_STATUSES
9 |
10 |
11 | def _json_or_none(value):
12 |     return json.dumps(value) if value is not None else None
13 |
14 |
15 | def _normalise(value):
16 |     if value is None:
17 |         return None
18 |     if isinstance(value, str):
19 |         return value
20 |     if hasattr(value, "isoformat"):
21 |         return value.isoformat()
22 |     return str(value)
23 |
24 |
25 | def start(
26 |     conn,
27 |     *,
28 |     run_id: str,
29 |     pipeline_type: str,
30 |     domain: str,
31 |     dataset: str,
32 |     business_date: str | None,
33 |     file_id: str | None = None,
34 |     kafka_topic: str | None = None,
35 |     config_version_id=None,
36 |     schema_version_id=None,
37 |     parents=None,
38 | ) -> None:
39 |     """Insert a new ``run_log`` row with ``status='running'``.
40 |
41 |     Duplicate ``run_id`` is allowed only when the supplied identifying
42 |     metadata matches the existing row. This preserves idempotency for retries
43 |     while surfacing accidental reuse across different pipeline types/files.
44 |     """
45 |     supplied = {
46 |         "pipeline_type": pipeline_type,
47 |         "domain": domain,
48 |         "dataset": dataset,
49 |         "business_date": str(business_date) if business_date is not None else None,
50 |         "file_id": file_id,
51 |         "kafka_topic": kafka_topic,
52 |         "config_version_id": config_version_id,
53 |         "schema_version_id": schema_version_id,
54 |         "parents": _json_or_none(parents),
55 |     }
56 |     try:
57 |         with conn.cursor() as cur:
58 |             cur.execute(
59 |                 """
60 |                 INSERT INTO pipeline.run_log
61 |                 (run_id, pipeline_type, domain, dataset, business_date,
62 |                  file_id, status, kafka_topic, config_version_id,
63 |                  schema_version_id, parents)
64 |                 VALUES (%s,%s,%s,%s,%s, %s, 'running',%s,%s,%s,%s)
65 |                 ON CONFLICT (run_id) DO NOTHING
66 |                 RETURNING run_id
67 |                 """,
68 |                 (
69 |                     run_id, pipeline_type, domain, dataset,
70 |                     supplied["business_date"],
71 |                     file_id, kafka_topic, config_version_id, schema_version_id,
72 |                     supplied["parents"],
73 |                 ),
74 |             )
75 |             inserted = cur.fetchone()
76 |             if not inserted:
77 |                 cur.execute(
78 |                     """
79 |                     SELECT pipeline_type, domain, dataset, business_date::text,
80 |                            file_id::text, kafka_topic, config_version_id,
81 |                            schema_version_id, parents::text
82 |                     FROM pipeline.run_log
83 |                     WHERE run_id=%s
84 |                     """,
85 |                     (run_id,),
86 |                 )
87 |                 row = cur.fetchone()
88 |                 if not row:
89 |                     raise RuntimeError(f"run_log conflict for {run_id} but row not found")
90 |                 existing = dict(zip(supplied.keys(), row))
91 |                 mismatches = {}
92 |                 for key, expected in supplied.items():
93 |                     if expected is None:
94 |                         continue
95 |                     if _normalise(existing.get(key)) != _normalise(expected):
96 |                         mismatches[key] = {
```

```

97 |                 "existing": existing.get(key),
98 |                 "requested": expected,
99 |             }
100 |             if mismatches:
101 |                 raise ValueError(
102 |                     f"run_id {run_id} already exists with different metadata: "
103 |                     f"{mismatches}"
104 |                 )
105 |             conn.commit()
106 |     except Exception:
107 |         conn.rollback()
108 |         raise
109 |
110 |
111 | def update(conn, run_id: str, *, commit: bool = True, **fields) -> None:
112 |     """Update arbitrary ``run_log`` fields for *run_id*.
113 |
114 |     Terminal status (succeeded / failed / partial) automatically sets
115 |     ``ended_at = COALESCE(ended_at, NOW())``.
116 |
117 |     Raises ``ValueError`` for unknown field names.
118 |
119 |     ``commit``: when True (default), the helper commits its own transaction –
120 |     behaviour preserved for all existing callers. When False, the caller is
121 |     responsible for the surrounding transaction (used by
122 |     ``ods_pipeline.messages.record_result`` for atomic multi-write flows).
123 |     """
124 |     if not fields:
125 |         return
126 |     invalid = set(fields) - ALLOWED_RUN_FIELDS
127 |     if invalid:
128 |         raise ValueError(f"Unknown run_log fields: {sorted(invalid)}")
129 |     cols = list(fields.keys())
130 |     # Defence in depth: every key MUST be a bare identifier *and* whitelisted.
131 |     # ALLOWED_RUN_FIELDS is the authoritative gate; isidentifier() is a belt-
132 |     # and-braces guard against future whitelist additions that contain unsafe
133 |     # characters by mistake.
134 |     for c in cols:
135 |         if not (isinstance(c, str) and c.isidentifier() and c in ALLOWED_RUN_FIELDS):
136 |             raise ValueError(f"Illegal run_log field name: {c!r}")
137 |     vals = [
138 |         json.dumps(v) if k == "parents" and v is not None else v
139 |         for k, v in fields.items()
140 |     ]
141 |     set_clause = sql.SQL(", ").join(
142 |         sql.SQL("{}=%s").format(sql.Identifier(c)) for c in cols
143 |     )
144 |     if fields.get("status") in TERMINAL_STATUSES:
145 |         set_clause = sql.SQL("{}").format(set_clause).format(set_clause)
146 |     statement = sql.SQL("UPDATE pipeline.run_log SET {sets} WHERE run_id=%s").format(
147 |         sets=set_clause
148 |     )
149 |     try:
150 |         with conn.cursor() as cur:
151 |             cur.execute(statement, vals + [run_id])
152 |             if commit:
153 |                 conn.commit()
154 |     except Exception:
155 |         if commit:
156 |             conn.rollback()
157 |         raise
158 |
159 |
160 | def finish(
161 |     conn,
162 |     run_id: str,
163 |     status: str,
164 |     error_summary: str | None = None,
165 | ) -> None:
166 |     """Convenience wrapper: mark a run terminal and optionally record error."""
167 |     kw: dict = {"status": status}
168 |     if error_summary is not None:
169 |         kw["error_summary"] = error_summary
170 |     update(conn, run_id, **kw)
171 |
172 |
173 | class LineageInvariantError(RuntimeError):
174 |     """Raised by ``finalise`` when run state violates the lineage contract."""
175 |
176 |
177 | def finalise(conn, run_id: str, *, commit: bool = True) -> None:
178 |     """Validate lineage closure invariants before marking a run succeeded.
179 |
180 |     Asserts:
181 |     1. If ``record_count_published > 0``, at least one ``lineage_edge`` row
182 |     exists with ``child_run_id = run_id`` (no orphan published runs).
183 |     2. No non-terminal ``run_stage_log`` rows exist for ``run_id`` – every
184 |     opened stage must have been closed.
185 |
186 |     On violation, marks the run ``failed`` with an explanatory
187 |     ``error_summary`` and raises :class:`LineageInvariantError`.
188 |
189 |     Closes architectural risk A3.
190 |     """
191 |     with conn.cursor() as cur:
192 |         cur.execute(
193 |             """
194 |             SELECT COALESCE(record_count_published, 0)
195 |             FROM pipeline.run_log
196 |             WHERE run_id = %s

```

```

197 |         """
198 |         (run_id,),
199 |     )
200 |     row = cur.fetchone()
201 |     if row is None:
202 |         raise LineageInvariantError(f"run_id {run_id} not found")
203 |     published = int(row[0] or 0)
204 |
205 |     cur.execute(
206 |         """
207 |         SELECT COUNT(*) FROM pipeline.lineage_edge
208 |         WHERE child_run_id = %s
209 |         """
210 |         (run_id,),
211 |     )
212 |     edges = int(cur.fetchone()[0])
213 |
214 |     cur.execute(
215 |         """
216 |         SELECT stage, attempt_number FROM pipeline.run_stage_log
217 |         WHERE run_id = %s
218 |         AND event_type NOT IN ('stage_completed', 'stage_failed',
219 |                                'stage_skipped', 'stage_warned')
220 |         """
221 |         (run_id,),
222 |     )
223 |     open_stages = cur.fetchall()
224 |
225 |     failures: list[str] = []
226 |     if published > 0 and edges == 0:
227 |         failures.append(
228 |             f"published={published} but no lineage_edge rows; orphaned run"
229 |         )
230 |     if open_stages:
231 |         names = ", ".join(f"{s}#{a}" for s, a in open_stages)
232 |         failures.append(f"non-terminal stages remain: {names}")
233 |
234 |     if failures:
235 |         summary = "lineage invariant violated: " + " | ".join(failures)
236 |         update(conn, run_id, status="failed",
237 |                error_summary=summary, commit=commit)
238 |         raise LineageInvariantError(summary)

```


36. ods_pipeline/stages.py

437 lines

```
1 | """pipeline.run_stage_log operations.
2 |
3 | Stateless control-plane contract
4 | -----
5 |
6 | Every helper here defaults to ``commit=True`` – each stage row is its own
7 | durable checkpoint. Long pipelines therefore expose live progress on the
8 | operator dashboard and survive worker death without leaving an open
9 | transaction holding row locks.
10 |
11 | The trade-off is that the application code must explicitly write a
12 | ``stage_failed`` row in its exception handler. The :func:`stage_scope`
13 | context manager below packages that discipline so callers can't forget:
14 |
15 |     >>> with stage_scope(conn, run_id=run_id, stage=Stage.MESSAGE_RECEIVE,
16 |         ...               record_count_in=1):
17 |         ...     do_work()           # success → stage_completed
18 |         ...                         # raise   → stage_failed
19 |
20 | If even the failure write fails (DB flapping, connection dead) the
21 | context manager swallows the secondary error so the original work
22 | exception still propagates; a heartbeat-staleness janitor closes any
23 | ``status='running'`` rows it leaves behind.
24 | """
25 | from __future__ import annotations
26 |
27 | import json
28 | from contextlib import contextmanager
29 |
30 | from ods_pipeline.models import StageEvent
31 |
32 |
33 | def write(
34 |     conn,
35 |     *,
36 |     run_id: str,
37 |     stage: str,
38 |     status: str,
39 |     event_type: str | None = None,
40 |     attempt_number: int = 1,
41 |     input_ref: str | None = None,
42 |     output_ref: str | None = None,
43 |     record_count_in: int | None = None,
44 |     record_count_out: int | None = None,
45 |     metrics: dict | None = None,
46 |     error: str | None = None,
47 |     airflow_dag_id: str | None = None,
48 |     airflow_run_id: str | None = None,
49 |     spark_app_id: str | None = None,
50 |     commit: bool = True,
51 | ) -> None:
52 |     """Append one row to ``pipeline.run_stage_log``.
53 |
54 |     ``started_at`` is always set to ``NOW()``.
55 |     ``ended_at`` is set to ``NOW()`` only for terminal events
56 |     (completed / failed / skipped / warned). For ``stage_started`` /
57 |     ``status='running'`` it is left NULL so the open interval is
58 |     queryable.
59 |
60 |     Idempotency (B4, migration 19): inserting a ``stage_started`` row that
61 |     duplicates an existing open attempt is a no-op (partial unique index
62 |     ``run_stage_log_started_unique``). Terminal events remain append-only.
63 |
64 |     ``commit``: when True (default), the helper commits its own transaction.
65 |     When False, the caller owns the surrounding tx (used by atomic
66 |     ``record_result`` flow).
67 |     """
68 |     is_open = (
69 |         event_type == StageEvent.STARTED
70 |         or (event_type is None and status == "running")
71 |     )
72 |     ended_at_sql = "NULL" if is_open else "NOW()"
73 |     # Only the started-event path participates in the partial unique index
74 |     # added by migration 19. Terminal events are append-only.
75 |     on_conflict_sql = (
76 |         "ON CONFLICT (run_id, stage, attempt_number) WHERE event_type = 'stage_started' DO NOTHING"
77 |         if is_open
78 |         else ""
79 |     )
80 |
81 |     try:
82 |         with conn.cursor() as cur:
83 |             cur.execute(
84 |                 f"""
85 |                 INSERT INTO pipeline.run_stage_log
86 |                     (run_id, stage, status, event_type, attempt_number,
87 |                      started_at, ended_at,
88 |                      input_ref, output_ref,
89 |                      record_count_in, record_count_out,
90 |                      metrics, error,
91 |                      airflow_dag_id, airflow_run_id, spark_app_id)
92 |                 VALUES (%s,%s,%s,%s,%s, NOW(), {ended_at_sql},
93 |                         %s,%s, %s,%s, %s,%s, %s,%s,%s)
94 |                 {on_conflict_sql}
95 |                 """
96 |                 (
```

```

97 |         run_id, stage, status, event_type, attempt_number,
98 |         input_ref, output_ref,
99 |         record_count_in, record_count_out,
100 |         json.dumps(metrics) if metrics else None, error,
101 |         airflow_dag_id, airflow_run_id, spark_app_id,
102 |     ),
103 | )
104 |     if commit:
105 |         conn.commit()
106 | except Exception:
107 |     if commit:
108 |         conn.rollback()
109 |     raise
110 |
111 |
112 | def next_attempt_number(conn, *, run_id: str, stage: str) -> int:
113 |     """Return the next free ``attempt_number`` for ``(run_id, stage)``.
114 |
115 |     Stateless control-plane retries leave durable stage rows from the
116 |     failed attempts; the unique index on ``(run_id, stage, event_type,
117 |     attempt_number)`` requires the new attempt to use ``MAX+1`` rather
118 |     than always 1.
119 |     """
120 |     with conn.cursor() as cur:
121 |         cur.execute(
122 |             """
123 |             SELECT COALESCE(MAX(attempt_number), 0) + 1
124 |             FROM pipeline.run_stage_log
125 |             WHERE run_id=%s AND stage=%s
126 |             """,
127 |             (run_id, stage),
128 |         )
129 |         row = cur.fetchone()
130 |         return int(row[0]) if row and row[0] is not None else 1
131 |
132 |
133 | def start(
134 |     conn,
135 |     *,
136 |     run_id: str,
137 |     stage: str,
138 |     attempt_number: int | None = None,
139 |     **kwargs,
140 | ) -> int:
141 |     """Open a stage attempt with ``status='running'`` and ``ended_at=NULL``.
142 |
143 |     When ``attempt_number`` is omitted, :func:`next_attempt_number` is
144 |     consulted so retries land on a fresh row instead of colliding on the
145 |     unique index. Returns the attempt number used so the caller can pass
146 |     it to a later :func:`finish` for the same attempt.
147 |     """
148 |     if attempt_number is None:
149 |         attempt_number = next_attempt_number(conn, run_id=run_id, stage=stage)
150 |     write(
151 |         conn,
152 |         run_id=run_id,
153 |         stage=stage,
154 |         status="running",
155 |         event_type=StageEvent.STARTED,
156 |         attempt_number=attempt_number,
157 |         **kwargs,
158 |     )
159 |     return attempt_number
160 |
161 |
162 | def finish(
163 |     conn,
164 |     *,
165 |     run_id: str,
166 |     stage: str,
167 |     status: str,
168 |     event_type: str | None = None,
169 |     attempt_number: int = 1,
170 |     input_ref: str | None = None,
171 |     output_ref: str | None = None,
172 |     record_count_in: int | None = None,
173 |     record_count_out: int | None = None,
174 |     metrics: dict | None = None,
175 |     error: str | None = None,
176 |     airflow_dag_id: str | None = None,
177 |     airflow_run_id: str | None = None,
178 |     spark_app_id: str | None = None,
179 |     commit: bool = True,
180 | ) -> None:
181 |     """Close the latest open stage row, falling back to append if none exists.
182 |
183 |     Concurrency contract (B3): the open-row claim uses
184 |     ``SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1`` and the subsequent
185 |     UPDATE-or-INSERT runs in the same transaction (single ``conn.commit()``
186 |     at the end). Two concurrent ``finish`` calls for the same
187 |     ``(run_id, stage, attempt_number)`` therefore resolve as: the lock
188 |     holder UPDATES the open row, the loser sees no claimable row and
189 |     INSERTS a fresh terminal row – never a silent no-op or double-update.
190 |     """
191 |     terminal_event = event_type or (
192 |         StageEvent.FAILED if status == "failed" else StageEvent.COMPLETED
193 |     )
194 |     metrics_json = json.dumps(metrics) if metrics else None
195 |     try:
196 |         with conn.cursor() as cur:

```

```

197 |         cur.execute(
198 |             """
199 |             SELECT id
200 |             FROM pipeline.run_stage_log
201 |             WHERE run_id=%s
202 |             AND stage=%s
203 |             AND attempt_number=%s
204 |             AND status='running'
205 |             AND ended_at IS NULL
206 |             ORDER BY started_at DESC, id DESC
207 |             LIMIT 1
208 |             FOR UPDATE SKIP LOCKED
209 |             """,
210 |             (run_id, stage, attempt_number),
211 |         )
212 |         row = cur.fetchone()
213 |         if row is not None:
214 |             cur.execute(
215 |                 """
216 |                 UPDATE pipeline.run_stage_log
217 |                 SET status=%s,
218 |                     event_type=%s,
219 |                     ended_at=NOW(),
220 |                     input_ref=COALESCE(%s, input_ref),
221 |                     output_ref=COALESCE(%s, output_ref),
222 |                     record_count_in=COALESCE(%s, record_count_in),
223 |                     record_count_out=COALESCE(%s, record_count_out),
224 |                     metrics=COALESCE(%s, metrics),
225 |                     error=COALESCE(%s, error),
226 |                     airflow_dag_id=COALESCE(%s, airflow_dag_id),
227 |                     airflow_run_id=COALESCE(%s, airflow_run_id),
228 |                     spark_app_id=COALESCE(%s, spark_app_id)
229 |                 WHERE id=%s
230 |                 """,
231 |                 (
232 |                     status, terminal_event,
233 |                     input_ref, output_ref,
234 |                     record_count_in, record_count_out,
235 |                     metrics_json, error,
236 |                     airflow_dag_id, airflow_run_id, spark_app_id,
237 |                     row[0],
238 |                 ),
239 |             )
240 |         else:
241 |             # Inline append kept inside the same tx so finish() is atomic.
242 |             # Mirrors the column set in write(); ended_at=NOW() because the
243 |             # event is terminal.
244 |             cur.execute(
245 |                 """
246 |                 INSERT INTO pipeline.run_stage_log
247 |                 (run_id, stage, status, event_type, attempt_number,
248 |                  started_at, ended_at,
249 |                  input_ref, output_ref,
250 |                  record_count_in, record_count_out,
251 |                  metrics, error,
252 |                  airflow_dag_id, airflow_run_id, spark_app_id)
253 |                 VALUES (%s,%s,%s,%s,%s, NOW(), NOW(),
254 |                          %s,%s, %s,%s, %s,%s, %s,%s,%s)
255 |                 """,
256 |                 (
257 |                     run_id, stage, status, terminal_event, attempt_number,
258 |                     input_ref, output_ref,
259 |                     record_count_in, record_count_out,
260 |                     metrics_json, error,
261 |                     airflow_dag_id, airflow_run_id, spark_app_id,
262 |                 ),
263 |             )
264 |             if commit:
265 |                 conn.commit()
266 |         except Exception:
267 |             if commit:
268 |                 conn.rollback()
269 |             raise
270 |
271 |
272 | @contextmanager
273 | def stage_scope(
274 |     conn,
275 |     *,
276 |     run_id: str,
277 |     stage: str,
278 |     record_count_in: int | None = None,
279 |     input_ref: str | None = None,
280 |     metrics: dict | None = None,
281 |     airflow_dag_id: str | None = None,
282 |     airflow_run_id: str | None = None,
283 |     spark_app_id: str | None = None,
284 |     truncate_error: int = 500,
285 | ):
286 |     """Context manager that opens a stage on entry and closes it on exit.
287 |
288 |     Stateless-control-plane contract:
289 |
290 |     * On entry: writes ``stage_started`` (with auto-incremented
291 |       ``attempt_number``) and commits. Live on the dashboard immediately.
292 |     * On clean exit: writes ``stage_completed``. The caller may yield a
293 |       result dict via ``ctx.set_result(...)`` – its keys merge into the
294 |       finish call so ``output_ref``, ``record_count_out``, etc. land on
295 |       the same row.
296 |     * On ``s.skip(reason)``: clean exit, but writes ``stage_skipped``

```

```

297 |         instead of ``stage_completed``. Use when a stage legitimately ran
298 |         but had nothing to do (api_pull saw no changes, file_pipeline saw
299 |         an empty file). ``reason`` is folded into the row's ``metrics`` so
300 |         the dashboard can surface why.
301 |     * On exception: writes ``stage_failed`` with the exception message
302 |       (truncated to ``truncate_error`` chars) and re-raises the original.
303 |       If the failure write itself fails, swallow that secondary error –
304 |       the heartbeat janitor will close the row.
305 |
306 | Usage:
307 |
308 |     with stage_scope(conn, run_id=rid, stage=Stage.MESSAGE_RECEIVE) as s:
309 |         count = do_work()
310 |         if count == 0:
311 |             s.skip("no_changes")
312 |         else:
313 |             s.set_result(record_count_out=count, output_ref="s3://...")
314 |
315 | """
316 | attempt = start(
317 |     conn,
318 |     run_id=run_id,
319 |     stage=stage,
320 |     record_count_in=record_count_in,
321 |     input_ref=input_ref,
322 |     metrics=metrics,
323 |     airflow_dag_id=airflow_dag_id,
324 |     airflow_run_id=airflow_run_id,
325 |     spark_app_id=spark_app_id,
326 | )
327 | box: dict = {
328 |     "record_count_out": None,
329 |     "output_ref": None,
330 |     "metrics": None,
331 | }
332 | state: dict = {
333 |     "outcome": "succeeded", # succeeded | skipped | warned
334 |     "reason": None,         # narrative for skipped / warned
335 | }
336 |
337 | class _Scope:
338 |     attempt_number = attempt
339 |
340 |     def set_result(self, **fields) -> None:
341 |         for key, value in fields.items():
342 |             if key not in box:
343 |                 raise KeyError(
344 |                     f"stage_scope only accepts {sorted(box)}; got {key!r}"
345 |                 )
346 |             box[key] = value
347 |
348 |     def skip(self, reason: str | None = None) -> None:
349 |         """Mark the stage as skipped on clean exit.
350 |
351 |         Idempotent: a second ``skip()`` overwrites the reason. A
352 |         subsequent exception still wins (skip is for clean exits
353 |         only).
354 |         """
355 |         state["outcome"] = "skipped"
356 |         state["reason"] = reason
357 |
358 |     def warn(self, reason: str | None = None) -> None:
359 |         """Mark the stage as warned on clean exit.
360 |
361 |         Use when work completed but produced soft-failure signals
362 |         (DQ rules fired, partial DLQ writes, schema drift below the
363 |         blocking threshold). Distinct from ``skip`` (which means
364 |         "ran but had nothing to do") and ``raise`` (which means
365 |         "failed hard"). ``reason`` lands on the stage row's
366 |         ``error`` column so the dashboard can surface it next to
367 |         other failure signals.
368 |         """
369 |         state["outcome"] = "warned"
370 |         state["reason"] = reason
371 |
372 |     try:
373 |         yield _Scope()
374 |     except BaseException as exc:
375 |         # BaseException catches SystemExit / KeyboardInterrupt – those
376 |         # also leave the stage row stuck if we don't close it.
377 |         try:
378 |             finish(
379 |                 conn,
380 |                 run_id=run_id,
381 |                 stage=stage,
382 |                 status="failed",
383 |                 event_type=StageEvent.FAILED,
384 |                 attempt_number=attempt,
385 |                 record_count_in=record_count_in,
386 |                 record_count_out=box["record_count_out"],
387 |                 output_ref=box["output_ref"],
388 |                 metrics=box["metrics"],
389 |                 error=str(exc)[:truncate_error] if str(exc) else type(exc).__name__,
390 |                 airflow_dag_id=airflow_dag_id,
391 |                 airflow_run_id=airflow_run_id,
392 |                 spark_app_id=spark_app_id,
393 |             )
394 |         except Exception:
395 |             # Last-ditch – heartbeat janitor catches any orphaned row.
396 |             try:
397 |                 conn.rollback()

```

```

397 |         except Exception:
398 |             pass
399 |         raise
400 |     else:
401 |         # Map clean-exit outcomes to status / event_type. ``skip`` /
402 |         # ``warn`` reasons fold into the appropriate column (metrics
403 |         # for skip – non-failure signal; error for warn – failure
404 |         # signal that wasn't blocking).
405 |         finish_metrics = box["metrics"]
406 |         finish_error: str | None = None
407 |         outcome = state["outcome"]
408 |         if outcome == "skipped":
409 |             if state["reason"] is not None:
410 |                 finish_metrics = dict(finish_metrics or {})
411 |                 finish_metrics.setdefault("skip_reason", state["reason"])
412 |                 finish_status = "skipped"
413 |                 finish_event = StageEvent.SKIPPED
414 |             elif outcome == "warned":
415 |                 finish_status = "warned"
416 |                 finish_event = StageEvent.WARNED
417 |                 if state["reason"] is not None:
418 |                     finish_error = state["reason"]
419 |             else:
420 |                 finish_status = "succeeded"
421 |                 finish_event = StageEvent.COMPLETED
422 |         finish(
423 |             conn,
424 |             run_id=run_id,
425 |             stage=stage,
426 |             status=finish_status,
427 |             event_type=finish_event,
428 |             attempt_number=attempt,
429 |             record_count_in=record_count_in,
430 |             record_count_out=box["record_count_out"],
431 |             output_ref=box["output_ref"],
432 |             metrics=finish_metrics,
433 |             error=finish_error,
434 |             airflow_dag_id=airflow_dag_id,
435 |             airflow_run_id=airflow_run_id,
436 |             spark_app_id=spark_app_id,
437 |         )

```